Claude Code Complete Guide: Hooks, Agents, MCP Tools & Professional Development

Transform your AI assistant from a simple command-line tool into a powerful, programmable engineering platform. This comprehensive guide covers everything you need to master Claude Code - from basic automation to advanced multi-agent systems.

Why This Guide?

While hooks were the starting point, Claude Code has evolved into a complete development ecosystem. This guide covers:

- **Hooks**: Automated triggers and quality gates
- **Agents & Subagents**: Specialized AI team members
- **MCP Tools**: Powerful integrations and servers
- **Professional Workflows**: Enterprise-ready development patterns
- **Custom Commands**: Reusable workflow automation
- **And much more...**

Whether you're automating simple tasks or building complex multi-agent systems, this guide has you covered.

Table of Contents

---

1. Introduction: What are Claude Code Hooks? {#introduction}

What They Are

Claude Code hooks are user-defined shell commands or scripts that run automatically at specific moments in Claude Code's operational lifecycle. Think of them as programmable triggers that allow you to inject custom logic directly into Claude's workflow.

The Professional Development Challenge

While Claude Code excels at generating code quickly, it often jumps straight into implementation, skipping critical professional development steps like:

- System architecture planning
- Security protocol design
- UI/UX design considerations
- Test strategy development
- Performance planning

This approach works for small scripts but falls apart for enterprise-level applications. Hooks provide the solution by adding structure, process, and professional guardrails to Claude's workflow.

Why They're Game-Changing

Hooks provide three critical capabilities essential for modern AI-driven development:

🔍 Observability & Logging

- Create detailed audit trails of every action Claude takes
- Debug long tasks by reviewing step-by-step logs
- Analyze agent behavior to improve your prompts
- Understand tool usage patterns and sequences

🛡️ Control & Safety

- Block dangerous commands (e.g., `rm -rf *`)
- Protect sensitive files (`.env`, config files)

- Enforce workflows (run linters after edits)
- Add deterministic guardrails to non-deterministic AI

⚡ **Automation & Personalization**

- Get voice alerts when tasks complete
- Add custom sounds for different actions
- Automate repetitive development tasks
- Create a personalized coding experience
- **Enforce TDD**: Automatically prompt for tests before implementation

---

2. Hook Configuration Basics {#configuration}

File Location

Create your hooks configuration at `.claude/settings.json` in your project's root directory. This makes hooks shareable and version-controlled.

Basic Structure

JSON

```
{
  "hooks": {
    "EventName": [
      {
        "matcher": "ToolPattern",
        "hooks": [
          {
            "type": "command",
            "command": "your_shell_command_or_script_here"
          }
        ]
      }
    ]
  }
}
```

Configuration Elements

- **`EventName`**: The lifecycle event to hook into (e.g., `PreToolUse`, `Stop`)
- **`matcher`**: Pattern to match against tools
  - `""` matches all tools
  - `"Bash"` or `"Edit"` matches specific tools
  - Regular expressions supported for complex matching
- **`command`**: Shell command to execute (can call scripts)

---

3. Hook Events and Payloads {#events}

Each hook receives a JSON payload via stdin containing event context. Your scripts can parse this data to make intelligent decisions.

Available Events

Event	When It Fires	Common Use Cases	Key Payload Fields
`PreToolUse`	Before any tool executes	Blocking dangerous commands, validation, logging	`hook_event_name`, `tool_name`, `tool_input`
`PostToolUse`	After tool completes	Logging results, running formatters, follow-up actions	`hook_event_name`, `tool_name`, `tool_input`, `tool_response`
`Notification`	When Claude needs input	Custom alerts, desktop notifications	`hook_event_name`, `message`
`Stop`	When main agent finishes	Task completion alerts, session logging	`hook_event_name`, `transcript_path`
`SubagentStop`	When subagent finishes	Multi-step task notifications	`hook_event_name`, `stop_hook_active`

---

4. Practical Examples {#examples}

Example 1: Simple Sound Notifications

Goal: Play a sound when Claude needs your input.

Configuration (`.claude/settings.json`):

```json
{
  "hooks": {
    "Notification": [
      {
        "matcher": "",
        "hooks": [
          {
            "type": "command",
            "command": "afplay /System/Library/Sounds/Glass.aiff"
          }
        ]
      }
    ]
  }
}
```

Note: Use `afplay` on macOS, `aplay` on Linux

Claude Prompt:

> "I want to create a Claude Code hook that plays a sound whenever a 'Notification' event occurs. I'm on macOS. Please write the settings.json configuration for this."

Example 2: Intelligent Voice Alerts

Goal: Have Claude speak status updates using text-to-speech.

Step 1: Install dependency

```shell
pip install pyttsx3
```

Step 2: Create script (`.claude/hooks/announcer.py`):

```python
#!/usr/bin/env python3
import sys
import json
import pyttsx3

def main():
    try:
        # Read event data from stdin
        input_data = json.load(sys.stdin)
        event_name = input_data.get("hook_event_name", "Unknown
event")

        message = ""
        if event_name == "Stop":
            message = "Main task complete. Ready for the next
step."
        elif event_name == "SubagentStop":
            message = "Sub-task finished."
        elif event_name == "Notification":
            message = "Your agent needs your input."

        if message:
            engine = pyttsx3.init()
            engine.say(message)
            engine.runAndWait()
```

```
    except Exception:
        # Fail silently to avoid interrupting Claude
        pass

if __name__ == "__main__":
    main()
```

Step 3: Configure hooks:

```JSON
{
  "hooks": {
    "Stop": [{
      "matcher": "",
      "hooks": [{"type": "command", "command": "python3
.claude/hooks/announcer.py"}]
    }],
    "SubagentStop": [{
      "matcher": "",
      "hooks": [{"type": "command", "command": "python3
.claude/hooks/announcer.py"}]
    }],
    "Notification": [{
      "matcher": "",
      "hooks": [{"type": "command", "command": "python3
.claude/hooks/announcer.py"}]
    }]
  }
}
```

Claude Prompt:

> "Write a Python script for a Claude Code hook that uses pyttsx3 to speak different messages for 'Stop', 'SubagentStop', and 'Notification' events. Include the settings.json configuration."

Example 3: Safety Guardrail - Block Dangerous Commands

Goal: Prevent Claude from using `rm -rf` commands.

Step 1: Create safety script (`.claude/hooks/safety_check.py`):

Python

```
#!/usr/bin/env python3
import sys
import json
import re

def is_dangerous_rm(command):
    # Regex to catch variations of rm -rf, rm -fr, etc.
    patterns = [
        r'\brm\s+-[^\s]*r[^\s]*f',
        r'\brm\s+-[^\s]*f[^\s]*r',
    ]
    for pattern in patterns:
        if re.search(pattern, command):
            return True
    return False

def main():
    input_data = json.load(sys.stdin)
    tool_name = input_data.get("tool_name")

    # Only check Bash commands
    if tool_name == "Bash":
        command = input_data.get("tool_input", {}).get("command",
"")
        if is_dangerous_rm(command):
            print("BLOCKED: Dangerous rm command detected and
prevented.", file=sys.stderr)
            # Exit with code 2 to block the tool and show error
to Claude
            sys.exit(2)
```

```
if __name__ == "__main__":
    main()
```

Step 2: Configure the hook:

JSON

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "python3 .claude/hooks/safety_check.py"
          }
        ]
      }
    ]
  }
}
```

Example 4: Enforce Test-Driven Development

Goal: Remind Claude to write tests before implementation.

Step 1: Create TDD enforcement hook (`.claude/hooks/tdd_check.py`):

Python

```
#!/usr/bin/env python3
import json
import sys
import os
import re
```

```
def should_have_test(file_path):
    # Check if this is a source file that should have tests
    patterns = [r'\.py

---

## 5. Best Practices and Debugging {#best-practices}

### 🔍 Debug First
Always start by logging the JSON payload to understand the data
structure:
```json
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "jq '.' >> .claude/hooks/log.jsonl"
      }]
    }]
  }
}
```

📁 Project Organization

- Keep `.claude/settings.json` in your project repository
- Store scripts in `.claude/hooks/` directory
- Make hooks shareable and version-controlled

🛡️ Error Handling

- Hooks should fail silently unless designed to block actions
- Exit with code 0 for normal operation
- Exit with code 2 to block an action and show an error
```

📦 Dependency Management

For Python scripts with dependencies, use **Astral's uv**:

```shell
uv run my_script.py
```

This handles environment setup automatically.

🎯 Key Principles

1. **Start simple**: Test with basic logging before complex logic
2. **Be defensive**: Handle missing data and errors gracefully
3. **Keep it fast**: Hooks run synchronously - avoid slow operations
4. **Document well**: Comment your scripts for future reference

💡 Pro Tips from the Community

1. **Session Management**: Instead of clearing sessions, start a new one. You can resume previous sessions with `claude --resume` for better context preservation
2. **Use Slash Commands**: Rather than copying and pasting prompts (like security checks), create custom slash commands for reusable workflows
3. **Multi-Tool Learning**: Use Claude Code alongside other tools like Cursor IDE - run Claude Code for tasks while using Cursor's AI to understand the code being generated
4. **Learning Workflow**: Create a learning loop by analyzing Claude's output in other AI tools to deepen your understanding of generated code

---

6. Advanced Hook Examples {#advanced-examples}

Multi-Tool Pattern Matching

Goal: Apply different actions based on file types across multiple tools.

JSON

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|MultiEdit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "if [[ \"$CLAUDE_FILE_PATHS\" =~ \\.(py)$
]]; then black $CLAUDE_FILE_PATHS && ruff check --fix
$CLAUDE_FILE_PATHS; fi"
          },
          {
            "type": "command",
            "command": "if [[ \"$CLAUDE_FILE_PATHS\" =~
\\.(ts|tsx)$ ]]; then prettier --write $CLAUDE_FILE_PATHS && npx
tsc --noEmit $CLAUDE_FILE_PATHS; fi"
          }
        ]
      }
    ]
  }
}
```

Session Context Loading

Goal: Automatically load project context when Claude Code starts.

Tip: Instead of clearing sessions when things get messy, start a new session and use `claude --resume` to restore previous context when needed. This preserves your conversation history for reference.

Script (`.claude/hooks/session_start.py`):

Python

```
#!/usr/bin/env python3
import json
import sys
import subprocess

def load_context():
    context = []

    # Git status
    try:
        git_status = subprocess.check_output(['git', 'status',
'--short'], text=True)
        if git_status:
            context.append(f"Current git changes:\n{git_status}")
    except:
        pass

    # Recent commits
    try:
        commits = subprocess.check_output(['git', 'log',
'--oneline', '-5'], text=True)
        context.append(f"Recent commits:\n{commits}")
    except:
        pass

    # TODO items
    try:
        todos = subprocess.check_output(['grep', '-r', 'TODO:',
'.', '--include=*.py'], text=True)
        if todos:
            context.append(f"TODO items
found:\n{todos[:500]}...")
    except:
        pass
```

```
    # Output context for Claude
    if context:
        print("\n=== PROJECT CONTEXT ===")
        print("\n".join(context))
        print("========================\n")

if __name__ == "__main__":
    load_context()
```

Configuration:

```JSON
{
  "hooks": {
    "SessionStart": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "python3 .claude/hooks/session_start.py"
      }]
    }]
  }
}
```

Advanced Command Validation with AI Feedback

Goal: Use AI to analyze potentially dangerous commands before execution.

```Python
#!/usr/bin/env python3
import json
import sys
import re
```

```
DANGEROUS_PATTERNS = [
    (r'rm\s+-[^\s]*[rf]', "Recursive deletion detected"),
    (r'chmod\s+777', "Overly permissive file permissions"),
    (r'curl.*\|\s*sh', "Piping curl to shell - potential security
risk"),
    (r'npm\s+install.*--global', "Global npm install detected"),
    (r'docker.*--privileged', "Privileged Docker container"),
]

def analyze_command(command):
    risks = []
    for pattern, message in DANGEROUS_PATTERNS:
        if re.search(pattern, command, re.IGNORECASE):
            risks.append(message)

    if risks:
        # Exit code 2 blocks execution and feeds stderr to Claude
        print(json.dumps({
            "error": "Command blocked due to security concerns",
            "risks": risks,
            "command": command,
            "suggestion": "Please use a safer alternative or
confirm this is intentional"
        }), file=sys.stderr)
        sys.exit(2)

if __name__ == "__main__":
    data = json.load(sys.stdin)
    if data.get("tool_name") == "Bash":
        command = data.get("tool_input", {}).get("command", "")
        analyze_command(command)
```

---

7. Troubleshooting Common Issues {#troubleshooting}

Hook Not Executing

Problem: Your hooks aren't firing when expected.

Solutions:

1. **Check file permissions**:

```
Shell
chmod +x .claude/hooks/*.py
```

2. **Verify JSON syntax**:

```
Shell
jq '.' .claude/settings.json
```

3. **Use absolute paths** (recommended by documentation):

```
JSON
{
  "command":
"/absolute/path/to/your/project/.claude/hooks/script.py"
}
```

4. **Debug with logging**:

```
JSON
{
```

```
  "command": "echo 'Hook fired at $(date)' >>
/tmp/claude-hook-debug.log"
}
```

Settings Not Updating

Problem: Changes to settings.json don't take effect.

Solutions:

1. **Restart Claude Code** - Settings are loaded at startup
2. **Check for syntax errors** in your JSON
3. **Ensure you're editing the correct file**:
    - Project-specific: `.claude/settings.json`
    - User-global: `~/.claude/settings.json`

Hook Blocking Unintentionally

Problem: Hook exits with non-zero code accidentally.

Solution: Always handle errors gracefully:

```python
try:
    # Your hook logic
    pass
except Exception as e:
    # Log error but don't block Claude
    with open('/tmp/hook-errors.log', 'a') as f:
        f.write(f"Error: {e}\n")
    sys.exit(0)  # Exit successfully
```

Performance Issues

Problem: Hooks slow down Claude's operations.

Solutions:

1. **Use background execution** for non-critical hooks:

```json
{
  "type": "command",
  "command": "your_command &"
}
```

2. **Set appropriate timeouts**:

```json
{
  "type": "command",
  "command": "your_command",
  "timeout": 5
}
```

3. **Optimize scripts** - avoid heavy operations in synchronous hooks

---

8. Quick Reference Cheat Sheet {#cheat-sheet}

Hook Events

Event	Purpose	Can Block?	Key Data
`UserPromptSubmit`	Modify user prompts	No	`prompt`
`SessionStart`	Load context	No	`session_id`, `source`
`PreToolUse`	Validate/block tools	Yes (exit 2)	`tool_name`, `tool_input`
`PostToolUse`	Process results	No	`tool_response`
`Notification`	Custom alerts	No	`message`
`Stop`	Task completion	No	`transcript_path`
`SubagentStop`	Subtask completion	Yes (exit 2)	`stop_hook_active`
`PreCompact`	Before compaction	No	`trigger`

Exit Codes

- **0**: Success (stdout shown in transcript mode)
- **2**: Block operation (stderr sent to Claude)
- **Other**: Non-blocking error (stderr shown to user)

Environment Variables

```shell
$CLAUDE_PROJECT_DIR    # Absolute path to project root
$CLAUDE_FILE_PATHS     # Space-separated file paths
$CLAUDE_TOOL_NAME      # Name of the tool being used
$CLAUDE_NOTIFICATION   # Notification message content
```

Common Patterns

```JSON
// Match all tools
"matcher": ""

// Match specific tools
"matcher": "Bash"

// Match multiple tools
"matcher": "Edit|Write|MultiEdit"

// Match MCP tools
"matcher": "mcp__github__*"
```

Session Management

- **Start fresh**: Use new sessions instead of clearing when context gets messy
- **Resume sessions**: `claude --resume` to restore previous context
- **Headless mode**: `claude -p "prompt" --output-format stream-json` for automation
- **Non-interactive**: `claude -p "prompt" --no-interactive` for CI/CD

Agent Commands (New!)

- **Create agents**: `/agents` or "Create a [type] specialist"
- **List agents**: `/agents list`
- **Edit agents**: `/agents edit [name]`
- **Delete agents**: `/agents delete [name]`
- **Automatic delegation**: Claude auto-selects appropriate agents
- **Explicit use**: "Use the [agent-name] to..."

---

9. Integration with Development Workflows {#integrations}

Git Workflow Integration

Pre-commit Hook Integration:

```JSON
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "git add $CLAUDE_FILE_PATHS &&
./.git/hooks/pre-commit --files $CLAUDE_FILE_PATHS || true"
      }]
    }]
  }
}
```

CI/CD Integration

Headless Mode for Automation:

```Shell
# Run Claude Code in CI pipeline
claude -p "Fix all linting errors in src/" --output-format
stream-json

# Pre-push validation
claude -p "Run all tests and fix any failures" --no-interactive
```

IDE Integration

VSCode Tasks Integration:

```json
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit",
      "hooks": [{
        "type": "command",
        "command": "code --goto $CLAUDE_FILE_PATHS:1"
      }]
    }]
  }
}
```

Docker Development

Container Management:

```json
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "python3 -c \"import json,sys;
d=json.load(sys.stdin);
cmd=d.get('tool_input',{}).get('command',''); sys.exit(2 if
'docker' in cmd and '--rm' not in cmd else 0)\""
      }]
    }]
  }
}
```

---

10. Security Considerations {#security}

Key Security Principles

1. **Hooks run with your credentials** - They have full access to your system
2. **Review all hook code** before enabling it
3. **Never run untrusted hooks** from unknown sources
4. **Use restrictive permissions** on hook scripts
5. **Validate all inputs** in your hook scripts

Secure Hook Template

Python

```python
#!/usr/bin/env python3
import json
import sys
import os
import re

# Whitelist approach for allowed operations
ALLOWED_COMMANDS = [
    r'^ls\s',
    r'^cat\s',
    r'^grep\s',
    r'^find\s.*-name',
]

FORBIDDEN_PATHS = [
    '.env',
    '.ssh/',
    'credentials',
    'secrets',
    '.git/config',
]

def validate_command(command):
    # Check against whitelist
    if not any(re.match(pattern, command) for pattern in
ALLOWED_COMMANDS):
```

```
        return False, "Command not in allowlist"

    # Check for forbidden paths
    for path in FORBIDDEN_PATHS:
        if path in command:
            return False, f"Access to {path} is forbidden"

    return True, ""

def main():
    try:
        data = json.load(sys.stdin)
        if data.get("tool_name") == "Bash":
            command = data.get("tool_input", {}).get("command",
"")
            valid, reason = validate_command(command)

            if not valid:
                print(f"Security block: {reason}",
file=sys.stderr)
                sys.exit(2)
    except Exception as e:
        # Fail open - don't block on errors
        pass

if __name__ == "__main__":
    main()
```

---

11. Multi-Agent Observability {#observability}

Setting Up Observability Dashboard

Goal: Monitor multiple Claude Code agents in real-time.

Architecture:

```
None
Claude Agents → Hooks → HTTP Server → SQLite → WebSocket →
Dashboard
```

Hook Configuration:

```
JSON
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "python3 .claude/hooks/send_event.py --event
PreToolUse --agent $USER"
      }]
    }],
    "PostToolUse": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "python3 .claude/hooks/send_event.py --event
PostToolUse --agent $USER --include-output"
      }]
    }]
  }
}
```

Event Sender Script:

Python

```
#!/usr/bin/env python3
import json
import sys
import requests
import argparse
from datetime import datetime

def send_event(event_type, agent_name, include_output=False):
    data = json.load(sys.stdin)

    event = {
        "timestamp": datetime.utcnow().isoformat(),
        "agent": agent_name,
        "event_type": event_type,
        "tool_name": data.get("tool_name"),
        "tool_input": data.get("tool_input"),
    }

    if include_output:
        event["tool_output"] = data.get("tool_response",
{}).get("output", "")[:500]

    try:
        requests.post("http://localhost:8080/events", json=event,
timeout=1)
    except:
        pass  # Don't block Claude if observability is down

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--event", required=True)
    parser.add_argument("--agent", required=True)
    parser.add_argument("--include-output", action="store_true")
    args = parser.parse_args()

    send_event(args.event, args.agent, args.include_output)
```

---

12. Custom Slash Commands {#slash-commands}

Creating Custom Commands

Custom slash commands extend Claude Code's functionality through natural language templates. **Pro tip**: Instead of repeatedly copying and pasting prompts (like security checks or review templates), convert them into slash commands for instant reuse.

Location: `.claude/commands/` (project) or `~/.claude/commands/` (global)

Example: `.claude/commands/review-pr.md`

```
None
Please review the changes in the current git branch:

1. First, check what branch we're on with `git branch
--show-current`
2. Show the diff with `git diff main...HEAD`
3. Analyze the changes for:
   - Potential bugs or logic errors
   - Security vulnerabilities
   - Performance issues
   - Missing error handling
4. Suggest improvements with specific code examples
5. Check if tests are needed for the changes

Focus on actual issues, not style preferences.
```

Example: Convert Frequent Prompts to Commands:
`.claude/commands/security-check.md`

```
None
Perform a comprehensive security audit on the codebase:

1. Check for hardcoded credentials or API keys
```

```
2. Review authentication and authorization logic
3. Look for SQL injection vulnerabilities
4. Check for XSS vulnerabilities in frontend code
5. Review file upload handling for security issues
6. Check dependency versions for known vulnerabilities
7. Review error handling to ensure no sensitive data leaks

Provide specific recommendations for each issue found.
```

Tip: If you find yourself copying and pasting the same prompt repeatedly, it's time to make it a slash command!

Example with Arguments: `.claude/commands/fix-issue.md`

None
```
Please fix GitHub issue: $ARGUMENTS

Steps:
1. Use `gh issue view $ARGUMENTS` to read the issue
2. Identify the root cause
3. Find relevant files with grep/find
4. Implement the fix
5. Write tests for the fix
6. Verify all tests pass
7. Create a commit with message "Fix #$ARGUMENTS: [description]"
```

Usage:

Shell
```
/project:review-pr
/project:fix-issue 123
```

---

13. Working with MCP Tools {#mcp-tools}

Hook Integration with MCP

Model Context Protocol (MCP) tools appear with special naming patterns in hooks and provide specialized capabilities.

Essential MCP Servers

Core MCP Tools:

1. **Sequential**: Enables complex multi-step thinking and problem-solving
2. **Context7**: Grabs official libraries and documentation from online sources
3. **Magic**: Generates modern UI components with best practices
4. **Playwright**: Browser automation and testing capabilities
5. **Memory**: Persistent context storage across sessions
6. **Filesystem**: Enhanced file operations
7. **GitHub**: Direct GitHub API integration

Installing MCP Servers

```shell
# Add MCP servers to Claude Code
claude mcp add sequential
claude mcp add context7
claude mcp add magic
claude mcp add playwright

# Check connected MCP servers
claude
# Then use: /mcp
```

MCP Tool Naming Patterns

MCP tools follow the pattern `mcp__<server>__<tool>`:

- `mcp__memory__create_entities`
- `mcp__filesystem__read_file`
- `mcp__github__search_repositories`
- `mcp__sequential__think_step_by_step`
- `mcp__context7__fetch_documentation`

- `mcp__magic__generate_component`

Example Hooks for MCP Tools

```JSON
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "mcp__github__*",
      "hooks": [{
        "type": "command",
        "command": "echo '[$(date)] GitHub MCP action:
$CLAUDE_TOOL_NAME' >> ~/.claude/mcp-audit.log"
      }]
    }],
    "PostToolUse": [{
      "matcher": "mcp__filesystem__write_file",
      "hooks": [{
        "type": "command",
        "command": "git add $CLAUDE_FILE_PATHS && echo
'Auto-staged changes from MCP filesystem write'"
      }]
    }]
  }
}
```

---

14. Learning and Development Workflows {#learning-workflows}

Using Claude Code with Other AI Tools

Multi-AI Workflow: Combine Claude Code with other AI assistants for enhanced learning and development.

Example: Claude Code + Cursor IDE

1. **Run Claude Code** for implementing features or fixes
2. **Use Cursor's AI** to ask questions about the generated code:

    - "Explain what this function does"
    - "List everything in the .claude directory and its purpose"
    - "What design patterns are being used here?"
3. **Deep dive with Claude Desktop** for complex concepts you don't understand

Creating a Personal Learning Assistant

Build a custom GPT or AI assistant trained on:

- Your preferred learning style
- Your tech stack and domain knowledge
- Your coding conventions

Then feed it Claude Code's output for personalized explanations and learning paths.

Hook for Learning Capture

JSON

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "echo '## Code Change\nTool:
$CLAUDE_TOOL_NAME\nFiles: $CLAUDE_FILE_PATHS\n' >>
.claude/learning-log.md"
      }]
    }]
  }
}
```

This creates a learning log you can review to understand what Claude did and why.

---

15. Sub-Agents and Multi-Agent Systems {#sub-agents}

Introduction to Sub-Agents (New Feature! 🎉)

Sub-agents in Claude Code are specialized AI assistants that operate independently with their own context and focused capabilities. Released recently, this feature transforms how you structure complex development workflows. Think of them as expert consultants you can call in for specific tasks.

Key Insight: Agents are essentially "pre-configured little prompts" that live within your projects. Claude automatically delegates work to them when it recognizes a match with their expertise - you're building a team of specialized AI helpers tailored to your workflow.

Creating Sub-Agents

Getting Started:

1. Open the sub-agents interface with `/agents`
2. Select "Create New Agent"
3. Choose scope:
    - **Project-level** (`.claude/agents/`): For project-specific tasks, shareable via Git
    - **Personal** (`~/.claude/agents/`): Available across all projects for general utilities
4. Describe what you want (e.g., "An agent that rolls dice")
5. Select which tools/permissions the agent needs
6. **Choose a color**: Helps identify which agent is running in terminal
7. Save and use

Natural Language Creation: Instead of using `/agents`, you can simply say:

- "Create a debugging specialist agent"
- "I need an agent that only reviews Python code"
- "Set up a data analysis assistant"
- "Make a security auditor agent"

Key Differences: Main Claude vs Agents

Aspect	Main Claude Code	Agents/Subagents
Context	Full conversation history	Fresh start, no prior context
Purpose	General-purpose assistant	Specialized single-task focus
Memory	Maintains state across messages	No memory between invocations

Best For	Iterative development, context-heavy work	Isolated tasks, specialized expertise
Tool Access	Full access as configured	Restricted based on purpose

Key Features

🎯 Context Preservation

Each sub-agent operates in its own context, preventing pollution of the main conversation and keeping focus on specific objectives.

🔧 Specialized Expertise

Sub-agents can be fine-tuned with detailed instructions for specific domains, leading to higher success rates on designated tasks.

♻️ Reusability

Once created, sub-agents can be used across different projects and shared with your team for consistent workflows.

🔒 Flexible Permissions

Each sub-agent can have different tool access levels, allowing you to limit powerful tools to specific agent types.

Sub-Agent Best Practices

1. System Prompt is Key

With sub-agents, you write the system prompt, not the user prompt. This gives you more control over the agent's behavior.

```
None
# Example Sub-Agent System Prompt for Code Review
You are a specialized code review agent. Your primary agent will
send you code changes to review.

Your responsibilities:
1. Check for bugs and logic errors
```

```
2. Identify security vulnerabilities
3. Suggest performance improvements
4. Ensure code follows project conventions

Always respond with structured feedback that the primary agent
can act upon.
```

2. Sub-Agents Respond to Your Primary Agent

Remember: sub-agents communicate with your primary agent, not directly with you. Write prompts accordingly.

3. Be Explicit in Your Instructions

The more specific your instructions, the better your agents perform. Avoid ambiguity:

❌ **Vague**: "Review this code" ✅ **Explicit**: "Review this Python code for SQL injection vulnerabilities, checking all database queries for parameterization"

4. Use the Meta-Agent

The meta-agent is a powerful tool for creating new agents from scratch. Use it to bootstrap your multi-agent system.

5. Chain Sub-Agents for Complex Workflows

```
None
Primary Agent → Code Generator Sub-Agent → Code Review Sub-Agent
→ Test Writer Sub-Agent
```

The "Big 3" of Agentic Coding

As you scale multi-agent systems, three elements become critical:

1. **Context**: What information each agent has access to
2. **Model**: Which AI model powers each agent
3. **Prompt**: How each agent is instructed

The flow and management of these three elements determines the success of your multi-agent system.

Hooks + Sub-Agents: Advanced Integration

Auto-Generate Sub-Agents with Hooks

Use hooks to automatically create specialized sub-agents based on project needs:

Python

```
#!/usr/bin/env python3
# .claude/hooks/auto_agent_creator.py
import json
import sys
import os

def should_create_agent(tool_input):
    # Check if working with a new framework/library
    file_path = tool_input.get("file_path", "")

    # Detect new patterns that might need specialized agents
    patterns = {
        "react": ["useState", "useEffect", "jsx"],
        "django": ["models.Model", "views", "urls.py"],
        "fastapi": ["FastAPI", "@app.", "BaseModel"],
    }

    for framework, keywords in patterns.items():
        agent_path = f".claude/agents/{framework}-specialist.md"
        if not os.path.exists(agent_path) and any(kw in
str(tool_input) for kw in keywords):
            create_specialist_agent(framework, agent_path)
            print(f"🤖 Created {framework} specialist agent!")

def create_specialist_agent(framework, path):
    template = f"""---
name: {framework}-specialist
description: Expert in {framework} development patterns and best
practices
---
```

```
You are a {framework} specialist. Help the primary agent with:
- {framework}-specific patterns and idioms
- Performance optimizations
- Security best practices
- Testing strategies

Always provide {framework}-idiomatic solutions.
"""
    os.makedirs(os.path.dirname(path), exist_ok=True)
    with open(path, 'w') as f:
        f.write(template)

if __name__ == "__main__":
    data = json.load(sys.stdin)
    if data.get("tool_name") in ["Edit", "Write"]:
        should_create_agent(data.get("tool_input", {}))
```

Advanced Sub-Agent Patterns

Proactive Triggering

Configure agents to automatically perform tasks when conditions are met:

```
None
# Auto-Summary Sub-Agent
When called, check if:
- More than 10 files have been modified
- More than 500 lines of code changed
- A major refactoring occurred
- Session duration exceeds 30 minutes

If any condition is true, automatically generate:
1. Executive summary of changes
2. Technical details of modifications
3. Next steps and recommendations
4. Any identified technical debt
```

```
Format the summary for easy sharing with team members.
```

Hook Integration for Auto-Summary:

```JSON
{
  "hooks": {
    "Stop": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "if [ $(git diff --stat | tail -1 | awk
'{print $4}') -gt 500 ]; then echo 'Major changes detected.
Consider running /agent:summarizer'; fi"
      }]
    }]
  }
}
```

Information-Dense Keywords

Claude Code provides special keywords and variables to give agents more context:

Environment Variables:

- `$CLAUDE_PROJECT_DIR` - Project root directory
- `$CLAUDE_FILE_PATHS` - Currently relevant files
- `$CLAUDE_TOOL_NAME` - Active tool being used
- `$CLAUDE_SESSION_ID` - Current session identifier

In Sub-Agent Prompts:

- Reference current state with `{{current_files}}`
- Access recent changes with `{{recent_changes}}`
- Include project context with `{{project_context}}`

Example Usage:

```
None
Review the changes in {{recent_changes}} and ensure they follow
the patterns established in {{project_context}}. Pay special
attention to files matching {{current_files}} pattern.
```

Sub-Agent Configuration with Hooks

Combine sub-agents with hooks for powerful automation:

```
JSON
{
  "hooks": {
    "SubagentStop": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "echo 'Sub-agent completed: Review the output
for quality' | tee -a .claude/subagent-log.txt"
      }]
    }]
  }
}
```

Monitoring Sub-Agent Performance

Create a hook to track sub-agent execution times and success rates:

```
Python
#!/usr/bin/env python3
import json
import sys
import time
from datetime import datetime

def log_subagent_performance():
    data = json.load(sys.stdin)
```

```
    # Extract sub-agent info from context
    timestamp = datetime.now().isoformat()
    session_id = data.get("session_id", "unknown")

    # Log performance metrics
    log_entry = {
        "timestamp": timestamp,
        "session_id": session_id,
        "event": "subagent_complete",
        "duration": time.time() - float(data.get("start_time",
time.time()))
    }

    with open(".claude/subagent-metrics.jsonl", "a") as f:
        f.write(json.dumps(log_entry) + "\n")

    # Alert if sub-agent took too long
    if log_entry["duration"] > 300:  # 5 minutes
        print("⚠️ Sub-agent took longer than expected!")

if __name__ == "__main__":
    log_subagent_performance()
```

Important Warnings

⚠️ YOLO Mode Caution

YOLO mode gives agents free rein to execute without confirmation. Use with extreme caution and only in safe environments.

🔐 Keep Agents Separate

As you build more agents, maintain clear separation to avoid confusion and keep your codebase clean.

🏝️ Use Isolated Workflows

Create isolated environments for different agent workflows to prevent interference:

- Separate directories for agent-specific files
- Dedicated configuration files
- Independent logging streams
- Isolated test environments

🔄 Don't Overload Sub-Agent Chains

Long chains of sub-agents can become difficult to manage and debug. Keep chains reasonable (3-5 agents max).

🧠 Don't Offload All Cognitive Work

While agents are powerful assistants, maintain your understanding of what they're doing. You need to:

- Understand the code being generated
- Make architectural decisions
- Review and validate agent outputs
- Maintain overall system vision

Agents augment your capabilities—they don't replace your expertise.

⏱️ Performance Considerations

- **Latency**: Agents start fresh, adding 1-2 seconds overhead per invocation
- **Token Efficiency**: Clean slate can be more efficient for focused tasks
- **Context Loss**: No memory of previous conversations - plan accordingly
- **Parallel Execution**: Multiple agents can work simultaneously for speed

Sub-Agent Frontmatter

The Description is EVERYTHING 🎯

The description field is the most critical part of your agent configuration. Claude uses this to automatically discover and decide when to use an agent. Get this right, and your agents will feel magical.

```
None
---
name: security-auditor
description: |
  Performs comprehensive security audits on code.
  Call this agent when:
```

```
  - New authentication code is added
  - File upload functionality is implemented
  - Database queries are modified
  - External API integrations are created
  - User says "security check" or "audit this code"

  This agent will return a structured security report with:
  - Identified vulnerabilities (scored by severity)
  - Specific remediation steps
  - Security best practices relevant to the code
tools:
  - read_file
  - search_code
  - analyze_dependencies
---
```

Description Best Practices:

- **Be Specific**: Vague descriptions like "reads files" cause wrong agent activation
- **Add Trigger Phrases**: Include examples like "Use when user says 'roll a d6'"
- **Specify Output Format**: Tell Claude what the agent returns
- **List Explicit Conditions**: When to use AND when NOT to use
- **Unique Purpose**: Each agent should have a clear, distinct role

Real-World Agent Examples

1. Strict Code Reviewer

```
None
---
name: strict-reviewer
description: |
  Enforces coding standards and security practices.
  Call after implementing features or before PRs.
  Triggers: "review this code", "check for issues", "security
review"
  Returns structured feedback on:
```

```
  - Security vulnerabilities
  - Performance issues
  - Code smells
  - Missing tests
  Never suggests implementation - only identifies issues.
tools:
  - read_file
  - search_files
---

You are a senior code reviewer with a security background. Be
strict but constructive. Focus on:
1. Security vulnerabilities (OWASP Top 10)
2. Performance bottlenecks
3. Code maintainability
4. Test coverage gaps

Output format:
- 🔴 Critical: Security/breaking issues
- 🟡 Warning: Performance/quality issues
- 🟢 Good: Positive patterns observed
```

Tool Permission Strategy

Configure agent tools based on the principle of least privilege:

Agent Type	Recommended Tools	Why
Reviewers	read_file, search_files	Read-only analysis
Writers	read_file, write_file	Create/modify code
Testers	All file tools + run_command	Need to execute tests
Analyzers	read_file only	Minimal access needed
Builders	All tools	Full development access

Security Tips:

- Start with minimal permissions
- Add tools only when needed
- Review agents can't modify = safer
- Restrict bash/command access carefully

2. Performance Optimizer

```
None
---
name: perf-optimizer
description: |
  Identifies and fixes performance bottlenecks.
  Call when:
  - Users report slowness
  - Before scaling up
  - After adding new features
  Outputs specific optimization recommendations with benchmarks.
tools:
  - read_file
  - write_file
  - run_command  # For profiling tools
---

You specialize in performance optimization. Always:
1. Measure before optimizing (no premature optimization)
2. Focus on algorithmic improvements over micro-optimizations
3. Consider caching strategies
4. Analyze database queries
5. Check for N+1 problems
6. Review memory usage

Provide specific before/after metrics when possible.
```

3. Test Generator

```
None
---
name: test-gen
description: |
  Creates comprehensive test suites.
  Call after implementing new features.
  Targets 90%+ coverage including:
  - Happy path tests
  - Edge cases
  - Error scenarios
  - Integration tests
tools:
  - read_file
  - write_file
  - search_files
---

You are a test-driven development expert. For any code provided:
1. Write comprehensive unit tests
2. Include edge cases and error paths
3. Add integration tests where appropriate
4. Use the project's existing test patterns
5. Aim for >90% coverage
6. Include performance benchmarks for critical paths

Test naming convention:
test_[function]_[scenario]_[expected_result]
```

4. Simple Dice Roller (Non-Code Example)

```
None
---
name: dice-roller
description: |
  Rolls dice for games or random decisions.
  Triggers: "roll a d6", "roll some dice", "roll 2d20"
```

```
  Returns the results of dice rolls.
tools: []  # No tools needed!
---

You are a dice rolling assistant. When asked to roll dice:
1. Parse the request (e.g., "2d6" = roll 2 six-sided dice)
2. Generate random results
3. Show individual rolls and total
4. Be enthusiastic about the results!

Format: 🎲 Rolling [dice]... Results: [individual rolls] =
[total]
```

5. Debugging Specialist

None

```
---
name: debugger
description: |
  Systematic debugging using root cause analysis.
  Call when:
  - Encountering difficult bugs
  - Intermittent issues
  - Performance degradation
  Uses Five Whys and systematic elimination.
tools:
  - read_file
  - search_files
  - run_command  # For running tests/logs
---

You are a debugging specialist. Approach every problem
systematically:

1. Reproduce the issue
2. Gather all relevant information
```

```
3. Form hypotheses
4. Test each hypothesis
5. Apply Five Whys technique
6. Document the root cause
7. Suggest both quick fixes and proper solutions

Never assume - always verify. Start fresh without preconceptions.
```

Visual Agent Identification

When agents run, Claude Code provides visual feedback:

```
None
🟦 dice-roller: Rolling 2d6...
   Results: [4, 2] = 6

🟩 summarizer: Processing results...
   Summary: Rolled moderate results

🟨 file-writer: Saving output...
   Created: dice-results.txt
```

Color Benefits:

- Quickly see which agent is active
- Track multi-agent workflows visually
- Identify agent hand-offs
- Debug agent chains easily
- Personal preference for organization

Example: Multi-Agent Development Workflow

```
None
# Primary Agent Instructions
You coordinate a team of specialized sub-agents:
```

```
1. **Architect Agent**: Design system architecture
2. **Implementation Agent**: Write code following the
architecture
3. **Test Agent**: Create comprehensive test suites
4. **Documentation Agent**: Generate docs and comments
5. **Review Agent**: Final quality check

For each feature request:
1. Have Architect design the approach
2. Implementation writes the code
3. Test creates tests
4. Documentation updates docs
5. Review ensures quality

Only proceed to the next step after the previous agent confirms
completion.
```

When to Use What: Agents vs Hooks vs Main Claude

Use Main Claude When:

- You need conversation context
- Working iteratively on the same code
- Debugging with context from previous attempts
- General development work
- Quick questions or clarifications

Use Agents When:

- Task requires fresh perspective (no context bias)
- Specialized expertise needed (security audit, performance analysis)
- Want consistent, repeatable process
- Need isolation for security/safety
- Task is self-contained

Use Hooks When:

- Need deterministic automation
- Want to enforce policies/standards
- Task is simple and well-defined

- Speed is critical
- Integration with external tools required

Combine All Three:

```shell
# Example workflow combining all three
1. Main Claude: "Plan the user authentication feature"
2. Hook: Auto-creates architecture document template
3. Agent: architect reviews and completes design
4. Main Claude: Implements based on architecture
5. Hook: TDD reminder before coding
6. Agent: test-writer creates test suite
7. Main Claude: Implements to pass tests
8. Hook: Auto-formats and lints code
9. Agent: security-auditor reviews implementation
10. Hook: Generates session summary
```

Agent Chaining Patterns

The True Power: Multi-Agent Workflows

The real magic happens when you chain agents together. Claude can identify and sequence multiple agents automatically based on your request.

Example: `"roll some dice and summarize and finally save the output to a file"`

Claude automatically:

1. Uses `dice-roller` agent → gets results
2. Passes to `summarizer` agent → creates summary
3. Passes to `file-writer` agent → saves output

Sequential Chain

```none
debugger → finds issue → test-writer → writes failing test →
fixer → implements solution → reviewer → validates fix
```

Parallel Analysis

```
None
            ┌→ security-auditor ─┐
code-change ├→ performance-opt   ├→ combined report
            └→ test-coverage    ─┘
```

Hierarchical Delegation

```
None
architect → creates design
    ├→ frontend-dev → implements UI
    ├→ backend-dev → implements API
    └→ test-writer → creates integration tests
```

The "Predictability Problem" is a Feature!

Agents are **probabilistic, not deterministic**. This might feel unpredictable, but it's actually their superpower:

Embrace It

- You don't need to remember exact agent names
- Just describe what you want: "roll a d6" triggers the dice-roller
- Natural language "just works"
- Claude figures out the best agent for the task

Control It

- Make descriptions very specific about when to activate
- Add negative examples: "Do NOT use for..."
- Use explicit trigger phrases
- Test agent activation with various prompts

Beyond Code: Universal Automation

Agents aren't just for programming! Create agents for:

- **Document Outlining**: Based on your templates
- **Meeting Summaries**: Transcripts → actionable items
- **Research Organization**: Categorize and tag notes
- **Content Creation**: Blog posts, emails, reports

- **Data Analysis**: Non-technical data insights

By building a library of specialized agents, you shift from telling AI *how* to do something to simply describing *what* you want done.

Quick Agent Commands Reference

Creating Agents

```shell
# Using slash command
/agents

# Natural language
"Create a Python debugging specialist"
"I need an agent for SQL optimization"
"Set up a security auditor"

# From template
"Create agent like test-gen but for integration tests"
```

Using Agents

```shell
# Automatic delegation - just describe what you want
"Review this code" → Claude auto-selects reviewer agent
"Debug this error" → Claude auto-selects debugger agent
"Roll a d20" → Claude auto-selects dice-roller agent

# Explicit invocation when you want specific agent
"Use the security-auditor to check this file"
"Have the test-gen create tests for this function"

# Natural language chaining
"Roll some dice and summarize the results"
"Review this code then generate tests for any issues found"

# Complex workflow in one command
"Analyze this bug, write a failing test, fix it, then review"
```

Managing Agents

```shell
/agents list              # See all agents
/agents edit [name]       # Modify agent
/agents delete [name]     # Remove agent
/agents export [name]     # Export for sharing
```

Hooks vs Sub-Agents: When to Use Each

Use Hooks When:

- You need deterministic, automated actions
- The task is simple and well-defined
- You want to enforce policies or safety checks
- Speed is critical (hooks are faster)
- You need to integrate with external tools

Use Sub-Agents When:

- The task requires AI reasoning or creativity
- You need specialized domain expertise
- The problem is complex and requires context
- You want reusable, shareable components
- You need natural language processing
- Fresh perspective without context bias is valuable

Combining Both:

```json
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "python3
.claude/hooks/check_if_needs_security_agent.py"
      }]
```

```
    }]
  }
}
```

This hook could analyze commands and automatically suggest calling the security sub-agent when needed.

Practical Integration Example:

```python
#!/usr/bin/env python3
# Hook that suggests relevant sub-agents based on activity
import json
import sys

AGENT_TRIGGERS = {
    "security-auditor": ["password", "auth", "token", "api_key"],
    "performance-optimizer": ["slow", "optimize", "benchmark"],
    "test-writer": ["test", "spec", "coverage"],
}

def suggest_agents(tool_input):
    suggestions = []
    content = str(tool_input).lower()

    for agent, triggers in AGENT_TRIGGERS.items():
        if any(trigger in content for trigger in triggers):
            suggestions.append(agent)

    if suggestions:
        print(f"💡 Consider using these sub-agents: {',
'.join(suggestions)}")

if __name__ == "__main__":
    data = json.load(sys.stdin)
    if data.get("tool_name") in ["Edit", "Write"]:
```

```
        suggest_agents(data.get("tool_input", {}))
```

---

16. Professional Development Patterns {#professional-patterns}

The Claude Code Gap: From Scripts to Systems

While Claude Code excels at generating code quickly, it often skips critical professional development steps. This works fine for small scripts but falls apart for enterprise-level applications. Professional developers don't just write code—they plan, architect, design, test, and secure their systems.

Professional Workflow Stages

Here's how to structure Claude Code for serious development:

1. Project Planning & Architecture

```
None
# Command: /project:architect-plan "Design a scalable e-commerce
platform"

Perform comprehensive project planning:
1. Define system requirements and constraints
2. Create high-level architecture diagrams
3. Design database schema
4. Plan API structure
5. Define security protocols
6. Create testing strategy
7. Estimate scalability needs (users, data, traffic)

Output a structured architecture document with:
- System overview
- Component breakdown
- Data flow diagrams
- Security considerations
```

```
- Scalability assessment
```

2. Frontend Development with Standards

```
None
# Command: /project:frontend-tdd "Implement user dashboard"

Develop frontend with professional standards:
1. Review UI/UX designs or create wireframes
2. Set up component architecture
3. Write tests FIRST (TDD approach)
4. Implement components
5. Ensure accessibility (WCAG compliance)
6. Optimize performance
7. Add proper error handling

Use modern patterns:
- React Query for server state
- Proper state management
- Component composition
- Performance optimization
```

3. Backend Development with TDD

```
None
# Command: /project:backend-secure "Build authentication system"

Professional backend development:
1. Design API contracts first
2. Write integration tests
3. Implement with security in mind
4. Add comprehensive error handling
5. Include logging and monitoring
6. Document all endpoints
```

```
7. Performance profiling

Follow patterns:
- Repository pattern for data access
- Service layer for business logic
- Proper validation and sanitization
- Rate limiting and security headers
```

Essential Personas for Professional Development

Create specialized sub-agents for each role:

System Architect

```
None
---
name: system-architect
description: |
  Enterprise architecture specialist focusing on:
  - Scalable system design
  - Technology selection
  - Integration patterns
  - Performance architecture
  - Security architecture

  Call when planning new systems or major refactors
---
```

Security Engineer

```
None
---
name: security-engineer
description: |
  Security specialist for:
  - Vulnerability assessment
```

```
  - Security architecture review
  - Penetration testing strategies
  - Compliance requirements
  - Security best practices

  Call for any security-sensitive features
---
```

Performance Engineer

```
None
---
name: performance-engineer
description: |
  Performance optimization expert for:
  - Bottleneck identification
  - Query optimization
  - Caching strategies
  - Load testing
  - Scalability planning
---
```

MCP Tools for Professional Development

Enhance Claude Code with specialized MCP servers:

1. **Sequential Thinking**: Complex multi-step problem solving
2. **Context7**: Access official documentation and libraries
3. **Magic**: Modern UI component generation
4. **Playwright**: Automated testing and browser automation

Professional Troubleshooting Workflow

Step 1: Problem Identification

```
None
# Command: /project:investigate "Production errors increasing"
```

```
Investigate the issue:
1. Analyze error logs
2. Check monitoring dashboards
3. Review recent deployments
4. Identify patterns
5. Document symptoms
```

Step 2: Root Cause Analysis

```
None
# Command: /project:five-whys "API timeout errors"

Apply Five Whys methodology:
1. Why is the API timing out?
2. Why is the database query slow?
3. Why is the index not being used?
4. Why was the index dropped?
5. Why wasn't this caught in testing?

Output: Root cause and remediation plan
```

Step 3: Performance Analysis

```
None
# Command: /project:profile "Analyze slow endpoints"

Performance profiling:
1. Run performance profiler
2. Identify bottlenecks
3. Analyze database queries
4. Check memory usage
5. Review caching effectiveness
6. Suggest optimizations
```

Architecture Analysis Hook

Create a hook that triggers architecture analysis for significant changes:

```JSON
{
  "hooks": {
    "Stop": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "python3 .claude/hooks/architecture_check.py"
      }]
    }]
  }
}
```

Architecture Check Script:

```Python
#!/usr/bin/env python3
import subprocess
import json

def check_architecture_impact():
    # Check changed files
    changed = subprocess.check_output(['git', 'diff',
'--name-only'], text=True)

    critical_patterns = [
        'schema', 'migration', 'auth', 'security',
        'api/', 'architecture', 'config'
    ]

    if any(pattern in changed.lower() for pattern in
critical_patterns):
        print("\n🏗️ ARCHITECTURE IMPACT DETECTED")
```

```
        print("Consider running: /project:architect-review")
        print("Critical files changed that may affect system
architecture")

        # Auto-generate mini architecture report
        with open('.claude/architecture-changes.md', 'w') as f:
            f.write(f"# Architecture Impact Analysis\n\n")
            f.write(f"## Changed Files\n{changed}\n")
            f.write(f"## Recommended Actions\n")
            f.write(f"- Review system architecture\n")
            f.write(f"- Update documentation\n")
            f.write(f"- Check security implications\n")
            f.write(f"- Verify scalability impact\n")

if __name__ == "__main__":
    check_architecture_impact()
```

Quality Gates with Hooks

Implement quality gates that enforce professional standards:

JSON

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": ".claude/hooks/quality_gate.sh"
      }]
    }]
  }
}
```

Quality Gate Script:

```shell
#!/bin/bash
# .claude/hooks/quality_gate.sh

# Check if tests exist for new code
if [[ "$CLAUDE_FILE_PATHS" =~ \.(ts|js|py)$ ]]; then
    base_name=$(basename "$CLAUDE_FILE_PATHS" | sed
's/\.[^.]*$//')
    test_file=$(find . -name "*${base_name}*test*" -o -name
"*${base_name}*spec*" | head -1)

    if [ -z "$test_file" ]; then
        echo "⚠️ WARNING: No tests found for $CLAUDE_FILE_PATHS"
>&2
        echo "Consider TDD: Write tests before implementation"
>&2
        # Don't block, just warn
        exit 0
    fi
fi
```

Scalability Assessment Patterns

Create commands that analyze scalability at different stages:

```none
# Command: /project:scale-assessment "Current system analysis"

Perform scalability assessment:
1. Current capacity (concurrent users, data volume)
2. Identify bottlenecks:
   - Database connections
   - API rate limits
   - Memory usage
   - Third-party service limits
```

```
3. Growth projections
4. Scaling recommendations:
   - Horizontal vs vertical scaling
   - Caching strategies
   - Database optimization
   - Service decomposition
5. Cost analysis

Output scaling roadmap with priorities
```

Architecture Scoring System

Implement automated architecture scoring:

Python

```python
#!/usr/bin/env python3
# .claude/hooks/architecture_scorer.py
import json
import subprocess

def calculate_architecture_score():
    score = 100
    feedback = []

    # Check for architecture documentation
    if not os.path.exists("ARCHITECTURE.md"):
        score -= 10
        feedback.append("Missing architecture documentation
(-10)")

    # Check for proper separation of concerns
    if os.path.exists("src"):
        structure = subprocess.check_output(["find", "src",
"-type", "d"], text=True)
        if "components" in structure and "services" in structure:
```

```
            feedback.append("Good separation of concerns (+5)")
        else:
            score -= 5
            feedback.append("Poor separation of concerns (-5)")

    # Check for tests
    test_coverage = subprocess.run(["npm", "run",
"test:coverage", "--silent"],
                                   capture_output=True, text=True)
    if "Coverage" in test_coverage.stdout:
        # Parse coverage percentage
        feedback.append("Test coverage detected (+10)")
    else:
        score -= 20
        feedback.append("No test coverage found (-20)")

    # Security checks
    if os.path.exists(".env.example") and not
os.path.exists(".env"):
        feedback.append("Good security practice: .env.example
exists (+5)")

    # Performance considerations
    if "react-query" in open("package.json").read() or "swr" in
open("package.json").read():
        feedback.append("Server state management detected (+5)")

    print(f"\n🏗️ Architecture Score: {score}/100")
    print("\nFeedback:")
    for item in feedback:
        print(f"  • {item}")

    if score < 70:
        print("\n⚠️ Architecture needs improvement!")
        print("Run: /project:architect-review for
recommendations")
```

```
if __name__ == "__main__":
    calculate_architecture_score()
```

Capacity Planning Matrix

Use this matrix for scalability planning:

Scale Level	Users	Requirements	Architecture Changes
Prototype	1-100	Single server	Monolithic OK
Small	100-1K	Basic caching	Add Redis
Medium	1K-10K	Load balancing	Horizontal scaling
Large	10K-100K	Microservices	Service decomposition
Enterprise	100K+	Full distribution	Multi-region, queues

Professional Development Checklist

Use this checklist for every serious project:

- [ ] Architecture documented before coding
- [ ] Security review completed
- [ ] API contracts defined
- [ ] Test strategy in place
- [ ] Performance benchmarks set
- [ ] Monitoring configured
- [ ] Documentation updated
- [ ] Code review process defined
- [ ] Deployment strategy planned
- [ ] Rollback procedures ready

Integration Example: Full Professional Workflow

Shell

```
# 1. Start with architecture
```

```
/project:architect-plan "Build a real-time collaboration tool"

# 2. Security review
/project:security-review "Review authentication approach"

# 3. Frontend with TDD
/project:frontend-tdd "Implement collaborative editor"

# 4. Backend with security
/project:backend-secure "Build WebSocket server"

# 5. Performance testing
/project:performance-test "Load test with 1000 concurrent users"

# 6. Deploy with confidence
/project:deploy-checklist "Production deployment readiness"
```

This professional approach transforms Claude Code from a code generator into a comprehensive development platform that follows enterprise best practices.

Conclusion

Claude Code hooks and sub-agents represent a paradigm shift in AI-assisted development. By combining deterministic automation (hooks) with intelligent reasoning (sub-agents), you create a development environment that's both powerful and predictable.

Your Journey: From Hooks to Professional Systems

🚀 **Phase 1: Basic Automation (Week 1)**

- Start with simple notification hooks
- Add basic logging and debugging hooks
- Create your first safety checks
- Convert repetitive prompts to slash commands

🛡️ **Phase 2: Safety and Quality (Week 2-3)**

- Implement comprehensive command validation
- Set up automatic formatting and linting
- Add pre-commit style checks
- Create file protection rules
- **Add TDD reminders and test coverage checks**

🔧 **Phase 3: Advanced Workflows (Month 2)**

- Build multi-tool pattern matching
- Integrate with your CI/CD pipeline
- Set up observability dashboards
- Create project-specific automations
- **Implement architecture scoring and quality gates**

🤖 **Phase 4: Multi-Agent Systems (Month 3+)**

- Design your first specialized sub-agents
- Build agent chains for complex tasks
- Implement proactive triggering
- Create meta-agents for agent generation
- **Add professional personas (architect, security, performance)**

🏗️ **Phase 5: Enterprise Development (Month 4+)**

- Enforce full professional workflows
- Automate architecture reviews
- Implement scalability assessments
- Create comprehensive troubleshooting chains
- Build capacity planning automation

Key Takeaways

1. **Start Simple**: Begin with basic hooks and gradually increase complexity
2. **Security First**: Always review hook code and use restrictive permissions
3. **Community Wisdom**: Use slash commands instead of copy-paste, prefer new sessions over clearing
4. **Combine Tools**: Use Claude Code with other AI tools for enhanced learning
5. **Think Systems**: As you scale, focus on the flow of context, model, and prompts
6. **Read the Docs**: The official Claude Code documentation at docs.anthropic.com is your best resource
7. **Don't Skip Planning**: Use hooks and workflows to enforce architecture, security, and testing phases
8. **Embrace TDD**: Let hooks remind you to write tests before implementation
9. **Measure Quality**: Implement architecture scoring and quality gates
10. **Scale Thoughtfully**: Plan for growth with proper bottleneck analysis

11. **Agent Specialization**: Keep agents single-purpose for maximum effectiveness
12. **Context Strategy**: Use main Claude for continuity, agents for fresh perspectives
13. **Combine Approaches**: Hooks for automation, agents for intelligence, main Claude for context
14. **Description is Key**: Agent descriptions determine automatic discovery - make them specific
15. **Chain for Power**: Multi-agent workflows unlock complex automation
16. **Location Matters**: Personal agents for utilities, project agents for team consistency
17. **Predictability is Good**: Embrace probabilistic agent selection as a feature
18. **Beyond Code**: Use agents for any workflow - writing, analysis, organization
19. **Least Privilege**: Give agents only the tools they absolutely need
20. **Color Code**: Use colors to quickly identify which agent is running

The Future of Development

With hooks and sub-agents, you're not just using an AI assistant—you're building an intelligent development platform tailored to your exact needs. Every hook you write, every sub-agent you create, and every workflow you automate brings you closer to a future where repetitive tasks disappear and creativity flourishes.

The Paradigm Shift: With a well-crafted agent library, you move from telling AI *how* to do things to simply describing *what* you want done. Your custom AI team figures out the rest.

Remember the Professional Gap: Claude Code's strength in rapid code generation can be a weakness for complex projects. Use the patterns in this guide to add the planning, architecture, security, and testing phases that professional development demands.

The goal isn't to replace human judgment but to augment it. Use these tools to:

- Eliminate toil and repetitive tasks
- Enforce best practices automatically
- Add professional development stages
- Create quality gates and checkpoints
- Free yourself to focus on architecture and problem-solving

Whether you're building a simple script or an enterprise application, the combination of hooks, sub-agents, and professional workflows ensures your development process scales with your ambitions.

Happy automating, and may your hooks always exit cleanly! 🚀🎣🤖, r'.js

---

5. Best Practices and Debugging {#best-practices}

🔍 Debug First

Always start by logging the JSON payload to understand the data structure:

```json
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "jq '.' >> .claude/hooks/log.jsonl"
      }]
    }]
  }
}
```

📁 Project Organization

- Keep `.claude/settings.json` in your project repository
- Store scripts in `.claude/hooks/` directory
- Make hooks shareable and version-controlled

🛡️ Error Handling

- Hooks should fail silently unless designed to block actions
- Exit with code 0 for normal operation
- Exit with code 2 to block an action and show an error

📦 Dependency Management

For Python scripts with dependencies, use **Astral's uv**:

```shell
uv run my_script.py
```

This handles environment setup automatically.

🎯 Key Principles

1. **Start simple**: Test with basic logging before complex logic
2. **Be defensive**: Handle missing data and errors gracefully
3. **Keep it fast**: Hooks run synchronously - avoid slow operations
4. **Document well**: Comment your scripts for future reference

💡 Pro Tips from the Community

1. **Session Management**: Instead of clearing sessions, start a new one. You can resume previous sessions with `claude --resume` for better context preservation
2. **Use Slash Commands**: Rather than copying and pasting prompts (like security checks), create custom slash commands for reusable workflows
3. **Multi-Tool Learning**: Use Claude Code alongside other tools like Cursor IDE - run Claude Code for tasks while using Cursor's AI to understand the code being generated
4. **Learning Workflow**: Create a learning loop by analyzing Claude's output in other AI tools to deepen your understanding of generated code

---

6. Advanced Hook Examples {#advanced-examples}

Multi-Tool Pattern Matching

Goal: Apply different actions based on file types across multiple tools.

```json
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|MultiEdit|Write",
        "hooks": [
          {
```

```
            "type": "command",
            "command": "if [[ \"$CLAUDE_FILE_PATHS\" =~ \\.(py)$
]]; then black $CLAUDE_FILE_PATHS && ruff check --fix
$CLAUDE_FILE_PATHS; fi"
          },
          {
            "type": "command",
            "command": "if [[ \"$CLAUDE_FILE_PATHS\" =~
\\.(ts|tsx)$ ]]; then prettier --write $CLAUDE_FILE_PATHS && npx
tsc --noEmit $CLAUDE_FILE_PATHS; fi"
          }
        ]
      }
    ]
  }
}
```

Session Context Loading

Goal: Automatically load project context when Claude Code starts.

Tip: Instead of clearing sessions when things get messy, start a new session and use `claude --resume` to restore previous context when needed. This preserves your conversation history for reference.

Script (`.claude/hooks/session_start.py`):

Python

```python
#!/usr/bin/env python3
import json
import sys
import subprocess

def load_context():
    context = []
```

```
    # Git status
    try:
        git_status = subprocess.check_output(['git', 'status',
'--short'], text=True)
        if git_status:
            context.append(f"Current git changes:\n{git_status}")
    except:
        pass

    # Recent commits
    try:
        commits = subprocess.check_output(['git', 'log',
'--oneline', '-5'], text=True)
        context.append(f"Recent commits:\n{commits}")
    except:
        pass

    # TODO items
    try:
        todos = subprocess.check_output(['grep', '-r', 'TODO:',
'.', '--include=*.py'], text=True)
        if todos:
            context.append(f"TODO items
found:\n{todos[:500]}...")
    except:
        pass

    # Output context for Claude
    if context:
        print("\n=== PROJECT CONTEXT ===")
        print("\n".join(context))
        print("========================\n")

if __name__ == "__main__":
    load_context()
```

Configuration:

```JSON
{
  "hooks": {
    "SessionStart": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "python3 .claude/hooks/session_start.py"
      }]
    }]
  }
}
```

Advanced Command Validation with AI Feedback

Goal: Use AI to analyze potentially dangerous commands before execution.

```Python
#!/usr/bin/env python3
import json
import sys
import re

DANGEROUS_PATTERNS = [
    (r'rm\s+-[^\s]*[rf]', "Recursive deletion detected"),
    (r'chmod\s+777', "Overly permissive file permissions"),
    (r'curl.*\|\s*sh', "Piping curl to shell - potential security
risk"),
    (r'npm\s+install.*--global', "Global npm install detected"),
    (r'docker.*--privileged', "Privileged Docker container"),
]

def analyze_command(command):
    risks = []
    for pattern, message in DANGEROUS_PATTERNS:
```

```
        if re.search(pattern, command, re.IGNORECASE):
            risks.append(message)

    if risks:
        # Exit code 2 blocks execution and feeds stderr to Claude
        print(json.dumps({
            "error": "Command blocked due to security concerns",
            "risks": risks,
            "command": command,
            "suggestion": "Please use a safer alternative or
confirm this is intentional"
        }), file=sys.stderr)
        sys.exit(2)

if __name__ == "__main__":
    data = json.load(sys.stdin)
    if data.get("tool_name") == "Bash":
        command = data.get("tool_input", {}).get("command", "")
        analyze_command(command)
```

---

7. Troubleshooting Common Issues {#troubleshooting}

Hook Not Executing

Problem: Your hooks aren't firing when expected.

Solutions:

1. **Check file permissions**:

```
Shell
chmod +x .claude/hooks/*.py
```

2. **Verify JSON syntax**:

```
Shell
jq '.' .claude/settings.json
```

3. **Use absolute paths** (recommended by documentation):

```
JSON
{
  "command":
"/absolute/path/to/your/project/.claude/hooks/script.py"
}
```

4. **Debug with logging**:

```
JSON
{
```

```
  "command": "echo 'Hook fired at $(date)' >>
/tmp/claude-hook-debug.log"
}
```

Settings Not Updating

Problem: Changes to settings.json don't take effect.

Solutions:

1. **Restart Claude Code** - Settings are loaded at startup
2. **Check for syntax errors** in your JSON
3. **Ensure you're editing the correct file**:
    - Project-specific: `.claude/settings.json`
    - User-global: `~/.claude/settings.json`

Hook Blocking Unintentionally

Problem: Hook exits with non-zero code accidentally.

Solution: Always handle errors gracefully:

```python
try:
    # Your hook logic
    pass
except Exception as e:
    # Log error but don't block Claude
    with open('/tmp/hook-errors.log', 'a') as f:
        f.write(f"Error: {e}\n")
    sys.exit(0)  # Exit successfully
```

Performance Issues

Problem: Hooks slow down Claude's operations.

Solutions:

1. **Use background execution** for non-critical hooks:

```
JSON
{
  "type": "command",
  "command": "your_command &"
}
```

2. **Set appropriate timeouts**:

```
JSON
{
  "type": "command",
  "command": "your_command",
  "timeout": 5
}
```

3. **Optimize scripts** - avoid heavy operations in synchronous hooks

---

8. Quick Reference Cheat Sheet {#cheat-sheet}

Hook Events

Event	Purpose	Can Block?	Key Data
`UserPromptSubmit`	Modify user prompts	No	`prompt`
`SessionStart`	Load context	No	`session_id`, `source`
`PreToolUse`	Validate/block tools	Yes (exit 2)	`tool_name`, `tool_input`
`PostToolUse`	Process results	No	`tool_response`
`Notification`	Custom alerts	No	`message`
`Stop`	Task completion	No	`transcript_path`
`SubagentStop`	Subtask completion	Yes (exit 2)	`stop_hook_active`
`PreCompact`	Before compaction	No	`trigger`

Exit Codes

- **0**: Success (stdout shown in transcript mode)
- **2**: Block operation (stderr sent to Claude)
- **Other**: Non-blocking error (stderr shown to user)

Environment Variables

```shell
$CLAUDE_PROJECT_DIR    # Absolute path to project root
$CLAUDE_FILE_PATHS     # Space-separated file paths
$CLAUDE_TOOL_NAME      # Name of the tool being used
$CLAUDE_NOTIFICATION   # Notification message content
```

Common Patterns

```JSON
// Match all tools
"matcher": ""

// Match specific tools
"matcher": "Bash"

// Match multiple tools
"matcher": "Edit|Write|MultiEdit"

// Match MCP tools
"matcher": "mcp__github__*"
```

Session Management

- **Start fresh**: Use new sessions instead of clearing when context gets messy
- **Resume sessions**: `claude --resume` to restore previous context
- **Headless mode**: `claude -p "prompt" --output-format stream-json` for automation
- **Non-interactive**: `claude -p "prompt" --no-interactive` for CI/CD

---

9. Integration with Development Workflows {#integrations}

Git Workflow Integration

Pre-commit Hook Integration:

```JSON
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
```

```
      "hooks": [{
        "type": "command",
        "command": "git add $CLAUDE_FILE_PATHS &&
./.git/hooks/pre-commit --files $CLAUDE_FILE_PATHS || true"
      }]
    }]
  }
}
```

CI/CD Integration

Headless Mode for Automation:

```shell
# Run Claude Code in CI pipeline
claude -p "Fix all linting errors in src/" --output-format
stream-json

# Pre-push validation
claude -p "Run all tests and fix any failures" --no-interactive
```

IDE Integration

VSCode Tasks Integration:

```json
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit",
      "hooks": [{
        "type": "command",
        "command": "code --goto $CLAUDE_FILE_PATHS:1"
      }]
    }]
```

```
  }
}
```

Docker Development

Container Management:

```JSON
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "python3 -c \"import json,sys;
d=json.load(sys.stdin);
cmd=d.get('tool_input',{}).get('command',''); sys.exit(2 if
'docker' in cmd and '--rm' not in cmd else 0)\""
      }]
    }]
  }
}
```

---

10. Security Considerations {#security}

Key Security Principles

1. **Hooks run with your credentials** - They have full access to your system
2. **Review all hook code** before enabling it
3. **Never run untrusted hooks** from unknown sources
4. **Use restrictive permissions** on hook scripts
5. **Validate all inputs** in your hook scripts

Secure Hook Template

Python

```
#!/usr/bin/env python3
import json
import sys
import os
import re

# Whitelist approach for allowed operations
ALLOWED_COMMANDS = [
    r'^ls\s',
    r'^cat\s',
    r'^grep\s',
    r'^find\s.*-name',
]

FORBIDDEN_PATHS = [
    '.env',
    '.ssh/',
    'credentials',
    'secrets',
    '.git/config',
]

def validate_command(command):
    # Check against whitelist
    if not any(re.match(pattern, command) for pattern in
ALLOWED_COMMANDS):
        return False, "Command not in allowlist"

    # Check for forbidden paths
    for path in FORBIDDEN_PATHS:
        if path in command:
            return False, f"Access to {path} is forbidden"

    return True, ""
```

```
def main():
    try:
        data = json.load(sys.stdin)
        if data.get("tool_name") == "Bash":
            command = data.get("tool_input", {}).get("command",
"")
            valid, reason = validate_command(command)

            if not valid:
                print(f"Security block: {reason}",
file=sys.stderr)
                sys.exit(2)
    except Exception as e:
        # Fail open - don't block on errors
        pass

if __name__ == "__main__":
    main()
```

---

11. Multi-Agent Observability {#observability}

Setting Up Observability Dashboard

Goal: Monitor multiple Claude Code agents in real-time.

Architecture:

```
None
Claude Agents → Hooks → HTTP Server → SQLite → WebSocket →
Dashboard
```

Hook Configuration:

```
JSON
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "python3 .claude/hooks/send_event.py --event
PreToolUse --agent $USER"
      }]
    }],
    "PostToolUse": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "python3 .claude/hooks/send_event.py --event
PostToolUse --agent $USER --include-output"
      }]
    }]
  }
}
```

Event Sender Script:

```python
#!/usr/bin/env python3
import json
import sys
import requests
import argparse
from datetime import datetime

def send_event(event_type, agent_name, include_output=False):
    data = json.load(sys.stdin)

    event = {
        "timestamp": datetime.utcnow().isoformat(),
        "agent": agent_name,
        "event_type": event_type,
        "tool_name": data.get("tool_name"),
        "tool_input": data.get("tool_input"),
    }

    if include_output:
        event["tool_output"] = data.get("tool_response",
{}).get("output", "")[:500]

    try:
        requests.post("http://localhost:8080/events", json=event,
timeout=1)
    except:
        pass  # Don't block Claude if observability is down

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--event", required=True)
    parser.add_argument("--agent", required=True)
    parser.add_argument("--include-output", action="store_true")
    args = parser.parse_args()
```

```
    send_event(args.event, args.agent, args.include_output)
```

---

12. Custom Slash Commands {#slash-commands}

Creating Custom Commands

Custom slash commands extend Claude Code's functionality through natural language templates. **Pro tip**: Instead of repeatedly copying and pasting prompts (like security checks or review templates), convert them into slash commands for instant reuse.

Location: `.claude/commands/` (project) or `~/.claude/commands/` (global)

Example: `.claude/commands/review-pr.md`

```
None
Please review the changes in the current git branch:

1. First, check what branch we're on with `git branch
--show-current`
2. Show the diff with `git diff main...HEAD`
3. Analyze the changes for:
   - Potential bugs or logic errors
   - Security vulnerabilities
   - Performance issues
   - Missing error handling
4. Suggest improvements with specific code examples
5. Check if tests are needed for the changes

Focus on actual issues, not style preferences.
```

Example: Convert Frequent Prompts to Commands:
`.claude/commands/security-check.md`

```
None
Perform a comprehensive security audit on the codebase:

1. Check for hardcoded credentials or API keys
2. Review authentication and authorization logic
3. Look for SQL injection vulnerabilities
4. Check for XSS vulnerabilities in frontend code
5. Review file upload handling for security issues
6. Check dependency versions for known vulnerabilities
7. Review error handling to ensure no sensitive data leaks

Provide specific recommendations for each issue found.
```

Tip: If you find yourself copying and pasting the same prompt repeatedly, it's time to make it a slash command!

Example with Arguments: `.claude/commands/fix-issue.md`

```
None
Please fix GitHub issue: $ARGUMENTS

Steps:
1. Use `gh issue view $ARGUMENTS` to read the issue
2. Identify the root cause
3. Find relevant files with grep/find
4. Implement the fix
5. Write tests for the fix
6. Verify all tests pass
7. Create a commit with message "Fix #$ARGUMENTS: [description]"
```

Usage:

```shell
/project:review-pr
/project:fix-issue 123
```

---

13. Working with MCP Tools {#mcp-tools}

Hook Integration with MCP

Model Context Protocol (MCP) tools appear with special naming patterns in hooks and provide specialized capabilities.

Essential MCP Servers

Core MCP Tools:

1. **Sequential**: Enables complex multi-step thinking and problem-solving
2. **Context7**: Grabs official libraries and documentation from online sources
3. **Magic**: Generates modern UI components with best practices
4. **Playwright**: Browser automation and testing capabilities
5. **Memory**: Persistent context storage across sessions
6. **Filesystem**: Enhanced file operations
7. **GitHub**: Direct GitHub API integration

Installing MCP Servers

```shell
# Add MCP servers to Claude Code
claude mcp add sequential
claude mcp add context7
claude mcp add magic
claude mcp add playwright

# Check connected MCP servers
claude
# Then use: /mcp
```

MCP Tool Naming Patterns

MCP tools follow the pattern `mcp__<server>__<tool>`:

- `mcp__memory__create_entities`
- `mcp__filesystem__read_file`
- `mcp__github__search_repositories`
- `mcp__sequential__think_step_by_step`
- `mcp__context7__fetch_documentation`
- `mcp__magic__generate_component`

Example Hooks for MCP Tools

JSON

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "mcp__github__*",
      "hooks": [{
        "type": "command",
        "command": "echo '[$(date)] GitHub MCP action:
$CLAUDE_TOOL_NAME' >> ~/.claude/mcp-audit.log"
      }]
    }],
    "PostToolUse": [{
      "matcher": "mcp__filesystem__write_file",
      "hooks": [{
        "type": "command",
        "command": "git add $CLAUDE_FILE_PATHS && echo
'Auto-staged changes from MCP filesystem write'"
      }]
    }]
  }
}
```

---

14. Learning and Development Workflows {#learning-workflows}

Using Claude Code with Other AI Tools

Multi-AI Workflow: Combine Claude Code with other AI assistants for enhanced learning and development.

Example: Claude Code + Cursor IDE

1. **Run Claude Code** for implementing features or fixes
2. **Use Cursor's AI** to ask questions about the generated code:
    - "Explain what this function does"
    - "List everything in the .claude directory and its purpose"
    - "What design patterns are being used here?"
3. **Deep dive with Claude Desktop** for complex concepts you don't understand

Creating a Personal Learning Assistant

Build a custom GPT or AI assistant trained on:

- Your preferred learning style
- Your tech stack and domain knowledge
- Your coding conventions

Then feed it Claude Code's output for personalized explanations and learning paths.

Hook for Learning Capture

JSON

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "echo '## Code Change\nTool:
$CLAUDE_TOOL_NAME\nFiles: $CLAUDE_FILE_PATHS\n' >>
.claude/learning-log.md"
      }]
    }]
  }
```

```
}
```

This creates a learning log you can review to understand what Claude did and why.

---

15. Sub-Agents and Multi-Agent Systems {#sub-agents}

Introduction to Sub-Agents

Sub-agents in Claude Code are specialized AI assistants that can be called by your primary Claude agent to handle specific tasks. They enable modular, scalable workflows by dividing complex problems into specialized domains.

Creating Sub-Agents

Getting Started:

1. Open the sub-agents interface with `/agents`
2. Select "Create New Agent"
3. Choose scope:
    - **Project-level**: Available only in current project
    - **User-level**: Available across all your projects
4. Edit the system prompt (press Enter to use your editor)
5. Save and use

Key Features

🎯 Context Preservation

Each sub-agent operates in its own context, preventing pollution of the main conversation and keeping focus on specific objectives.

🔧 Specialized Expertise

Sub-agents can be fine-tuned with detailed instructions for specific domains, leading to higher success rates on designated tasks.

♻️ Reusability

Once created, sub-agents can be used across different projects and shared with your team for consistent workflows.

🔒 **Flexible Permissions**

Each sub-agent can have different tool access levels, allowing you to limit powerful tools to specific agent types.

Sub-Agent Best Practices

1. System Prompt is Key

With sub-agents, you write the system prompt, not the user prompt. This gives you more control over the agent's behavior.

```
None
# Example Sub-Agent System Prompt for Code Review
You are a specialized code review agent. Your primary agent will
send you code changes to review.

Your responsibilities:
1. Check for bugs and logic errors
2. Identify security vulnerabilities
3. Suggest performance improvements
4. Ensure code follows project conventions

Always respond with structured feedback that the primary agent
can act upon.
```

2. Sub-Agents Respond to Your Primary Agent

Remember: sub-agents communicate with your primary agent, not directly with you. Write prompts accordingly.

3. Be Explicit in Your Instructions

The more specific your instructions, the better your agents perform. Avoid ambiguity:

❌ **Vague**: "Review this code" ✅ **Explicit**: "Review this Python code for SQL injection vulnerabilities, checking all database queries for parameterization"

4. Use the Meta-Agent

The meta-agent is a powerful tool for creating new agents from scratch. Use it to bootstrap your multi-agent system.

5. Chain Sub-Agents for Complex Workflows

```
None
Primary Agent → Code Generator Sub-Agent → Code Review Sub-Agent
→ Test Writer Sub-Agent
```

The "Big 3" of Agentic Coding

As you scale multi-agent systems, three elements become critical:

1. **Context**: What information each agent has access to
2. **Model**: Which AI model powers each agent
3. **Prompt**: How each agent is instructed

The flow and management of these three elements determines the success of your multi-agent system.

Hooks + Sub-Agents: Advanced Integration

Auto-Generate Sub-Agents with Hooks

Use hooks to automatically create specialized sub-agents based on project needs:

```python
#!/usr/bin/env python3
# .claude/hooks/auto_agent_creator.py
import json
import sys
import os

def should_create_agent(tool_input):
    # Check if working with a new framework/library
    file_path = tool_input.get("file_path", "")

    # Detect new patterns that might need specialized agents
    patterns = {
        "react": ["useState", "useEffect", "jsx"],
        "django": ["models.Model", "views", "urls.py"],
        "fastapi": ["FastAPI", "@app.", "BaseModel"],
```

```
    }

    for framework, keywords in patterns.items():
        agent_path = f".claude/agents/{framework}-specialist.md"
        if not os.path.exists(agent_path) and any(kw in
str(tool_input) for kw in keywords):
            create_specialist_agent(framework, agent_path)
            print(f"🤖 Created {framework} specialist agent!")

def create_specialist_agent(framework, path):
    template = f"""---
name: {framework}-specialist
description: Expert in {framework} development patterns and best
practices
---

You are a {framework} specialist. Help the primary agent with:
- {framework}-specific patterns and idioms
- Performance optimizations
- Security best practices
- Testing strategies

Always provide {framework}-idiomatic solutions.
"""
    os.makedirs(os.path.dirname(path), exist_ok=True)
    with open(path, 'w') as f:
        f.write(template)

if __name__ == "__main__":
    data = json.load(sys.stdin)
    if data.get("tool_name") in ["Edit", "Write"]:
        should_create_agent(data.get("tool_input", {}))
```

Advanced Sub-Agent Patterns

Proactive Triggering

Configure agents to automatically perform tasks when conditions are met:

```
None
# Auto-Summary Sub-Agent
When called, check if:
- More than 10 files have been modified
- More than 500 lines of code changed
- A major refactoring occurred
- Session duration exceeds 30 minutes

If any condition is true, automatically generate:
1. Executive summary of changes
2. Technical details of modifications
3. Next steps and recommendations
4. Any identified technical debt

Format the summary for easy sharing with team members.
```

Hook Integration for Auto-Summary:

```
JSON
{
  "hooks": {
    "Stop": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "if [ $(git diff --stat | tail -1 | awk
'{print $4}') -gt 500 ]; then echo 'Major changes detected.
Consider running /agent:summarizer'; fi"
      }]
    }]
  }
```

```
}
```

Information-Dense Keywords

Claude Code provides special keywords and variables to give agents more context:

Environment Variables:

- `$CLAUDE_PROJECT_DIR` - Project root directory
- `$CLAUDE_FILE_PATHS` - Currently relevant files
- `$CLAUDE_TOOL_NAME` - Active tool being used
- `$CLAUDE_SESSION_ID` - Current session identifier

In Sub-Agent Prompts:

- Reference current state with `{{current_files}}`
- Access recent changes with `{{recent_changes}}`
- Include project context with `{{project_context}}`

Example Usage:

```
None
Review the changes in {{recent_changes}} and ensure they follow
the patterns established in {{project_context}}. Pay special
attention to files matching {{current_files}} pattern.
```

Sub-Agent Configuration with Hooks

Combine sub-agents with hooks for powerful automation:

```
JSON
{
  "hooks": {
    "SubagentStop": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
```

```
        "command": "echo 'Sub-agent completed: Review the output
for quality' | tee -a .claude/subagent-log.txt"
      }]
    }]
  }
}
```

Monitoring Sub-Agent Performance

Create a hook to track sub-agent execution times and success rates:

Python

```python
#!/usr/bin/env python3
import json
import sys
import time
from datetime import datetime

def log_subagent_performance():
    data = json.load(sys.stdin)

    # Extract sub-agent info from context
    timestamp = datetime.now().isoformat()
    session_id = data.get("session_id", "unknown")

    # Log performance metrics
    log_entry = {
        "timestamp": timestamp,
        "session_id": session_id,
        "event": "subagent_complete",
        "duration": time.time() - float(data.get("start_time",
time.time()))
    }

    with open(".claude/subagent-metrics.jsonl", "a") as f:
        f.write(json.dumps(log_entry) + "\n")
```

```
    # Alert if sub-agent took too long
    if log_entry["duration"] > 300:  # 5 minutes
        print("⚠️ Sub-agent took longer than expected!")

if __name__ == "__main__":
    log_subagent_performance()
```

Important Warnings

⚠️ YOLO Mode Caution

YOLO mode gives agents free rein to execute without confirmation. Use with extreme caution and only in safe environments.

🔐 Keep Agents Separate

As you build more agents, maintain clear separation to avoid confusion and keep your codebase clean.

🏝️ Use Isolated Workflows

Create isolated environments for different agent workflows to prevent interference:

- Separate directories for agent-specific files
- Dedicated configuration files
- Independent logging streams
- Isolated test environments

🔄 Don't Overload Sub-Agent Chains

Long chains of sub-agents can become difficult to manage and debug. Keep chains reasonable (3-5 agents max).

🧠 Don't Offload All Cognitive Work

While agents are powerful assistants, maintain your understanding of what they're doing. You need to:

- Understand the code being generated
- Make architectural decisions
- Review and validate agent outputs

- Maintain overall system vision

Agents augment your capabilities—they don't replace your expertise.

Sub-Agent Frontmatter

The description variable is crucial for guiding your primary agent. It's the most important variable after the name:

```
None
---
name: security-auditor
description: |
  Performs comprehensive security audits on code.
  Call this agent when:
  - New authentication code is added
  - File upload functionality is implemented
  - Database queries are modified
  - External API integrations are created

  This agent will return a structured security report with:
  - Identified vulnerabilities (scored by severity)
  - Specific remediation steps
  - Security best practices relevant to the code
tools:
  - read_file
  - search_code
  - analyze_dependencies
---
```

Description Best Practices:

- Be explicit about when to call the agent
- Specify what the agent returns
- List specific triggers or conditions
- Include expected output format

Example: Multi-Agent Development Workflow

```
None
# Primary Agent Instructions
You coordinate a team of specialized sub-agents:

1. **Architect Agent**: Design system architecture
2. **Implementation Agent**: Write code following the
architecture
3. **Test Agent**: Create comprehensive test suites
4. **Documentation Agent**: Generate docs and comments
5. **Review Agent**: Final quality check

For each feature request:
1. Have Architect design the approach
2. Implementation writes the code
3. Test creates tests
4. Documentation updates docs
5. Review ensures quality

Only proceed to the next step after the previous agent confirms
completion.
```

Hooks vs Sub-Agents: When to Use Each

Use Hooks When:

- You need deterministic, automated actions
- The task is simple and well-defined
- You want to enforce policies or safety checks
- Speed is critical (hooks are faster)
- You need to integrate with external tools

Use Sub-Agents When:

- The task requires AI reasoning or creativity
- You need specialized domain expertise
- The problem is complex and requires context
- You want reusable, shareable components
- You need natural language processing

Combining Both:

```JSON
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "python3
.claude/hooks/check_if_needs_security_agent.py"
      }]
    }]
  }
}
```

This hook could analyze commands and automatically suggest calling the security sub-agent when needed.

Practical Integration Example:

```Python
#!/usr/bin/env python3
# Hook that suggests relevant sub-agents based on activity
import json
import sys

AGENT_TRIGGERS = {
    "security-auditor": ["password", "auth", "token", "api_key"],
    "performance-optimizer": ["slow", "optimize", "benchmark"],
    "test-writer": ["test", "spec", "coverage"],
}

def suggest_agents(tool_input):
    suggestions = []
    content = str(tool_input).lower()

    for agent, triggers in AGENT_TRIGGERS.items():
```

```
        if any(trigger in content for trigger in triggers):
            suggestions.append(agent)

    if suggestions:
        print(f"💡 Consider using these sub-agents: {',
'.join(suggestions)}")

if __name__ == "__main__":
    data = json.load(sys.stdin)
    if data.get("tool_name") in ["Edit", "Write"]:
        suggest_agents(data.get("tool_input", {}))
```

---

16. Professional Development Patterns {#professional-patterns}

The Claude Code Gap: From Scripts to Systems

While Claude Code excels at generating code quickly, it often skips critical professional development steps. This works fine for small scripts but falls apart for enterprise-level applications. Professional developers don't just write code—they plan, architect, design, test, and secure their systems.

Professional Workflow Stages

Here's how to structure Claude Code for serious development:

1. Project Planning & Architecture

```
None
# Command: /project:architect-plan "Design a scalable e-commerce
platform"

Perform comprehensive project planning:
1. Define system requirements and constraints
2. Create high-level architecture diagrams
3. Design database schema
```

```
4. Plan API structure
5. Define security protocols
6. Create testing strategy
7. Estimate scalability needs (users, data, traffic)

Output a structured architecture document with:
- System overview
- Component breakdown
- Data flow diagrams
- Security considerations
- Scalability assessment
```

2. Frontend Development with Standards

```
None
# Command: /project:frontend-tdd "Implement user dashboard"

Develop frontend with professional standards:
1. Review UI/UX designs or create wireframes
2. Set up component architecture
3. Write tests FIRST (TDD approach)
4. Implement components
5. Ensure accessibility (WCAG compliance)
6. Optimize performance
7. Add proper error handling

Use modern patterns:
- React Query for server state
- Proper state management
- Component composition
- Performance optimization
```

3. Backend Development with TDD

```
None
# Command: /project:backend-secure "Build authentication system"

Professional backend development:
1. Design API contracts first
2. Write integration tests
3. Implement with security in mind
4. Add comprehensive error handling
5. Include logging and monitoring
6. Document all endpoints
7. Performance profiling

Follow patterns:
- Repository pattern for data access
- Service layer for business logic
- Proper validation and sanitization
- Rate limiting and security headers
```

Essential Personas for Professional Development

Create specialized sub-agents for each role:

System Architect

```
None
---
name: system-architect
description: |
  Enterprise architecture specialist focusing on:
  - Scalable system design
  - Technology selection
  - Integration patterns
  - Performance architecture
  - Security architecture

  Call when planning new systems or major refactors
```

```
---
```

Security Engineer

```
None
---
name: security-engineer
description: |
  Security specialist for:
  - Vulnerability assessment
  - Security architecture review
  - Penetration testing strategies
  - Compliance requirements
  - Security best practices

  Call for any security-sensitive features
---
```

Performance Engineer

```
None
---
name: performance-engineer
description: |
  Performance optimization expert for:
  - Bottleneck identification
  - Query optimization
  - Caching strategies
  - Load testing
  - Scalability planning
---
```

MCP Tools for Professional Development

Enhance Claude Code with specialized MCP servers:

1. **Sequential Thinking**: Complex multi-step problem solving
2. **Context7**: Access official documentation and libraries
3. **Magic**: Modern UI component generation
4. **Playwright**: Automated testing and browser automation

Professional Troubleshooting Workflow

Step 1: Problem Identification

```
None
# Command: /project:investigate "Production errors increasing"

Investigate the issue:
1. Analyze error logs
2. Check monitoring dashboards
3. Review recent deployments
4. Identify patterns
5. Document symptoms
```

Step 2: Root Cause Analysis

```
None
# Command: /project:five-whys "API timeout errors"

Apply Five Whys methodology:
1. Why is the API timing out?
2. Why is the database query slow?
3. Why is the index not being used?
4. Why was the index dropped?
5. Why wasn't this caught in testing?

Output: Root cause and remediation plan
```

Step 3: Performance Analysis

```
None
# Command: /project:profile "Analyze slow endpoints"
```

```
Performance profiling:
1. Run performance profiler
2. Identify bottlenecks
3. Analyze database queries
4. Check memory usage
5. Review caching effectiveness
6. Suggest optimizations
```

Architecture Analysis Hook

Create a hook that triggers architecture analysis for significant changes:

```json
{
  "hooks": {
    "Stop": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "python3 .claude/hooks/architecture_check.py"
      }]
    }]
  }
}
```

Architecture Check Script:

```python
#!/usr/bin/env python3
import subprocess
import json

def check_architecture_impact():
    # Check changed files
```

```
    changed = subprocess.check_output(['git', 'diff',
'--name-only'], text=True)

    critical_patterns = [
        'schema', 'migration', 'auth', 'security',
        'api/', 'architecture', 'config'
    ]

    if any(pattern in changed.lower() for pattern in
critical_patterns):
        print("\n🏗️ ARCHITECTURE IMPACT DETECTED")
        print("Consider running: /project:architect-review")
        print("Critical files changed that may affect system
architecture")

        # Auto-generate mini architecture report
        with open('.claude/architecture-changes.md', 'w') as f:
            f.write(f"# Architecture Impact Analysis\n\n")
            f.write(f"## Changed Files\n{changed}\n")
            f.write(f"## Recommended Actions\n")
            f.write(f"- Review system architecture\n")
            f.write(f"- Update documentation\n")
            f.write(f"- Check security implications\n")
            f.write(f"- Verify scalability impact\n")

if __name__ == "__main__":
    check_architecture_impact()
```

Quality Gates with Hooks

Implement quality gates that enforce professional standards:

JSON

```
{
  "hooks": {
```

```
    "PreToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": ".claude/hooks/quality_gate.sh"
      }]
    }]
  }
}
```

Quality Gate Script:

```shell
#!/bin/bash
# .claude/hooks/quality_gate.sh

# Check if tests exist for new code
if [[ "$CLAUDE_FILE_PATHS" =~ \.(ts|js|py)$ ]]; then
    base_name=$(basename "$CLAUDE_FILE_PATHS" | sed
's/\.[^.]*$//')
    test_file=$(find . -name "*${base_name}*test*" -o -name
"*${base_name}*spec*" | head -1)

    if [ -z "$test_file" ]; then
        echo "⚠️ WARNING: No tests found for $CLAUDE_FILE_PATHS"
>&2
        echo "Consider TDD: Write tests before implementation"
>&2
        # Don't block, just warn
        exit 0
    fi
fi
```

Scalability Assessment Patterns

Create commands that analyze scalability at different stages:

```
None
# Command: /project:scale-assessment "Current system analysis"

Perform scalability assessment:
1. Current capacity (concurrent users, data volume)
2. Identify bottlenecks:
   - Database connections
   - API rate limits
   - Memory usage
   - Third-party service limits
3. Growth projections
4. Scaling recommendations:
   - Horizontal vs vertical scaling
   - Caching strategies
   - Database optimization
   - Service decomposition
5. Cost analysis

Output scaling roadmap with priorities
```

Professional Development Checklist

Use this checklist for every serious project:

- [ ] Architecture documented before coding
- [ ] Security review completed
- [ ] API contracts defined
- [ ] Test strategy in place
- [ ] Performance benchmarks set
- [ ] Monitoring configured
- [ ] Documentation updated
- [ ] Code review process defined
- [ ] Deployment strategy planned
- [ ] Rollback procedures ready

Integration Example: Full Professional Workflow

```shell
# 1. Start with architecture
/project:architect-plan "Build a real-time collaboration tool"

# 2. Security review
/project:security-review "Review authentication approach"

# 3. Frontend with TDD
/project:frontend-tdd "Implement collaborative editor"

# 4. Backend with security
/project:backend-secure "Build WebSocket server"

# 5. Performance testing
/project:performance-test "Load test with 1000 concurrent users"

# 6. Deploy with confidence
/project:deploy-checklist "Production deployment readiness"
```

This professional approach transforms Claude Code from a code generator into a comprehensive development platform that follows enterprise best practices.

Conclusion

Claude Code hooks and sub-agents represent a paradigm shift in AI-assisted development. By combining deterministic automation (hooks) with intelligent reasoning (sub-agents), you create a development environment that's both powerful and predictable.

Your Journey: From Hooks to Multi-Agent Systems

🚀 Phase 1: Basic Automation (Week 1)

- Start with simple notification hooks
- Add basic logging and debugging hooks
- Create your first safety checks
- Convert repetitive prompts to slash commands

🛡️ Phase 2: Safety and Quality (Week 2-3)

- Implement comprehensive command validation
- Set up automatic formatting and linting
- Add pre-commit style checks
- Create file protection rules

🔧 **Phase 3: Advanced Workflows (Month 2)**

- Build multi-tool pattern matching
- Integrate with your CI/CD pipeline
- Set up observability dashboards
- Create project-specific automations

🤖 **Phase 4: Multi-Agent Systems (Month 3+)**

- Design your first specialized sub-agents
- Build agent chains for complex tasks
- Implement proactive triggering
- Create meta-agents for agent generation

Key Takeaways

1. **Start Simple**: Begin with basic hooks and gradually increase complexity
2. **Security First**: Always review hook code and use restrictive permissions
3. **Community Wisdom**: Use slash commands instead of copy-paste, prefer new sessions over clearing
4. **Combine Tools**: Use Claude Code with other AI tools for enhanced learning
5. **Think Systems**: As you scale, focus on the flow of context, model, and prompts
6. **Read the Docs**: The official Claude Code documentation at docs.anthropic.com is your best resource for detailed information and updates

The Future of Development

With hooks and sub-agents, you're not just using an AI assistant—you're building an intelligent development platform tailored to your exact needs. Every hook you write, every sub-agent you create, and every workflow you automate brings you closer to a future where repetitive tasks disappear and creativity flourishes.

Remember the Professional Gap: Claude Code's strength in rapid code generation can be a weakness for complex projects. Use the patterns in this guide to add the planning, architecture, security, and testing phases that professional development demands.

The goal isn't to replace human judgment but to augment it. Use these tools to:

- Eliminate toil and repetitive tasks
- Enforce best practices automatically

- Add professional development stages
- Create quality gates and checkpoints
- Free yourself to focus on architecture and problem-solving

Whether you're building a simple script or an enterprise application, the combination of hooks, sub-agents, and professional workflows ensures your development process scales with your ambitions.

Happy automating, and may your hooks always exit cleanly! 🚀🎣🤖, r'.ts

---

5. Best Practices and Debugging {#best-practices}

🔍 Debug First

Always start by logging the JSON payload to understand the data structure:

```JSON
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "jq '.' >> .claude/hooks/log.jsonl"
      }]
    }]
  }
}
```

📁 Project Organization

- Keep `.claude/settings.json` in your project repository
- Store scripts in `.claude/hooks/` directory
- Make hooks shareable and version-controlled

🛡️ Error Handling

- Hooks should fail silently unless designed to block actions

- Exit with code 0 for normal operation
- Exit with code 2 to block an action and show an error

📦 Dependency Management

For Python scripts with dependencies, use **Astral's uv**:

```shell
uv run my_script.py
```

This handles environment setup automatically.

🎯 Key Principles

1. **Start simple**: Test with basic logging before complex logic
2. **Be defensive**: Handle missing data and errors gracefully
3. **Keep it fast**: Hooks run synchronously - avoid slow operations
4. **Document well**: Comment your scripts for future reference

💡 Pro Tips from the Community

1. **Session Management**: Instead of clearing sessions, start a new one. You can resume previous sessions with `claude --resume` for better context preservation
2. **Use Slash Commands**: Rather than copying and pasting prompts (like security checks), create custom slash commands for reusable workflows
3. **Multi-Tool Learning**: Use Claude Code alongside other tools like Cursor IDE - run Claude Code for tasks while using Cursor's AI to understand the code being generated
4. **Learning Workflow**: Create a learning loop by analyzing Claude's output in other AI tools to deepen your understanding of generated code

---

6. Advanced Hook Examples {#advanced-examples}

Multi-Tool Pattern Matching

Goal: Apply different actions based on file types across multiple tools.

```JSON
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|MultiEdit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "if [[ \"$CLAUDE_FILE_PATHS\" =~ \\.(py)$
]]; then black $CLAUDE_FILE_PATHS && ruff check --fix
$CLAUDE_FILE_PATHS; fi"
          },
          {
            "type": "command",
            "command": "if [[ \"$CLAUDE_FILE_PATHS\" =~
\\.(ts|tsx)$ ]]; then prettier --write $CLAUDE_FILE_PATHS && npx
tsc --noEmit $CLAUDE_FILE_PATHS; fi"
          }
        ]
      }
    ]
  }
}
```

Session Context Loading

Goal: Automatically load project context when Claude Code starts.

Tip: Instead of clearing sessions when things get messy, start a new session and use `claude --resume` to restore previous context when needed. This preserves your conversation history for reference.

Script (`.claude/hooks/session_start.py`):

Python

```
#!/usr/bin/env python3
import json
import sys
import subprocess

def load_context():
    context = []

    # Git status
    try:
        git_status = subprocess.check_output(['git', 'status',
'--short'], text=True)
        if git_status:
            context.append(f"Current git changes:\n{git_status}")
    except:
        pass

    # Recent commits
    try:
        commits = subprocess.check_output(['git', 'log',
'--oneline', '-5'], text=True)
        context.append(f"Recent commits:\n{commits}")
    except:
        pass

    # TODO items
    try:
        todos = subprocess.check_output(['grep', '-r', 'TODO:',
'.', '--include=*.py'], text=True)
        if todos:
            context.append(f"TODO items
found:\n{todos[:500]}...")
    except:
        pass
```

```
    # Output context for Claude
    if context:
        print("\n=== PROJECT CONTEXT ===")
        print("\n".join(context))
        print("========================\n")

if __name__ == "__main__":
    load_context()
```

Configuration:

```JSON
{
  "hooks": {
    "SessionStart": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "python3 .claude/hooks/session_start.py"
      }]
    }]
  }
}
```

Advanced Command Validation with AI Feedback

Goal: Use AI to analyze potentially dangerous commands before execution.

```Python
#!/usr/bin/env python3
import json
import sys
import re
```

```
DANGEROUS_PATTERNS = [
    (r'rm\s+-[^\s]*[rf]', "Recursive deletion detected"),
    (r'chmod\s+777', "Overly permissive file permissions"),
    (r'curl.*\|\s*sh', "Piping curl to shell - potential security
risk"),
    (r'npm\s+install.*--global', "Global npm install detected"),
    (r'docker.*--privileged', "Privileged Docker container"),
]

def analyze_command(command):
    risks = []
    for pattern, message in DANGEROUS_PATTERNS:
        if re.search(pattern, command, re.IGNORECASE):
            risks.append(message)

    if risks:
        # Exit code 2 blocks execution and feeds stderr to Claude
        print(json.dumps({
            "error": "Command blocked due to security concerns",
            "risks": risks,
            "command": command,
            "suggestion": "Please use a safer alternative or
confirm this is intentional"
        }), file=sys.stderr)
        sys.exit(2)

if __name__ == "__main__":
    data = json.load(sys.stdin)
    if data.get("tool_name") == "Bash":
        command = data.get("tool_input", {}).get("command", "")
        analyze_command(command)
```

---

7. Troubleshooting Common Issues {#troubleshooting}

Hook Not Executing

Problem: Your hooks aren't firing when expected.

Solutions:

1. **Check file permissions**:

```shell
chmod +x .claude/hooks/*.py
```

2. **Verify JSON syntax**:

```shell
jq '.' .claude/settings.json
```

3. **Use absolute paths** (recommended by documentation):

```json
{
  "command":
"/absolute/path/to/your/project/.claude/hooks/script.py"
}
```

4. **Debug with logging**:

```JSON
{
  "command": "echo 'Hook fired at $(date)' >>
/tmp/claude-hook-debug.log"
}
```

Settings Not Updating

Problem: Changes to settings.json don't take effect.

Solutions:

1. **Restart Claude Code** - Settings are loaded at startup
2. **Check for syntax errors** in your JSON
3. **Ensure you're editing the correct file**:
    - Project-specific: `.claude/settings.json`
    - User-global: `~/.claude/settings.json`

Hook Blocking Unintentionally

Problem: Hook exits with non-zero code accidentally.

Solution: Always handle errors gracefully:

```Python
try:
    # Your hook logic
    pass
except Exception as e:
    # Log error but don't block Claude
    with open('/tmp/hook-errors.log', 'a') as f:
        f.write(f"Error: {e}\n")
    sys.exit(0)  # Exit successfully
```

Performance Issues

Problem: Hooks slow down Claude's operations.

Solutions:

1. **Use background execution** for non-critical hooks:

```
JSON
{
  "type": "command",
  "command": "your_command &"
}
```

2. **Set appropriate timeouts**:

```
JSON
{
  "type": "command",
  "command": "your_command",
  "timeout": 5
}
```

3. **Optimize scripts** - avoid heavy operations in synchronous hooks

---

8. Quick Reference Cheat Sheet {#cheat-sheet}

Hook Events

Event	Purpose	Can Block?	Key Data
`UserPromptSubmit`	Modify user prompts	No	`prompt`
`SessionStart`	Load context	No	`session_id`, `source`
`PreToolUse`	Validate/block tools	Yes (exit 2)	`tool_name`, `tool_input`
`PostToolUse`	Process results	No	`tool_response`
`Notification`	Custom alerts	No	`message`
`Stop`	Task completion	No	`transcript_path`
`SubagentStop`	Subtask completion	Yes (exit 2)	`stop_hook_active`
`PreCompact`	Before compaction	No	`trigger`

Exit Codes

- **0**: Success (stdout shown in transcript mode)
- **2**: Block operation (stderr sent to Claude)
- **Other**: Non-blocking error (stderr shown to user)

Environment Variables

```shell
$CLAUDE_PROJECT_DIR    # Absolute path to project root
$CLAUDE_FILE_PATHS     # Space-separated file paths
$CLAUDE_TOOL_NAME      # Name of the tool being used
$CLAUDE_NOTIFICATION   # Notification message content
```

Common Patterns

```JSON
// Match all tools
"matcher": ""

// Match specific tools
"matcher": "Bash"

// Match multiple tools
"matcher": "Edit|Write|MultiEdit"

// Match MCP tools
"matcher": "mcp__github__*"
```

Session Management

- **Start fresh**: Use new sessions instead of clearing when context gets messy
- **Resume sessions**: `claude --resume` to restore previous context
- **Headless mode**: `claude -p "prompt" --output-format stream-json` for automation
- **Non-interactive**: `claude -p "prompt" --no-interactive` for CI/CD

---

9. Integration with Development Workflows {#integrations}

Git Workflow Integration

Pre-commit Hook Integration:

```JSON
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
```

```
        "command": "git add $CLAUDE_FILE_PATHS &&
./.git/hooks/pre-commit --files $CLAUDE_FILE_PATHS || true"
      }]
    }]
  }
}
```

CI/CD Integration

Headless Mode for Automation:

Shell

```
# Run Claude Code in CI pipeline
claude -p "Fix all linting errors in src/" --output-format
stream-json

# Pre-push validation
claude -p "Run all tests and fix any failures" --no-interactive
```

IDE Integration

VSCode Tasks Integration:

JSON

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit",
      "hooks": [{
        "type": "command",
        "command": "code --goto $CLAUDE_FILE_PATHS:1"
      }]
    }]
  }
}
```

Docker Development

Container Management:

```json
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "python3 -c \"import json,sys;
d=json.load(sys.stdin);
cmd=d.get('tool_input',{}).get('command',''); sys.exit(2 if
'docker' in cmd and '--rm' not in cmd else 0)\""
      }]
    }]
  }
}
```

---

10. Security Considerations {#security}

Key Security Principles

1. **Hooks run with your credentials** - They have full access to your system
2. **Review all hook code** before enabling it
3. **Never run untrusted hooks** from unknown sources
4. **Use restrictive permissions** on hook scripts
5. **Validate all inputs** in your hook scripts

Secure Hook Template

Python

```python
#!/usr/bin/env python3
import json
import sys
import os
import re

# Whitelist approach for allowed operations
ALLOWED_COMMANDS = [
    r'^ls\s',
    r'^cat\s',
    r'^grep\s',
    r'^find\s.*-name',
]

FORBIDDEN_PATHS = [
    '.env',
    '.ssh/',
    'credentials',
    'secrets',
    '.git/config',
]

def validate_command(command):
    # Check against whitelist
    if not any(re.match(pattern, command) for pattern in
ALLOWED_COMMANDS):
```

```
        return False, "Command not in allowlist"

    # Check for forbidden paths
    for path in FORBIDDEN_PATHS:
        if path in command:
            return False, f"Access to {path} is forbidden"

    return True, ""

def main():
    try:
        data = json.load(sys.stdin)
        if data.get("tool_name") == "Bash":
            command = data.get("tool_input", {}).get("command",
"")
            valid, reason = validate_command(command)

            if not valid:
                print(f"Security block: {reason}",
file=sys.stderr)
                sys.exit(2)
    except Exception as e:
        # Fail open - don't block on errors
        pass

if __name__ == "__main__":
    main()
```

---

11. Multi-Agent Observability {#observability}

Setting Up Observability Dashboard

Goal: Monitor multiple Claude Code agents in real-time.

Architecture:

```
None
Claude Agents → Hooks → HTTP Server → SQLite → WebSocket →
Dashboard
```

Hook Configuration:

```
JSON
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "python3 .claude/hooks/send_event.py --event
PreToolUse --agent $USER"
      }]
    }],
    "PostToolUse": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "python3 .claude/hooks/send_event.py --event
PostToolUse --agent $USER --include-output"
      }]
    }]
  }
}
```

Event Sender Script:

```python
#!/usr/bin/env python3
import json
import sys
import requests
import argparse
from datetime import datetime

def send_event(event_type, agent_name, include_output=False):
    data = json.load(sys.stdin)

    event = {
        "timestamp": datetime.utcnow().isoformat(),
        "agent": agent_name,
        "event_type": event_type,
        "tool_name": data.get("tool_name"),
        "tool_input": data.get("tool_input"),
    }

    if include_output:
        event["tool_output"] = data.get("tool_response",
{}).get("output", "")[:500]

    try:
        requests.post("http://localhost:8080/events", json=event,
timeout=1)
    except:
        pass  # Don't block Claude if observability is down

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--event", required=True)
    parser.add_argument("--agent", required=True)
    parser.add_argument("--include-output", action="store_true")
    args = parser.parse_args()
```

```
    send_event(args.event, args.agent, args.include_output)
```

---

12. Custom Slash Commands {#slash-commands}

Creating Custom Commands

Custom slash commands extend Claude Code's functionality through natural language templates. **Pro tip**: Instead of repeatedly copying and pasting prompts (like security checks or review templates), convert them into slash commands for instant reuse.

Location: `.claude/commands/` (project) or `~/.claude/commands/` (global)

Example: `.claude/commands/review-pr.md`

```
None
Please review the changes in the current git branch:

1. First, check what branch we're on with `git branch
--show-current`
2. Show the diff with `git diff main...HEAD`
3. Analyze the changes for:
   - Potential bugs or logic errors
   - Security vulnerabilities
   - Performance issues
   - Missing error handling
4. Suggest improvements with specific code examples
5. Check if tests are needed for the changes

Focus on actual issues, not style preferences.
```

Example: Convert Frequent Prompts to Commands:
`.claude/commands/security-check.md`

```
None
Perform a comprehensive security audit on the codebase:

1. Check for hardcoded credentials or API keys
2. Review authentication and authorization logic
3. Look for SQL injection vulnerabilities
4. Check for XSS vulnerabilities in frontend code
5. Review file upload handling for security issues
6. Check dependency versions for known vulnerabilities
7. Review error handling to ensure no sensitive data leaks

Provide specific recommendations for each issue found.
```

Tip: If you find yourself copying and pasting the same prompt repeatedly, it's time to make it a slash command!

Example with Arguments: `.claude/commands/fix-issue.md`

```
None
Please fix GitHub issue: $ARGUMENTS

Steps:
1. Use `gh issue view $ARGUMENTS` to read the issue
2. Identify the root cause
3. Find relevant files with grep/find
4. Implement the fix
5. Write tests for the fix
6. Verify all tests pass
7. Create a commit with message "Fix #$ARGUMENTS: [description]"
```

Usage:

```
Shell
/project:review-pr
/project:fix-issue 123
```

---

13. Working with MCP Tools {#mcp-tools}

Hook Integration with MCP

Model Context Protocol (MCP) tools appear with special naming patterns in hooks and provide specialized capabilities.

Essential MCP Servers

Core MCP Tools:

1. **Sequential**: Enables complex multi-step thinking and problem-solving
2. **Context7**: Grabs official libraries and documentation from online sources
3. **Magic**: Generates modern UI components with best practices
4. **Playwright**: Browser automation and testing capabilities
5. **Memory**: Persistent context storage across sessions
6. **Filesystem**: Enhanced file operations
7. **GitHub**: Direct GitHub API integration

Installing MCP Servers

```
Shell
# Add MCP servers to Claude Code
claude mcp add sequential
claude mcp add context7
claude mcp add magic
claude mcp add playwright

# Check connected MCP servers
claude
# Then use: /mcp
```

MCP Tool Naming Patterns

MCP tools follow the pattern `mcp__<server>__<tool>`:

- `mcp__memory__create_entities`
- `mcp__filesystem__read_file`
- `mcp__github__search_repositories`
- `mcp__sequential__think_step_by_step`
- `mcp__context7__fetch_documentation`
- `mcp__magic__generate_component`

Example Hooks for MCP Tools

JSON

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "mcp__github__*",
      "hooks": [{
        "type": "command",
        "command": "echo '[$(date)] GitHub MCP action:
$CLAUDE_TOOL_NAME' >> ~/.claude/mcp-audit.log"
      }]
    }],
    "PostToolUse": [{
      "matcher": "mcp__filesystem__write_file",
      "hooks": [{
        "type": "command",
        "command": "git add $CLAUDE_FILE_PATHS && echo
'Auto-staged changes from MCP filesystem write'"
      }]
    }]
  }
}
```

---

14. Learning and Development Workflows {#learning-workflows}

Using Claude Code with Other AI Tools

Multi-AI Workflow: Combine Claude Code with other AI assistants for enhanced learning and development.

Example: Claude Code + Cursor IDE

1. **Run Claude Code** for implementing features or fixes
2. **Use Cursor's AI** to ask questions about the generated code:
    - "Explain what this function does"
    - "List everything in the .claude directory and its purpose"
    - "What design patterns are being used here?"
3. **Deep dive with Claude Desktop** for complex concepts you don't understand

Creating a Personal Learning Assistant

Build a custom GPT or AI assistant trained on:

- Your preferred learning style
- Your tech stack and domain knowledge
- Your coding conventions

Then feed it Claude Code's output for personalized explanations and learning paths.

Hook for Learning Capture

JSON

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "echo '## Code Change\nTool:
$CLAUDE_TOOL_NAME\nFiles: $CLAUDE_FILE_PATHS\n' >>
.claude/learning-log.md"
      }]
    }]
  }
```

```
}
```

This creates a learning log you can review to understand what Claude did and why.

---

15. Sub-Agents and Multi-Agent Systems {#sub-agents}

Introduction to Sub-Agents

Sub-agents in Claude Code are specialized AI assistants that can be called by your primary Claude agent to handle specific tasks. They enable modular, scalable workflows by dividing complex problems into specialized domains.

Creating Sub-Agents

Getting Started:

1. Open the sub-agents interface with `/agents`
2. Select "Create New Agent"
3. Choose scope:
    - **Project-level**: Available only in current project
    - **User-level**: Available across all your projects
4. Edit the system prompt (press Enter to use your editor)
5. Save and use

Key Features

🎯 Context Preservation

Each sub-agent operates in its own context, preventing pollution of the main conversation and keeping focus on specific objectives.

🔧 Specialized Expertise

Sub-agents can be fine-tuned with detailed instructions for specific domains, leading to higher success rates on designated tasks.

♻️ Reusability

Once created, sub-agents can be used across different projects and shared with your team for consistent workflows.

🔒 **Flexible Permissions**

Each sub-agent can have different tool access levels, allowing you to limit powerful tools to specific agent types.

Sub-Agent Best Practices

1. System Prompt is Key

With sub-agents, you write the system prompt, not the user prompt. This gives you more control over the agent's behavior.

```
None
# Example Sub-Agent System Prompt for Code Review
You are a specialized code review agent. Your primary agent will
send you code changes to review.

Your responsibilities:
1. Check for bugs and logic errors
2. Identify security vulnerabilities
3. Suggest performance improvements
4. Ensure code follows project conventions

Always respond with structured feedback that the primary agent
can act upon.
```

2. Sub-Agents Respond to Your Primary Agent

Remember: sub-agents communicate with your primary agent, not directly with you. Write prompts accordingly.

3. Be Explicit in Your Instructions

The more specific your instructions, the better your agents perform. Avoid ambiguity:

❌ **Vague**: "Review this code" ✅ **Explicit**: "Review this Python code for SQL injection vulnerabilities, checking all database queries for parameterization"

4. Use the Meta-Agent

The meta-agent is a powerful tool for creating new agents from scratch. Use it to bootstrap your multi-agent system.

5. Chain Sub-Agents for Complex Workflows

```
None
Primary Agent → Code Generator Sub-Agent → Code Review Sub-Agent
→ Test Writer Sub-Agent
```

The "Big 3" of Agentic Coding

As you scale multi-agent systems, three elements become critical:

1. **Context**: What information each agent has access to
2. **Model**: Which AI model powers each agent
3. **Prompt**: How each agent is instructed

The flow and management of these three elements determines the success of your multi-agent system.

Hooks + Sub-Agents: Advanced Integration

Auto-Generate Sub-Agents with Hooks

Use hooks to automatically create specialized sub-agents based on project needs:

```python
#!/usr/bin/env python3
# .claude/hooks/auto_agent_creator.py
import json
import sys
import os

def should_create_agent(tool_input):
    # Check if working with a new framework/library
    file_path = tool_input.get("file_path", "")

    # Detect new patterns that might need specialized agents
    patterns = {
        "react": ["useState", "useEffect", "jsx"],
        "django": ["models.Model", "views", "urls.py"],
        "fastapi": ["FastAPI", "@app.", "BaseModel"],
```

```
    }

    for framework, keywords in patterns.items():
        agent_path = f".claude/agents/{framework}-specialist.md"
        if not os.path.exists(agent_path) and any(kw in
str(tool_input) for kw in keywords):
            create_specialist_agent(framework, agent_path)
            print(f"🤖 Created {framework} specialist agent!")

def create_specialist_agent(framework, path):
    template = f"""---
name: {framework}-specialist
description: Expert in {framework} development patterns and best
practices
---

You are a {framework} specialist. Help the primary agent with:
- {framework}-specific patterns and idioms
- Performance optimizations
- Security best practices
- Testing strategies

Always provide {framework}-idiomatic solutions.
"""
    os.makedirs(os.path.dirname(path), exist_ok=True)
    with open(path, 'w') as f:
        f.write(template)

if __name__ == "__main__":
    data = json.load(sys.stdin)
    if data.get("tool_name") in ["Edit", "Write"]:
        should_create_agent(data.get("tool_input", {}))
```

Advanced Sub-Agent Patterns

Proactive Triggering

Configure agents to automatically perform tasks when conditions are met:

```
None
# Auto-Summary Sub-Agent
When called, check if:
- More than 10 files have been modified
- More than 500 lines of code changed
- A major refactoring occurred
- Session duration exceeds 30 minutes

If any condition is true, automatically generate:
1. Executive summary of changes
2. Technical details of modifications
3. Next steps and recommendations
4. Any identified technical debt

Format the summary for easy sharing with team members.
```

Hook Integration for Auto-Summary:

```
JSON
{
  "hooks": {
    "Stop": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "if [ $(git diff --stat | tail -1 | awk
'{print $4}') -gt 500 ]; then echo 'Major changes detected.
Consider running /agent:summarizer'; fi"
      }]
    }]
  }
```

```
}
```

Information-Dense Keywords

Claude Code provides special keywords and variables to give agents more context:

Environment Variables:

- `$CLAUDE_PROJECT_DIR` - Project root directory
- `$CLAUDE_FILE_PATHS` - Currently relevant files
- `$CLAUDE_TOOL_NAME` - Active tool being used
- `$CLAUDE_SESSION_ID` - Current session identifier

In Sub-Agent Prompts:

- Reference current state with `{{current_files}}`
- Access recent changes with `{{recent_changes}}`
- Include project context with `{{project_context}}`

Example Usage:

```
None
Review the changes in {{recent_changes}} and ensure they follow
the patterns established in {{project_context}}. Pay special
attention to files matching {{current_files}} pattern.
```

Sub-Agent Configuration with Hooks

Combine sub-agents with hooks for powerful automation:

```
JSON
{
  "hooks": {
    "SubagentStop": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
```

```
        "command": "echo 'Sub-agent completed: Review the output
for quality' | tee -a .claude/subagent-log.txt"
      }]
    }]
  }
}
```

Monitoring Sub-Agent Performance

Create a hook to track sub-agent execution times and success rates:

Python

```python
#!/usr/bin/env python3
import json
import sys
import time
from datetime import datetime

def log_subagent_performance():
    data = json.load(sys.stdin)

    # Extract sub-agent info from context
    timestamp = datetime.now().isoformat()
    session_id = data.get("session_id", "unknown")

    # Log performance metrics
    log_entry = {
        "timestamp": timestamp,
        "session_id": session_id,
        "event": "subagent_complete",
        "duration": time.time() - float(data.get("start_time",
time.time()))
    }

    with open(".claude/subagent-metrics.jsonl", "a") as f:
        f.write(json.dumps(log_entry) + "\n")
```

```
    # Alert if sub-agent took too long
    if log_entry["duration"] > 300:  # 5 minutes
        print("⚠️ Sub-agent took longer than expected!")

if __name__ == "__main__":
    log_subagent_performance()
```

Important Warnings

⚠️ YOLO Mode Caution

YOLO mode gives agents free rein to execute without confirmation. Use with extreme caution and only in safe environments.

🔐 Keep Agents Separate

As you build more agents, maintain clear separation to avoid confusion and keep your codebase clean.

🏝️ Use Isolated Workflows

Create isolated environments for different agent workflows to prevent interference:

- Separate directories for agent-specific files
- Dedicated configuration files
- Independent logging streams
- Isolated test environments

🔄 Don't Overload Sub-Agent Chains

Long chains of sub-agents can become difficult to manage and debug. Keep chains reasonable (3-5 agents max).

🧠 Don't Offload All Cognitive Work

While agents are powerful assistants, maintain your understanding of what they're doing. You need to:

- Understand the code being generated
- Make architectural decisions
- Review and validate agent outputs

- Maintain overall system vision

Agents augment your capabilities—they don't replace your expertise.

Sub-Agent Frontmatter

The description variable is crucial for guiding your primary agent. It's the most important variable after the name:

```
None
---
name: security-auditor
description: |
  Performs comprehensive security audits on code.
  Call this agent when:
  - New authentication code is added
  - File upload functionality is implemented
  - Database queries are modified
  - External API integrations are created

  This agent will return a structured security report with:
  - Identified vulnerabilities (scored by severity)
  - Specific remediation steps
  - Security best practices relevant to the code
tools:
  - read_file
  - search_code
  - analyze_dependencies
---
```

Description Best Practices:

- Be explicit about when to call the agent
- Specify what the agent returns
- List specific triggers or conditions
- Include expected output format

Example: Multi-Agent Development Workflow

```
None
# Primary Agent Instructions
You coordinate a team of specialized sub-agents:

1. **Architect Agent**: Design system architecture
2. **Implementation Agent**: Write code following the
architecture
3. **Test Agent**: Create comprehensive test suites
4. **Documentation Agent**: Generate docs and comments
5. **Review Agent**: Final quality check

For each feature request:
1. Have Architect design the approach
2. Implementation writes the code
3. Test creates tests
4. Documentation updates docs
5. Review ensures quality

Only proceed to the next step after the previous agent confirms
completion.
```

Hooks vs Sub-Agents: When to Use Each

Use Hooks When:

- You need deterministic, automated actions
- The task is simple and well-defined
- You want to enforce policies or safety checks
- Speed is critical (hooks are faster)
- You need to integrate with external tools

Use Sub-Agents When:

- The task requires AI reasoning or creativity
- You need specialized domain expertise
- The problem is complex and requires context
- You want reusable, shareable components

- You need natural language processing

Combining Both:

```JSON
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "python3
.claude/hooks/check_if_needs_security_agent.py"
      }]
    }]
  }
}
```

This hook could analyze commands and automatically suggest calling the security sub-agent when needed.

Practical Integration Example:

```Python
#!/usr/bin/env python3
# Hook that suggests relevant sub-agents based on activity
import json
import sys

AGENT_TRIGGERS = {
    "security-auditor": ["password", "auth", "token", "api_key"],
    "performance-optimizer": ["slow", "optimize", "benchmark"],
    "test-writer": ["test", "spec", "coverage"],
}

def suggest_agents(tool_input):
    suggestions = []
    content = str(tool_input).lower()
```

```
    for agent, triggers in AGENT_TRIGGERS.items():
        if any(trigger in content for trigger in triggers):
            suggestions.append(agent)

    if suggestions:
        print(f"💡 Consider using these sub-agents: {',
'.join(suggestions)}")

if __name__ == "__main__":
    data = json.load(sys.stdin)
    if data.get("tool_name") in ["Edit", "Write"]:
        suggest_agents(data.get("tool_input", {}))
```

---

16. Professional Development Patterns {#professional-patterns}

The Claude Code Gap: From Scripts to Systems

While Claude Code excels at generating code quickly, it often skips critical professional development steps. This works fine for small scripts but falls apart for enterprise-level applications. Professional developers don't just write code—they plan, architect, design, test, and secure their systems.

Professional Workflow Stages

Here's how to structure Claude Code for serious development:

1. Project Planning & Architecture

```
None
# Command: /project:architect-plan "Design a scalable e-commerce
platform"

Perform comprehensive project planning:
1. Define system requirements and constraints
2. Create high-level architecture diagrams
```

```
3. Design database schema
4. Plan API structure
5. Define security protocols
6. Create testing strategy
7. Estimate scalability needs (users, data, traffic)

Output a structured architecture document with:
- System overview
- Component breakdown
- Data flow diagrams
- Security considerations
- Scalability assessment
```

2. Frontend Development with Standards

```
None
# Command: /project:frontend-tdd "Implement user dashboard"

Develop frontend with professional standards:
1. Review UI/UX designs or create wireframes
2. Set up component architecture
3. Write tests FIRST (TDD approach)
4. Implement components
5. Ensure accessibility (WCAG compliance)
6. Optimize performance
7. Add proper error handling

Use modern patterns:
- React Query for server state
- Proper state management
- Component composition
- Performance optimization
```

3. Backend Development with TDD

```
None
# Command: /project:backend-secure "Build authentication system"

Professional backend development:
1. Design API contracts first
2. Write integration tests
3. Implement with security in mind
4. Add comprehensive error handling
5. Include logging and monitoring
6. Document all endpoints
7. Performance profiling

Follow patterns:
- Repository pattern for data access
- Service layer for business logic
- Proper validation and sanitization
- Rate limiting and security headers
```

Essential Personas for Professional Development

Create specialized sub-agents for each role:

System Architect

```
None
---
name: system-architect
description: |
  Enterprise architecture specialist focusing on:
  - Scalable system design
  - Technology selection
  - Integration patterns
  - Performance architecture
  - Security architecture

  Call when planning new systems or major refactors
---
```

Security Engineer

```
None
---
name: security-engineer
description: |
  Security specialist for:
  - Vulnerability assessment
  - Security architecture review
  - Penetration testing strategies
  - Compliance requirements
  - Security best practices

  Call for any security-sensitive features
---
```

Performance Engineer

```
None
---
name: performance-engineer
description: |
  Performance optimization expert for:
  - Bottleneck identification
  - Query optimization
  - Caching strategies
  - Load testing
  - Scalability planning
---
```

MCP Tools for Professional Development

Enhance Claude Code with specialized MCP servers:

1. **Sequential Thinking**: Complex multi-step problem solving
2. **Context7**: Access official documentation and libraries
3. **Magic**: Modern UI component generation
4. **Playwright**: Automated testing and browser automation

Professional Troubleshooting Workflow

Step 1: Problem Identification

```
None
# Command: /project:investigate "Production errors increasing"

Investigate the issue:
1. Analyze error logs
2. Check monitoring dashboards
3. Review recent deployments
4. Identify patterns
5. Document symptoms
```

Step 2: Root Cause Analysis

```
None
# Command: /project:five-whys "API timeout errors"

Apply Five Whys methodology:
1. Why is the API timing out?
2. Why is the database query slow?
3. Why is the index not being used?
4. Why was the index dropped?
5. Why wasn't this caught in testing?

Output: Root cause and remediation plan
```

Step 3: Performance Analysis

```
None
# Command: /project:profile "Analyze slow endpoints"

Performance profiling:
1. Run performance profiler
2. Identify bottlenecks
3. Analyze database queries
4. Check memory usage
```

```
5. Review caching effectiveness
6. Suggest optimizations
```

Architecture Analysis Hook

Create a hook that triggers architecture analysis for significant changes:

JSON
```
{
  "hooks": {
    "Stop": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "python3 .claude/hooks/architecture_check.py"
      }]
    }]
  }
}
```

Architecture Check Script:

Python
```
#!/usr/bin/env python3
import subprocess
import json

def check_architecture_impact():
    # Check changed files
    changed = subprocess.check_output(['git', 'diff',
'--name-only'], text=True)
```

```
    critical_patterns = [
        'schema', 'migration', 'auth', 'security',
        'api/', 'architecture', 'config'
    ]

    if any(pattern in changed.lower() for pattern in
critical_patterns):
        print("\n🏗️ ARCHITECTURE IMPACT DETECTED")
        print("Consider running: /project:architect-review")
        print("Critical files changed that may affect system
architecture")

        # Auto-generate mini architecture report
        with open('.claude/architecture-changes.md', 'w') as f:
            f.write(f"# Architecture Impact Analysis\n\n")
            f.write(f"## Changed Files\n{changed}\n")
            f.write(f"## Recommended Actions\n")
            f.write(f"- Review system architecture\n")
            f.write(f"- Update documentation\n")
            f.write(f"- Check security implications\n")
            f.write(f"- Verify scalability impact\n")

if __name__ == "__main__":
    check_architecture_impact()
```

Quality Gates with Hooks

Implement quality gates that enforce professional standards:

JSON

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Edit|Write",
```

```
      "hooks": [{
        "type": "command",
        "command": ".claude/hooks/quality_gate.sh"
      }]
    }]
  }
}
```

Quality Gate Script:

Shell

```
#!/bin/bash
# .claude/hooks/quality_gate.sh

# Check if tests exist for new code
if [[ "$CLAUDE_FILE_PATHS" =~ \.(ts|js|py)$ ]]; then
    base_name=$(basename "$CLAUDE_FILE_PATHS" | sed
's/\.[^.]*$//')
    test_file=$(find . -name "*${base_name}*test*" -o -name
"*${base_name}*spec*" | head -1)

    if [ -z "$test_file" ]; then
        echo "⚠️ WARNING: No tests found for $CLAUDE_FILE_PATHS"
>&2
        echo "Consider TDD: Write tests before implementation"
>&2
        # Don't block, just warn
        exit 0
    fi
fi
```

Scalability Assessment Patterns

Create commands that analyze scalability at different stages:

```
None
# Command: /project:scale-assessment "Current system analysis"

Perform scalability assessment:
1. Current capacity (concurrent users, data volume)
2. Identify bottlenecks:
   - Database connections
   - API rate limits
   - Memory usage
   - Third-party service limits
3. Growth projections
4. Scaling recommendations:
   - Horizontal vs vertical scaling
   - Caching strategies
   - Database optimization
   - Service decomposition
5. Cost analysis

Output scaling roadmap with priorities
```

Professional Development Checklist

Use this checklist for every serious project:

- [ ] Architecture documented before coding
- [ ] Security review completed
- [ ] API contracts defined
- [ ] Test strategy in place
- [ ] Performance benchmarks set
- [ ] Monitoring configured
- [ ] Documentation updated
- [ ] Code review process defined
- [ ] Deployment strategy planned
- [ ] Rollback procedures ready

Integration Example: Full Professional Workflow

```shell
# 1. Start with architecture
/project:architect-plan "Build a real-time collaboration tool"

# 2. Security review
/project:security-review "Review authentication approach"

# 3. Frontend with TDD
/project:frontend-tdd "Implement collaborative editor"

# 4. Backend with security
/project:backend-secure "Build WebSocket server"

# 5. Performance testing
/project:performance-test "Load test with 1000 concurrent users"

# 6. Deploy with confidence
/project:deploy-checklist "Production deployment readiness"
```

This professional approach transforms Claude Code from a code generator into a comprehensive development platform that follows enterprise best practices.

Conclusion

Claude Code hooks and sub-agents represent a paradigm shift in AI-assisted development. By combining deterministic automation (hooks) with intelligent reasoning (sub-agents), you create a development environment that's both powerful and predictable.

Your Journey: From Hooks to Multi-Agent Systems

🚀 Phase 1: Basic Automation (Week 1)

- Start with simple notification hooks
- Add basic logging and debugging hooks
- Create your first safety checks
- Convert repetitive prompts to slash commands

🛡️ **Phase 2: Safety and Quality (Week 2-3)**

- Implement comprehensive command validation
- Set up automatic formatting and linting
- Add pre-commit style checks
- Create file protection rules

🔧 **Phase 3: Advanced Workflows (Month 2)**

- Build multi-tool pattern matching
- Integrate with your CI/CD pipeline
- Set up observability dashboards
- Create project-specific automations

🤖 **Phase 4: Multi-Agent Systems (Month 3+)**

- Design your first specialized sub-agents
- Build agent chains for complex tasks
- Implement proactive triggering
- Create meta-agents for agent generation

Key Takeaways

1. **Start Simple**: Begin with basic hooks and gradually increase complexity
2. **Security First**: Always review hook code and use restrictive permissions
3. **Community Wisdom**: Use slash commands instead of copy-paste, prefer new sessions over clearing
4. **Combine Tools**: Use Claude Code with other AI tools for enhanced learning
5. **Think Systems**: As you scale, focus on the flow of context, model, and prompts
6. **Read the Docs**: The official Claude Code documentation at docs.anthropic.com is your best resource for detailed information and updates

The Future of Development

With hooks and sub-agents, you're not just using an AI assistant—you're building an intelligent development platform tailored to your exact needs. Every hook you write, every sub-agent you create, and every workflow you automate brings you closer to a future where repetitive tasks disappear and creativity flourishes.

Remember the Professional Gap: Claude Code's strength in rapid code generation can be a weakness for complex projects. Use the patterns in this guide to add the planning, architecture, security, and testing phases that professional development demands.

The goal isn't to replace human judgment but to augment it. Use these tools to:

- Eliminate toil and repetitive tasks

- Enforce best practices automatically
- Add professional development stages
- Create quality gates and checkpoints
- Free yourself to focus on architecture and problem-solving

Whether you're building a simple script or an enterprise application, the combination of hooks, sub-agents, and professional workflows ensures your development process scales with your ambitions.

Happy automating, and may your hooks always exit cleanly! 🚀🎣🤖, r'.java

---

5. Best Practices and Debugging {#best-practices}

🔍 Debug First

Always start by logging the JSON payload to understand the data structure:

```JSON
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "jq '.' >> .claude/hooks/log.jsonl"
      }]
    }]
  }
}
```

📁 Project Organization

- Keep `.claude/settings.json` in your project repository
- Store scripts in `.claude/hooks/` directory
- Make hooks shareable and version-controlled

🛡️ Error Handling

- Hooks should fail silently unless designed to block actions
- Exit with code 0 for normal operation
- Exit with code 2 to block an action and show an error

📦 Dependency Management

For Python scripts with dependencies, use **Astral's uv**:

```shell
uv run my_script.py
```

This handles environment setup automatically.

🎯 Key Principles

1. **Start simple**: Test with basic logging before complex logic
2. **Be defensive**: Handle missing data and errors gracefully
3. **Keep it fast**: Hooks run synchronously - avoid slow operations
4. **Document well**: Comment your scripts for future reference

💡 Pro Tips from the Community

1. **Session Management**: Instead of clearing sessions, start a new one. You can resume previous sessions with `claude --resume` for better context preservation
2. **Use Slash Commands**: Rather than copying and pasting prompts (like security checks), create custom slash commands for reusable workflows
3. **Multi-Tool Learning**: Use Claude Code alongside other tools like Cursor IDE - run Claude Code for tasks while using Cursor's AI to understand the code being generated
4. **Learning Workflow**: Create a learning loop by analyzing Claude's output in other AI tools to deepen your understanding of generated code

---

6. Advanced Hook Examples {#advanced-examples}

Multi-Tool Pattern Matching

Goal: Apply different actions based on file types across multiple tools.

JSON

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|MultiEdit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "if [[ \"$CLAUDE_FILE_PATHS\" =~ \\.(py)$
]]; then black $CLAUDE_FILE_PATHS && ruff check --fix
$CLAUDE_FILE_PATHS; fi"
          },
          {
            "type": "command",
            "command": "if [[ \"$CLAUDE_FILE_PATHS\" =~
\\.(ts|tsx)$ ]]; then prettier --write $CLAUDE_FILE_PATHS && npx
tsc --noEmit $CLAUDE_FILE_PATHS; fi"
          }
        ]
      }
    ]
  }
}
```

Session Context Loading

Goal: Automatically load project context when Claude Code starts.

Tip: Instead of clearing sessions when things get messy, start a new session and use `claude --resume` to restore previous context when needed. This preserves your conversation history for reference.

Script (`.claude/hooks/session_start.py`):

Python

```
#!/usr/bin/env python3
import json
import sys
import subprocess

def load_context():
    context = []

    # Git status
    try:
        git_status = subprocess.check_output(['git', 'status',
'--short'], text=True)
        if git_status:
            context.append(f"Current git changes:\n{git_status}")
    except:
        pass

    # Recent commits
    try:
        commits = subprocess.check_output(['git', 'log',
'--oneline', '-5'], text=True)
        context.append(f"Recent commits:\n{commits}")
    except:
        pass

    # TODO items
    try:
        todos = subprocess.check_output(['grep', '-r', 'TODO:',
'.', '--include=*.py'], text=True)
        if todos:
            context.append(f"TODO items
found:\n{todos[:500]}...")
    except:
        pass
```

```
    # Output context for Claude
    if context:
        print("\n=== PROJECT CONTEXT ===")
        print("\n".join(context))
        print("========================\n")

if __name__ == "__main__":
    load_context()
```

Configuration:

```JSON
{
  "hooks": {
    "SessionStart": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "python3 .claude/hooks/session_start.py"
      }]
    }]
  }
}
```

Advanced Command Validation with AI Feedback

Goal: Use AI to analyze potentially dangerous commands before execution.

```Python
#!/usr/bin/env python3
import json
import sys
import re
```

```
DANGEROUS_PATTERNS = [
    (r'rm\s+-[^\s]*[rf]', "Recursive deletion detected"),
    (r'chmod\s+777', "Overly permissive file permissions"),
    (r'curl.*\|\s*sh', "Piping curl to shell - potential security
risk"),
    (r'npm\s+install.*--global', "Global npm install detected"),
    (r'docker.*--privileged', "Privileged Docker container"),
]

def analyze_command(command):
    risks = []
    for pattern, message in DANGEROUS_PATTERNS:
        if re.search(pattern, command, re.IGNORECASE):
            risks.append(message)

    if risks:
        # Exit code 2 blocks execution and feeds stderr to Claude
        print(json.dumps({
            "error": "Command blocked due to security concerns",
            "risks": risks,
            "command": command,
            "suggestion": "Please use a safer alternative or
confirm this is intentional"
        }), file=sys.stderr)
        sys.exit(2)

if __name__ == "__main__":
    data = json.load(sys.stdin)
    if data.get("tool_name") == "Bash":
        command = data.get("tool_input", {}).get("command", "")
        analyze_command(command)
```

---

7. Troubleshooting Common Issues {#troubleshooting}

Hook Not Executing

Problem: Your hooks aren't firing when expected.

Solutions:

1. **Check file permissions**:

```
Shell
chmod +x .claude/hooks/*.py
```

2. **Verify JSON syntax**:

```
Shell
jq '.' .claude/settings.json
```

3. **Use absolute paths** (recommended by documentation):

```
JSON
{
  "command":
"/absolute/path/to/your/project/.claude/hooks/script.py"
}
```

4. **Debug with logging**:

```
JSON
{
```

```
  "command": "echo 'Hook fired at $(date)' >>
/tmp/claude-hook-debug.log"
}
```

Settings Not Updating

Problem: Changes to settings.json don't take effect.

Solutions:

1. **Restart Claude Code** - Settings are loaded at startup
2. **Check for syntax errors** in your JSON
3. **Ensure you're editing the correct file**:
    - Project-specific: `.claude/settings.json`
    - User-global: `~/.claude/settings.json`

Hook Blocking Unintentionally

Problem: Hook exits with non-zero code accidentally.

Solution: Always handle errors gracefully:

```python
try:
    # Your hook logic
    pass
except Exception as e:
    # Log error but don't block Claude
    with open('/tmp/hook-errors.log', 'a') as f:
        f.write(f"Error: {e}\n")
    sys.exit(0)  # Exit successfully
```

Performance Issues

Problem: Hooks slow down Claude's operations.

Solutions:

1. **Use background execution** for non-critical hooks:

```
JSON
{
  "type": "command",
  "command": "your_command &"
}
```

2. **Set appropriate timeouts**:

```
JSON
{
  "type": "command",
  "command": "your_command",
  "timeout": 5
}
```

3. **Optimize scripts** - avoid heavy operations in synchronous hooks

---

8. Quick Reference Cheat Sheet {#cheat-sheet}

Hook Events

Event	Purpose	Can Block?	Key Data
`UserPromptSubmit`	Modify user prompts	No	`prompt`
`SessionStart`	Load context	No	`session_id`, `source`
`PreToolUse`	Validate/block tools	Yes (exit 2)	`tool_name`, `tool_input`
`PostToolUse`	Process results	No	`tool_response`
`Notification`	Custom alerts	No	`message`
`Stop`	Task completion	No	`transcript_path`
`SubagentStop`	Subtask completion	Yes (exit 2)	`stop_hook_active`
`PreCompact`	Before compaction	No	`trigger`

Exit Codes

- **0**: Success (stdout shown in transcript mode)
- **2**: Block operation (stderr sent to Claude)
- **Other**: Non-blocking error (stderr shown to user)

Environment Variables

```shell
$CLAUDE_PROJECT_DIR    # Absolute path to project root
$CLAUDE_FILE_PATHS     # Space-separated file paths
$CLAUDE_TOOL_NAME      # Name of the tool being used
$CLAUDE_NOTIFICATION   # Notification message content
```

Common Patterns

```JSON
// Match all tools
"matcher": ""

// Match specific tools
"matcher": "Bash"

// Match multiple tools
"matcher": "Edit|Write|MultiEdit"

// Match MCP tools
"matcher": "mcp__github__*"
```

Session Management

- **Start fresh**: Use new sessions instead of clearing when context gets messy
- **Resume sessions**: `claude --resume` to restore previous context
- **Headless mode**: `claude -p "prompt" --output-format stream-json` for automation
- **Non-interactive**: `claude -p "prompt" --no-interactive` for CI/CD

---

9. Integration with Development Workflows {#integrations}

Git Workflow Integration

Pre-commit Hook Integration:

```JSON
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
```

```
      "hooks": [{
        "type": "command",
        "command": "git add $CLAUDE_FILE_PATHS &&
./.git/hooks/pre-commit --files $CLAUDE_FILE_PATHS || true"
      }]
    }]
  }
}
```

CI/CD Integration

Headless Mode for Automation:

```shell
# Run Claude Code in CI pipeline
claude -p "Fix all linting errors in src/" --output-format
stream-json

# Pre-push validation
claude -p "Run all tests and fix any failures" --no-interactive
```

IDE Integration

VSCode Tasks Integration:

```json
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit",
      "hooks": [{
        "type": "command",
        "command": "code --goto $CLAUDE_FILE_PATHS:1"
      }]
    }]
```

```
  }
}
```

Docker Development

Container Management:

JSON

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "python3 -c \"import json,sys;
d=json.load(sys.stdin);
cmd=d.get('tool_input',{}).get('command',''); sys.exit(2 if
'docker' in cmd and '--rm' not in cmd else 0)\""
      }]
    }]
  }
}
```

---

10. Security Considerations {#security}

Key Security Principles

1. **Hooks run with your credentials** - They have full access to your system
2. **Review all hook code** before enabling it
3. **Never run untrusted hooks** from unknown sources
4. **Use restrictive permissions** on hook scripts
5. **Validate all inputs** in your hook scripts

Secure Hook Template

Python

```
#!/usr/bin/env python3
import json
import sys
import os
import re

# Whitelist approach for allowed operations
ALLOWED_COMMANDS = [
    r'^ls\s',
    r'^cat\s',
    r'^grep\s',
    r'^find\s.*-name',
]

FORBIDDEN_PATHS = [
    '.env',
    '.ssh/',
    'credentials',
    'secrets',
    '.git/config',
]

def validate_command(command):
    # Check against whitelist
    if not any(re.match(pattern, command) for pattern in
ALLOWED_COMMANDS):
        return False, "Command not in allowlist"

    # Check for forbidden paths
    for path in FORBIDDEN_PATHS:
        if path in command:
            return False, f"Access to {path} is forbidden"

    return True, ""
```

```
def main():
    try:
        data = json.load(sys.stdin)
        if data.get("tool_name") == "Bash":
            command = data.get("tool_input", {}).get("command",
"")
            valid, reason = validate_command(command)

            if not valid:
                print(f"Security block: {reason}",
file=sys.stderr)
                sys.exit(2)
    except Exception as e:
        # Fail open - don't block on errors
        pass

if __name__ == "__main__":
    main()
```

---

11. Multi-Agent Observability {#observability}

Setting Up Observability Dashboard

Goal: Monitor multiple Claude Code agents in real-time.

Architecture:

```
None
Claude Agents → Hooks → HTTP Server → SQLite → WebSocket →
Dashboard
```

Hook Configuration:

```
JSON
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "python3 .claude/hooks/send_event.py --event
PreToolUse --agent $USER"
      }]
    }],
    "PostToolUse": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "python3 .claude/hooks/send_event.py --event
PostToolUse --agent $USER --include-output"
      }]
    }]
  }
}
```

Event Sender Script:

```python
#!/usr/bin/env python3
import json
import sys
import requests
import argparse
from datetime import datetime

def send_event(event_type, agent_name, include_output=False):
    data = json.load(sys.stdin)

    event = {
        "timestamp": datetime.utcnow().isoformat(),
        "agent": agent_name,
        "event_type": event_type,
        "tool_name": data.get("tool_name"),
        "tool_input": data.get("tool_input"),
    }

    if include_output:
        event["tool_output"] = data.get("tool_response",
{}).get("output", "")[:500]

    try:
        requests.post("http://localhost:8080/events", json=event,
timeout=1)
    except:
        pass  # Don't block Claude if observability is down

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--event", required=True)
    parser.add_argument("--agent", required=True)
    parser.add_argument("--include-output", action="store_true")
    args = parser.parse_args()
```

```
    send_event(args.event, args.agent, args.include_output)
```

---

12. Custom Slash Commands {#slash-commands}

Creating Custom Commands

Custom slash commands extend Claude Code's functionality through natural language templates. **Pro tip**: Instead of repeatedly copying and pasting prompts (like security checks or review templates), convert them into slash commands for instant reuse.

Location: `.claude/commands/` (project) or `~/.claude/commands/` (global)

Example: `.claude/commands/review-pr.md`

```
None
Please review the changes in the current git branch:

1. First, check what branch we're on with `git branch
--show-current`
2. Show the diff with `git diff main...HEAD`
3. Analyze the changes for:
   - Potential bugs or logic errors
   - Security vulnerabilities
   - Performance issues
   - Missing error handling
4. Suggest improvements with specific code examples
5. Check if tests are needed for the changes

Focus on actual issues, not style preferences.
```

Example: Convert Frequent Prompts to Commands:
`.claude/commands/security-check.md`

```
None
Perform a comprehensive security audit on the codebase:

1. Check for hardcoded credentials or API keys
2. Review authentication and authorization logic
3. Look for SQL injection vulnerabilities
4. Check for XSS vulnerabilities in frontend code
5. Review file upload handling for security issues
6. Check dependency versions for known vulnerabilities
7. Review error handling to ensure no sensitive data leaks

Provide specific recommendations for each issue found.
```

Tip: If you find yourself copying and pasting the same prompt repeatedly, it's time to make it a slash command!

Example with Arguments: `.claude/commands/fix-issue.md`

```
None
Please fix GitHub issue: $ARGUMENTS

Steps:
1. Use `gh issue view $ARGUMENTS` to read the issue
2. Identify the root cause
3. Find relevant files with grep/find
4. Implement the fix
5. Write tests for the fix
6. Verify all tests pass
7. Create a commit with message "Fix #$ARGUMENTS: [description]"
```

Usage:

```
Shell
/project:review-pr
/project:fix-issue 123
```

---

13. Working with MCP Tools {#mcp-tools}

Hook Integration with MCP

Model Context Protocol (MCP) tools appear with special naming patterns in hooks and provide specialized capabilities.

Essential MCP Servers

Core MCP Tools:

1. **Sequential**: Enables complex multi-step thinking and problem-solving
2. **Context7**: Grabs official libraries and documentation from online sources
3. **Magic**: Generates modern UI components with best practices
4. **Playwright**: Browser automation and testing capabilities
5. **Memory**: Persistent context storage across sessions
6. **Filesystem**: Enhanced file operations
7. **GitHub**: Direct GitHub API integration

Installing MCP Servers

```shell
# Add MCP servers to Claude Code
claude mcp add sequential
claude mcp add context7
claude mcp add magic
claude mcp add playwright

# Check connected MCP servers
claude
# Then use: /mcp
```

MCP Tool Naming Patterns

MCP tools follow the pattern `mcp__<server>__<tool>`:

- `mcp__memory__create_entities`
- `mcp__filesystem__read_file`
- `mcp__github__search_repositories`
- `mcp__sequential__think_step_by_step`
- `mcp__context7__fetch_documentation`

- `mcp__magic__generate_component`

Example Hooks for MCP Tools

JSON

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "mcp__github__*",
      "hooks": [{
        "type": "command",
        "command": "echo '[$(date)] GitHub MCP action:
$CLAUDE_TOOL_NAME' >> ~/.claude/mcp-audit.log"
      }]
    }],
    "PostToolUse": [{
      "matcher": "mcp__filesystem__write_file",
      "hooks": [{
        "type": "command",
        "command": "git add $CLAUDE_FILE_PATHS && echo
'Auto-staged changes from MCP filesystem write'"
      }]
    }]
  }
}
```

---

14. Learning and Development Workflows {#learning-workflows}

Using Claude Code with Other AI Tools

Multi-AI Workflow: Combine Claude Code with other AI assistants for enhanced learning and development.

Example: Claude Code + Cursor IDE

1. **Run Claude Code** for implementing features or fixes
2. **Use Cursor's AI** to ask questions about the generated code:
    - "Explain what this function does"
    - "List everything in the .claude directory and its purpose"
    - "What design patterns are being used here?"
3. **Deep dive with Claude Desktop** for complex concepts you don't understand

Creating a Personal Learning Assistant

Build a custom GPT or AI assistant trained on:

- Your preferred learning style
- Your tech stack and domain knowledge
- Your coding conventions

Then feed it Claude Code's output for personalized explanations and learning paths.

Hook for Learning Capture

JSON

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "echo '## Code Change\nTool:
$CLAUDE_TOOL_NAME\nFiles: $CLAUDE_FILE_PATHS\n' >>
.claude/learning-log.md"
      }]
    }]
  }
```

```
}
```

This creates a learning log you can review to understand what Claude did and why.

---

15. Sub-Agents and Multi-Agent Systems {#sub-agents}

Introduction to Sub-Agents

Sub-agents in Claude Code are specialized AI assistants that can be called by your primary Claude agent to handle specific tasks. They enable modular, scalable workflows by dividing complex problems into specialized domains.

Creating Sub-Agents

Getting Started:

1. Open the sub-agents interface with `/agents`
2. Select "Create New Agent"
3. Choose scope:
    - **Project-level**: Available only in current project
    - **User-level**: Available across all your projects
4. Edit the system prompt (press Enter to use your editor)
5. Save and use

Key Features

🎯 Context Preservation

Each sub-agent operates in its own context, preventing pollution of the main conversation and keeping focus on specific objectives.

🔧 Specialized Expertise

Sub-agents can be fine-tuned with detailed instructions for specific domains, leading to higher success rates on designated tasks.

♻️ Reusability

Once created, sub-agents can be used across different projects and shared with your team for consistent workflows.

🔒 **Flexible Permissions**

Each sub-agent can have different tool access levels, allowing you to limit powerful tools to specific agent types.

Sub-Agent Best Practices

1. System Prompt is Key

With sub-agents, you write the system prompt, not the user prompt. This gives you more control over the agent's behavior.

```
None
# Example Sub-Agent System Prompt for Code Review
You are a specialized code review agent. Your primary agent will
send you code changes to review.

Your responsibilities:
1. Check for bugs and logic errors
2. Identify security vulnerabilities
3. Suggest performance improvements
4. Ensure code follows project conventions

Always respond with structured feedback that the primary agent
can act upon.
```

2. Sub-Agents Respond to Your Primary Agent

Remember: sub-agents communicate with your primary agent, not directly with you. Write prompts accordingly.

3. Be Explicit in Your Instructions

The more specific your instructions, the better your agents perform. Avoid ambiguity:

❌ **Vague**: "Review this code" ✅ **Explicit**: "Review this Python code for SQL injection vulnerabilities, checking all database queries for parameterization"

4. Use the Meta-Agent

The meta-agent is a powerful tool for creating new agents from scratch. Use it to bootstrap your multi-agent system.

5. Chain Sub-Agents for Complex Workflows

```
None
Primary Agent → Code Generator Sub-Agent → Code Review Sub-Agent
→ Test Writer Sub-Agent
```

The "Big 3" of Agentic Coding

As you scale multi-agent systems, three elements become critical:

1. **Context**: What information each agent has access to
2. **Model**: Which AI model powers each agent
3. **Prompt**: How each agent is instructed

The flow and management of these three elements determines the success of your multi-agent system.

Hooks + Sub-Agents: Advanced Integration

Auto-Generate Sub-Agents with Hooks

Use hooks to automatically create specialized sub-agents based on project needs:

```python
#!/usr/bin/env python3
# .claude/hooks/auto_agent_creator.py
import json
import sys
import os

def should_create_agent(tool_input):
    # Check if working with a new framework/library
    file_path = tool_input.get("file_path", "")

    # Detect new patterns that might need specialized agents
    patterns = {
        "react": ["useState", "useEffect", "jsx"],
        "django": ["models.Model", "views", "urls.py"],
        "fastapi": ["FastAPI", "@app.", "BaseModel"],
```

```
    }

    for framework, keywords in patterns.items():
        agent_path = f".claude/agents/{framework}-specialist.md"
        if not os.path.exists(agent_path) and any(kw in
str(tool_input) for kw in keywords):
            create_specialist_agent(framework, agent_path)
            print(f"🤖 Created {framework} specialist agent!")

def create_specialist_agent(framework, path):
    template = f"""---
name: {framework}-specialist
description: Expert in {framework} development patterns and best
practices
---

You are a {framework} specialist. Help the primary agent with:
- {framework}-specific patterns and idioms
- Performance optimizations
- Security best practices
- Testing strategies

Always provide {framework}-idiomatic solutions.
"""
    os.makedirs(os.path.dirname(path), exist_ok=True)
    with open(path, 'w') as f:
        f.write(template)

if __name__ == "__main__":
    data = json.load(sys.stdin)
    if data.get("tool_name") in ["Edit", "Write"]:
        should_create_agent(data.get("tool_input", {}))
```

Advanced Sub-Agent Patterns

Proactive Triggering

Configure agents to automatically perform tasks when conditions are met:

```
None
# Auto-Summary Sub-Agent
When called, check if:
- More than 10 files have been modified
- More than 500 lines of code changed
- A major refactoring occurred
- Session duration exceeds 30 minutes

If any condition is true, automatically generate:
1. Executive summary of changes
2. Technical details of modifications
3. Next steps and recommendations
4. Any identified technical debt

Format the summary for easy sharing with team members.
```

Hook Integration for Auto-Summary:

```
JSON
{
  "hooks": {
    "Stop": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "if [ $(git diff --stat | tail -1 | awk
'{print $4}') -gt 500 ]; then echo 'Major changes detected.
Consider running /agent:summarizer'; fi"
      }]
    }]
  }
```

```
}
```

Information-Dense Keywords

Claude Code provides special keywords and variables to give agents more context:

Environment Variables:

- `$CLAUDE_PROJECT_DIR` - Project root directory
- `$CLAUDE_FILE_PATHS` - Currently relevant files
- `$CLAUDE_TOOL_NAME` - Active tool being used
- `$CLAUDE_SESSION_ID` - Current session identifier

In Sub-Agent Prompts:

- Reference current state with `{{current_files}}`
- Access recent changes with `{{recent_changes}}`
- Include project context with `{{project_context}}`

Example Usage:

```
None
Review the changes in {{recent_changes}} and ensure they follow
the patterns established in {{project_context}}. Pay special
attention to files matching {{current_files}} pattern.
```

Sub-Agent Configuration with Hooks

Combine sub-agents with hooks for powerful automation:

JSON

```
{
  "hooks": {
    "SubagentStop": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "echo 'Sub-agent completed: Review the output
for quality' | tee -a .claude/subagent-log.txt"
      }]
    }]
  }
}
```

Monitoring Sub-Agent Performance

Create a hook to track sub-agent execution times and success rates:

Python

```
#!/usr/bin/env python3
import json
import sys
import time
from datetime import datetime

def log_subagent_performance():
    data = json.load(sys.stdin)

    # Extract sub-agent info from context
    timestamp = datetime.now().isoformat()
    session_id = data.get("session_id", "unknown")

    # Log performance metrics
    log_entry = {
        "timestamp": timestamp,
        "session_id": session_id,
        "event": "subagent_complete",
```

```
        "duration": time.time() - float(data.get("start_time",
time.time()))
    }

    with open(".claude/subagent-metrics.jsonl", "a") as f:
        f.write(json.dumps(log_entry) + "\n")

    # Alert if sub-agent took too long
    if log_entry["duration"] > 300:  # 5 minutes
        print("⚠️ Sub-agent took longer than expected!")

if __name__ == "__main__":
    log_subagent_performance()
```

Important Warnings

⚠️ YOLO Mode Caution

YOLO mode gives agents free rein to execute without confirmation. Use with extreme caution and only in safe environments.

🔐 Keep Agents Separate

As you build more agents, maintain clear separation to avoid confusion and keep your codebase clean.

🏝️ Use Isolated Workflows

Create isolated environments for different agent workflows to prevent interference:

- Separate directories for agent-specific files
- Dedicated configuration files
- Independent logging streams
- Isolated test environments

🔄 Don't Overload Sub-Agent Chains

Long chains of sub-agents can become difficult to manage and debug. Keep chains reasonable (3-5 agents max).

🧠 Don't Offload All Cognitive Work

While agents are powerful assistants, maintain your understanding of what they're doing. You need to:

- Understand the code being generated
- Make architectural decisions
- Review and validate agent outputs
- Maintain overall system vision

Agents augment your capabilities—they don't replace your expertise.

Sub-Agent Frontmatter

The description variable is crucial for guiding your primary agent. It's the most important variable after the name:

```
None
---
name: security-auditor
description: |
  Performs comprehensive security audits on code.
  Call this agent when:
  - New authentication code is added
  - File upload functionality is implemented
  - Database queries are modified
  - External API integrations are created

  This agent will return a structured security report with:
  - Identified vulnerabilities (scored by severity)
  - Specific remediation steps
  - Security best practices relevant to the code
tools:
  - read_file
  - search_code
  - analyze_dependencies
---
```

Description Best Practices:

- Be explicit about when to call the agent
- Specify what the agent returns
- List specific triggers or conditions
- Include expected output format

Example: Multi-Agent Development Workflow

```
None
# Primary Agent Instructions
You coordinate a team of specialized sub-agents:

1. **Architect Agent**: Design system architecture
2. **Implementation Agent**: Write code following the
architecture
3. **Test Agent**: Create comprehensive test suites
4. **Documentation Agent**: Generate docs and comments
5. **Review Agent**: Final quality check

For each feature request:
1. Have Architect design the approach
2. Implementation writes the code
3. Test creates tests
4. Documentation updates docs
5. Review ensures quality

Only proceed to the next step after the previous agent confirms
completion.
```

Hooks vs Sub-Agents: When to Use Each

Use Hooks When:

- You need deterministic, automated actions
- The task is simple and well-defined
- You want to enforce policies or safety checks
- Speed is critical (hooks are faster)
- You need to integrate with external tools

Use Sub-Agents When:

- The task requires AI reasoning or creativity
- You need specialized domain expertise
- The problem is complex and requires context
- You want reusable, shareable components
- You need natural language processing

Combining Both:

```json
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "python3
.claude/hooks/check_if_needs_security_agent.py"
      }]
    }]
  }
}
```

This hook could analyze commands and automatically suggest calling the security sub-agent when needed.

Practical Integration Example:

```python
#!/usr/bin/env python3
# Hook that suggests relevant sub-agents based on activity
import json
import sys

AGENT_TRIGGERS = {
    "security-auditor": ["password", "auth", "token", "api_key"],
    "performance-optimizer": ["slow", "optimize", "benchmark"],
    "test-writer": ["test", "spec", "coverage"],
}
```

```
def suggest_agents(tool_input):
    suggestions = []
    content = str(tool_input).lower()

    for agent, triggers in AGENT_TRIGGERS.items():
        if any(trigger in content for trigger in triggers):
            suggestions.append(agent)

    if suggestions:
        print(f"💡 Consider using these sub-agents: {',
'.join(suggestions)}")

if __name__ == "__main__":
    data = json.load(sys.stdin)
    if data.get("tool_name") in ["Edit", "Write"]:
        suggest_agents(data.get("tool_input", {}))
```

---

16. Professional Development Patterns {#professional-patterns}

The Claude Code Gap: From Scripts to Systems

While Claude Code excels at generating code quickly, it often skips critical professional development steps. This works fine for small scripts but falls apart for enterprise-level applications. Professional developers don't just write code—they plan, architect, design, test, and secure their systems.

Professional Workflow Stages

Here's how to structure Claude Code for serious development:

1. Project Planning & Architecture

```
None
# Command: /project:architect-plan "Design a scalable e-commerce
platform"

Perform comprehensive project planning:
1. Define system requirements and constraints
2. Create high-level architecture diagrams
3. Design database schema
4. Plan API structure
5. Define security protocols
6. Create testing strategy
7. Estimate scalability needs (users, data, traffic)

Output a structured architecture document with:
- System overview
- Component breakdown
- Data flow diagrams
- Security considerations
- Scalability assessment
```

2. Frontend Development with Standards

```
None
# Command: /project:frontend-tdd "Implement user dashboard"

Develop frontend with professional standards:
1. Review UI/UX designs or create wireframes
2. Set up component architecture
3. Write tests FIRST (TDD approach)
4. Implement components
5. Ensure accessibility (WCAG compliance)
6. Optimize performance
7. Add proper error handling

Use modern patterns:
- React Query for server state
- Proper state management
```

```
- Component composition
- Performance optimization
```

3. Backend Development with TDD

```
None
# Command: /project:backend-secure "Build authentication system"

Professional backend development:
1. Design API contracts first
2. Write integration tests
3. Implement with security in mind
4. Add comprehensive error handling
5. Include logging and monitoring
6. Document all endpoints
7. Performance profiling

Follow patterns:
- Repository pattern for data access
- Service layer for business logic
- Proper validation and sanitization
- Rate limiting and security headers
```

Essential Personas for Professional Development

Create specialized sub-agents for each role:

System Architect

```
None
---
name: system-architect
description: |
  Enterprise architecture specialist focusing on:
  - Scalable system design
  - Technology selection
```

```
  - Integration patterns
  - Performance architecture
  - Security architecture

  Call when planning new systems or major refactors
---
```

Security Engineer

```
None
---
name: security-engineer
description: |
  Security specialist for:
  - Vulnerability assessment
  - Security architecture review
  - Penetration testing strategies
  - Compliance requirements
  - Security best practices

  Call for any security-sensitive features
---
```

Performance Engineer

```
None
---
name: performance-engineer
description: |
  Performance optimization expert for:
  - Bottleneck identification
  - Query optimization
  - Caching strategies
  - Load testing
  - Scalability planning
```

```
---
```

MCP Tools for Professional Development

Enhance Claude Code with specialized MCP servers:

1. **Sequential Thinking**: Complex multi-step problem solving
2. **Context7**: Access official documentation and libraries
3. **Magic**: Modern UI component generation
4. **Playwright**: Automated testing and browser automation

Professional Troubleshooting Workflow

Step 1: Problem Identification

```
None
# Command: /project:investigate "Production errors increasing"

Investigate the issue:
1. Analyze error logs
2. Check monitoring dashboards
3. Review recent deployments
4. Identify patterns
5. Document symptoms
```

Step 2: Root Cause Analysis

```
None
# Command: /project:five-whys "API timeout errors"

Apply Five Whys methodology:
1. Why is the API timing out?
2. Why is the database query slow?
3. Why is the index not being used?
4. Why was the index dropped?
5. Why wasn't this caught in testing?
```

```
Output: Root cause and remediation plan
```

Step 3: Performance Analysis

```
None
# Command: /project:profile "Analyze slow endpoints"

Performance profiling:
1. Run performance profiler
2. Identify bottlenecks
3. Analyze database queries
4. Check memory usage
5. Review caching effectiveness
6. Suggest optimizations
```

Architecture Analysis Hook

Create a hook that triggers architecture analysis for significant changes:

```
JSON
{
  "hooks": {
    "Stop": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "python3 .claude/hooks/architecture_check.py"
      }]
    }]
  }
}
```

Architecture Check Script:

```python
#!/usr/bin/env python3
import subprocess
import json

def check_architecture_impact():
    # Check changed files
    changed = subprocess.check_output(['git', 'diff',
'--name-only'], text=True)

    critical_patterns = [
        'schema', 'migration', 'auth', 'security',
        'api/', 'architecture', 'config'
    ]

    if any(pattern in changed.lower() for pattern in
critical_patterns):
        print("\n🏗️ ARCHITECTURE IMPACT DETECTED")
        print("Consider running: /project:architect-review")
        print("Critical files changed that may affect system
architecture")

        # Auto-generate mini architecture report
        with open('.claude/architecture-changes.md', 'w') as f:
            f.write(f"# Architecture Impact Analysis\n\n")
            f.write(f"## Changed Files\n{changed}\n")
            f.write(f"## Recommended Actions\n")
            f.write(f"- Review system architecture\n")
            f.write(f"- Update documentation\n")
            f.write(f"- Check security implications\n")
            f.write(f"- Verify scalability impact\n")

if __name__ == "__main__":
    check_architecture_impact()
```

Quality Gates with Hooks

Implement quality gates that enforce professional standards:

```json
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": ".claude/hooks/quality_gate.sh"
      }]
    }]
  }
}
```

Quality Gate Script:

```shell
#!/bin/bash
# .claude/hooks/quality_gate.sh

# Check if tests exist for new code
if [[ "$CLAUDE_FILE_PATHS" =~ \.(ts|js|py)$ ]]; then
    base_name=$(basename "$CLAUDE_FILE_PATHS" | sed
's/\.[^.]*$//')
    test_file=$(find . -name "*${base_name}*test*" -o -name
"*${base_name}*spec*" | head -1)

    if [ -z "$test_file" ]; then
        echo "⚠️ WARNING: No tests found for $CLAUDE_FILE_PATHS"
>&2
        echo "Consider TDD: Write tests before implementation"
>&2
        # Don't block, just warn
        exit 0
```

```
    fi
fi
```

Scalability Assessment Patterns

Create commands that analyze scalability at different stages:

```
None
# Command: /project:scale-assessment "Current system analysis"

Perform scalability assessment:
1. Current capacity (concurrent users, data volume)
2. Identify bottlenecks:
   - Database connections
   - API rate limits
   - Memory usage
   - Third-party service limits
3. Growth projections
4. Scaling recommendations:
   - Horizontal vs vertical scaling
   - Caching strategies
   - Database optimization
   - Service decomposition
5. Cost analysis

Output scaling roadmap with priorities
```

Professional Development Checklist

Use this checklist for every serious project:

- [ ] Architecture documented before coding
- [ ] Security review completed
- [ ] API contracts defined
- [ ] Test strategy in place
- [ ] Performance benchmarks set
- [ ] Monitoring configured

- [ ] Documentation updated
- [ ] Code review process defined
- [ ] Deployment strategy planned
- [ ] Rollback procedures ready

Integration Example: Full Professional Workflow

```shell
# 1. Start with architecture
/project:architect-plan "Build a real-time collaboration tool"

# 2. Security review
/project:security-review "Review authentication approach"

# 3. Frontend with TDD
/project:frontend-tdd "Implement collaborative editor"

# 4. Backend with security
/project:backend-secure "Build WebSocket server"

# 5. Performance testing
/project:performance-test "Load test with 1000 concurrent users"

# 6. Deploy with confidence
/project:deploy-checklist "Production deployment readiness"
```

This professional approach transforms Claude Code from a code generator into a comprehensive development platform that follows enterprise best practices.

Conclusion

Claude Code hooks and sub-agents represent a paradigm shift in AI-assisted development. By combining deterministic automation (hooks) with intelligent reasoning (sub-agents), you create a development environment that's both powerful and predictable.

Your Journey: From Hooks to Multi-Agent Systems

🚀 Phase 1: Basic Automation (Week 1)

- Start with simple notification hooks
- Add basic logging and debugging hooks
- Create your first safety checks
- Convert repetitive prompts to slash commands

🛡️ **Phase 2: Safety and Quality (Week 2-3)**

- Implement comprehensive command validation
- Set up automatic formatting and linting
- Add pre-commit style checks
- Create file protection rules

🔧 **Phase 3: Advanced Workflows (Month 2)**

- Build multi-tool pattern matching
- Integrate with your CI/CD pipeline
- Set up observability dashboards
- Create project-specific automations

🤖 **Phase 4: Multi-Agent Systems (Month 3+)**

- Design your first specialized sub-agents
- Build agent chains for complex tasks
- Implement proactive triggering
- Create meta-agents for agent generation

Key Takeaways

1. **Start Simple**: Begin with basic hooks and gradually increase complexity
2. **Security First**: Always review hook code and use restrictive permissions
3. **Community Wisdom**: Use slash commands instead of copy-paste, prefer new sessions over clearing
4. **Combine Tools**: Use Claude Code with other AI tools for enhanced learning
5. **Think Systems**: As you scale, focus on the flow of context, model, and prompts
6. **Read the Docs**: The official Claude Code documentation at docs.anthropic.com is your best resource for detailed information and updates

The Future of Development

With hooks and sub-agents, you're not just using an AI assistant—you're building an intelligent development platform tailored to your exact needs. Every hook you write, every sub-agent you create, and every workflow you automate brings you closer to a future where repetitive tasks disappear and creativity flourishes.

Remember the Professional Gap: Claude Code's strength in rapid code generation can be a weakness for complex projects. Use the patterns in this guide to add the planning, architecture, security, and testing phases that professional development demands.

The goal isn't to replace human judgment but to augment it. Use these tools to:

- Eliminate toil and repetitive tasks
- Enforce best practices automatically
- Add professional development stages
- Create quality gates and checkpoints
- Free yourself to focus on architecture and problem-solving

Whether you're building a simple script or an enterprise application, the combination of hooks, sub-agents, and professional workflows ensures your development process scales with your ambitions.

Happy automating, and may your hooks always exit cleanly! 🚀🎣🤖, r'.go

---

5. Best Practices and Debugging {#best-practices}

🔍 Debug First

Always start by logging the JSON payload to understand the data structure:

JSON

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "jq '.' >> .claude/hooks/log.jsonl"
      }]
    }]
  }
}
```

📁 Project Organization

- Keep `.claude/settings.json` in your project repository
- Store scripts in `.claude/hooks/` directory
- Make hooks shareable and version-controlled

🛡️ Error Handling

- Hooks should fail silently unless designed to block actions
- Exit with code 0 for normal operation
- Exit with code 2 to block an action and show an error

📦 Dependency Management

For Python scripts with dependencies, use **Astral's uv**:

```shell
uv run my_script.py
```

This handles environment setup automatically.

🎯 Key Principles

1. **Start simple**: Test with basic logging before complex logic
2. **Be defensive**: Handle missing data and errors gracefully
3. **Keep it fast**: Hooks run synchronously - avoid slow operations
4. **Document well**: Comment your scripts for future reference

💡 Pro Tips from the Community

1. **Session Management**: Instead of clearing sessions, start a new one. You can resume previous sessions with `claude --resume` for better context preservation
2. **Use Slash Commands**: Rather than copying and pasting prompts (like security checks), create custom slash commands for reusable workflows
3. **Multi-Tool Learning**: Use Claude Code alongside other tools like Cursor IDE - run Claude Code for tasks while using Cursor's AI to understand the code being generated
4. **Learning Workflow**: Create a learning loop by analyzing Claude's output in other AI tools to deepen your understanding of generated code

---

6. Advanced Hook Examples {#advanced-examples}

Multi-Tool Pattern Matching

Goal: Apply different actions based on file types across multiple tools.

JSON

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|MultiEdit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "if [[ \"$CLAUDE_FILE_PATHS\" =~ \\.(py)$
]]; then black $CLAUDE_FILE_PATHS && ruff check --fix
$CLAUDE_FILE_PATHS; fi"
          },
          {
            "type": "command",
            "command": "if [[ \"$CLAUDE_FILE_PATHS\" =~
\\.(ts|tsx)$ ]]; then prettier --write $CLAUDE_FILE_PATHS && npx
tsc --noEmit $CLAUDE_FILE_PATHS; fi"
          }
        ]
      }
    ]
  }
}
```

Session Context Loading

Goal: Automatically load project context when Claude Code starts.

Tip: Instead of clearing sessions when things get messy, start a new session and use `claude --resume` to restore previous context when needed. This preserves your conversation history for reference.

Script (`.claude/hooks/session_start.py`):

Python

```
#!/usr/bin/env python3
import json
import sys
import subprocess

def load_context():
    context = []

    # Git status
    try:
        git_status = subprocess.check_output(['git', 'status',
'--short'], text=True)
        if git_status:
            context.append(f"Current git changes:\n{git_status}")
    except:
        pass

    # Recent commits
    try:
        commits = subprocess.check_output(['git', 'log',
'--oneline', '-5'], text=True)
        context.append(f"Recent commits:\n{commits}")
    except:
        pass

    # TODO items
    try:
        todos = subprocess.check_output(['grep', '-r', 'TODO:',
'.', '--include=*.py'], text=True)
        if todos:
            context.append(f"TODO items
found:\n{todos[:500]}...")
    except:
        pass
```

```
    # Output context for Claude
    if context:
        print("\n=== PROJECT CONTEXT ===")
        print("\n".join(context))
        print("========================\n")

if __name__ == "__main__":
    load_context()
```

Configuration:

JSON

```
{
  "hooks": {
    "SessionStart": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "python3 .claude/hooks/session_start.py"
      }]
    }]
  }
}
```

Advanced Command Validation with AI Feedback

Goal: Use AI to analyze potentially dangerous commands before execution.

Python

```
#!/usr/bin/env python3
import json
import sys
import re
```

```
DANGEROUS_PATTERNS = [
    (r'rm\s+-[^\s]*[rf]', "Recursive deletion detected"),
    (r'chmod\s+777', "Overly permissive file permissions"),
    (r'curl.*\|\s*sh', "Piping curl to shell - potential security
risk"),
    (r'npm\s+install.*--global', "Global npm install detected"),
    (r'docker.*--privileged', "Privileged Docker container"),
]

def analyze_command(command):
    risks = []
    for pattern, message in DANGEROUS_PATTERNS:
        if re.search(pattern, command, re.IGNORECASE):
            risks.append(message)

    if risks:
        # Exit code 2 blocks execution and feeds stderr to Claude
        print(json.dumps({
            "error": "Command blocked due to security concerns",
            "risks": risks,
            "command": command,
            "suggestion": "Please use a safer alternative or
confirm this is intentional"
        }), file=sys.stderr)
        sys.exit(2)

if __name__ == "__main__":
    data = json.load(sys.stdin)
    if data.get("tool_name") == "Bash":
        command = data.get("tool_input", {}).get("command", "")
        analyze_command(command)
```

---

7. Troubleshooting Common Issues {#troubleshooting}

Hook Not Executing

Problem: Your hooks aren't firing when expected.

Solutions:

1. **Check file permissions**:

```
Shell
chmod +x .claude/hooks/*.py
```

2. **Verify JSON syntax**:

```
Shell
jq '.' .claude/settings.json
```

3. **Use absolute paths** (recommended by documentation):

```
JSON
{
  "command":
"/absolute/path/to/your/project/.claude/hooks/script.py"
}
```

4. **Debug with logging**:

```
JSON
{
```

```
  "command": "echo 'Hook fired at $(date)' >>
/tmp/claude-hook-debug.log"
}
```

Settings Not Updating

Problem: Changes to settings.json don't take effect.

Solutions:

1. **Restart Claude Code** - Settings are loaded at startup
2. **Check for syntax errors** in your JSON
3. **Ensure you're editing the correct file**:
    - Project-specific: `.claude/settings.json`
    - User-global: `~/.claude/settings.json`

Hook Blocking Unintentionally

Problem: Hook exits with non-zero code accidentally.

Solution: Always handle errors gracefully:

```python
try:
    # Your hook logic
    pass
except Exception as e:
    # Log error but don't block Claude
    with open('/tmp/hook-errors.log', 'a') as f:
        f.write(f"Error: {e}\n")
    sys.exit(0)  # Exit successfully
```

Performance Issues

Problem: Hooks slow down Claude's operations.

Solutions:

1. **Use background execution** for non-critical hooks:

```
JSON
{
  "type": "command",
  "command": "your_command &"
}
```

2. **Set appropriate timeouts**:

```
JSON
{
  "type": "command",
  "command": "your_command",
  "timeout": 5
}
```

3. **Optimize scripts** - avoid heavy operations in synchronous hooks

---

8. Quick Reference Cheat Sheet {#cheat-sheet}

Hook Events

Event	Purpose	Can Block?	Key Data
`UserPromptSubmit`	Modify user prompts	No	`prompt`
`SessionStart`	Load context	No	`session_id`, `source`
`PreToolUse`	Validate/block tools	Yes (exit 2)	`tool_name`, `tool_input`
`PostToolUse`	Process results	No	`tool_response`
`Notification`	Custom alerts	No	`message`
`Stop`	Task completion	No	`transcript_path`
`SubagentStop`	Subtask completion	Yes (exit 2)	`stop_hook_active`
`PreCompact`	Before compaction	No	`trigger`

Exit Codes

- **0**: Success (stdout shown in transcript mode)
- **2**: Block operation (stderr sent to Claude)
- **Other**: Non-blocking error (stderr shown to user)

Environment Variables

```shell
$CLAUDE_PROJECT_DIR    # Absolute path to project root
$CLAUDE_FILE_PATHS     # Space-separated file paths
$CLAUDE_TOOL_NAME      # Name of the tool being used
$CLAUDE_NOTIFICATION   # Notification message content
```

Common Patterns

```JSON
// Match all tools
"matcher": ""

// Match specific tools
"matcher": "Bash"

// Match multiple tools
"matcher": "Edit|Write|MultiEdit"

// Match MCP tools
"matcher": "mcp__github__*"
```

Session Management

- **Start fresh**: Use new sessions instead of clearing when context gets messy
- **Resume sessions**: `claude --resume` to restore previous context
- **Headless mode**: `claude -p "prompt" --output-format stream-json` for automation
- **Non-interactive**: `claude -p "prompt" --no-interactive` for CI/CD

---

9. Integration with Development Workflows {#integrations}

Git Workflow Integration

Pre-commit Hook Integration:

```JSON
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "git add $CLAUDE_FILE_PATHS &&
./.git/hooks/pre-commit --files $CLAUDE_FILE_PATHS || true"
      }]
    }]
  }
}
```

CI/CD Integration

Headless Mode for Automation:

```Shell
# Run Claude Code in CI pipeline
claude -p "Fix all linting errors in src/" --output-format
stream-json

# Pre-push validation
claude -p "Run all tests and fix any failures" --no-interactive
```

IDE Integration

VSCode Tasks Integration:

```JSON
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit",
      "hooks": [{
        "type": "command",
        "command": "code --goto $CLAUDE_FILE_PATHS:1"
      }]
    }]
  }
}
```

Docker Development

Container Management:

```JSON
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "python3 -c \"import json,sys;
d=json.load(sys.stdin);
cmd=d.get('tool_input',{}).get('command',''); sys.exit(2 if
'docker' in cmd and '--rm' not in cmd else 0)\""
      }]
    }]
  }
}
```

---

10. Security Considerations {#security}

Key Security Principles

1. **Hooks run with your credentials** - They have full access to your system
2. **Review all hook code** before enabling it
3. **Never run untrusted hooks** from unknown sources
4. **Use restrictive permissions** on hook scripts
5. **Validate all inputs** in your hook scripts

Secure Hook Template

Python

```python
#!/usr/bin/env python3
import json
import sys
import os
import re

# Whitelist approach for allowed operations
ALLOWED_COMMANDS = [
    r'^ls\s',
    r'^cat\s',
    r'^grep\s',
    r'^find\s.*-name',
]

FORBIDDEN_PATHS = [
    '.env',
    '.ssh/',
    'credentials',
    'secrets',
    '.git/config',
]

def validate_command(command):
    # Check against whitelist
```

```
    if not any(re.match(pattern, command) for pattern in
ALLOWED_COMMANDS):
        return False, "Command not in allowlist"

    # Check for forbidden paths
    for path in FORBIDDEN_PATHS:
        if path in command:
            return False, f"Access to {path} is forbidden"

    return True, ""

def main():
    try:
        data = json.load(sys.stdin)
        if data.get("tool_name") == "Bash":
            command = data.get("tool_input", {}).get("command",
"")
            valid, reason = validate_command(command)

            if not valid:
                print(f"Security block: {reason}",
file=sys.stderr)
                sys.exit(2)
    except Exception as e:
        # Fail open - don't block on errors
        pass

if __name__ == "__main__":
    main()
```

---

11. Multi-Agent Observability {#observability}

Setting Up Observability Dashboard

Goal: Monitor multiple Claude Code agents in real-time.

Architecture:

```
None
Claude Agents → Hooks → HTTP Server → SQLite → WebSocket →
Dashboard
```

Hook Configuration:

```
JSON
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "python3 .claude/hooks/send_event.py --event
PreToolUse --agent $USER"
      }]
    }],
    "PostToolUse": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "python3 .claude/hooks/send_event.py --event
PostToolUse --agent $USER --include-output"
      }]
```

```
    }]
  }
}
```

Event Sender Script:

Python

```python
#!/usr/bin/env python3
import json
import sys
import requests
import argparse
from datetime import datetime

def send_event(event_type, agent_name, include_output=False):
    data = json.load(sys.stdin)

    event = {
        "timestamp": datetime.utcnow().isoformat(),
        "agent": agent_name,
        "event_type": event_type,
        "tool_name": data.get("tool_name"),
        "tool_input": data.get("tool_input"),
    }

    if include_output:
        event["tool_output"] = data.get("tool_response",
{}).get("output", "")[:500]

    try:
        requests.post("http://localhost:8080/events", json=event,
timeout=1)
    except:
        pass  # Don't block Claude if observability is down
```

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--event", required=True)
    parser.add_argument("--agent", required=True)
    parser.add_argument("--include-output", action="store_true")
    args = parser.parse_args()

    send_event(args.event, args.agent, args.include_output)
```

---

12. Custom Slash Commands {#slash-commands}

Creating Custom Commands

Custom slash commands extend Claude Code's functionality through natural language templates. **Pro tip**: Instead of repeatedly copying and pasting prompts (like security checks or review templates), convert them into slash commands for instant reuse.

Location: `.claude/commands/` (project) or `~/.claude/commands/` (global)

Example: `.claude/commands/review-pr.md`

None
```
Please review the changes in the current git branch:

1. First, check what branch we're on with `git branch
--show-current`
2. Show the diff with `git diff main...HEAD`
3. Analyze the changes for:
   - Potential bugs or logic errors
   - Security vulnerabilities
   - Performance issues
   - Missing error handling
4. Suggest improvements with specific code examples
5. Check if tests are needed for the changes
```

```
Focus on actual issues, not style preferences.
```

Example: Convert Frequent Prompts to Commands:
`.claude/commands/security-check.md`

```
None
Perform a comprehensive security audit on the codebase:

1. Check for hardcoded credentials or API keys
2. Review authentication and authorization logic
3. Look for SQL injection vulnerabilities
4. Check for XSS vulnerabilities in frontend code
5. Review file upload handling for security issues
6. Check dependency versions for known vulnerabilities
7. Review error handling to ensure no sensitive data leaks

Provide specific recommendations for each issue found.
```

Tip: If you find yourself copying and pasting the same prompt repeatedly, it's time to make it a slash command!

Example with Arguments: `.claude/commands/fix-issue.md`

```
None
Please fix GitHub issue: $ARGUMENTS

Steps:
1. Use `gh issue view $ARGUMENTS` to read the issue
2. Identify the root cause
3. Find relevant files with grep/find
4. Implement the fix
5. Write tests for the fix
6. Verify all tests pass
7. Create a commit with message "Fix #$ARGUMENTS: [description]"
```

Usage:

```
Shell
/project:review-pr
/project:fix-issue 123
```

---

13. Working with MCP Tools {#mcp-tools}

Hook Integration with MCP

Model Context Protocol (MCP) tools appear with special naming patterns in hooks and provide specialized capabilities.

Essential MCP Servers

Core MCP Tools:

1. **Sequential**: Enables complex multi-step thinking and problem-solving
2. **Context7**: Grabs official libraries and documentation from online sources
3. **Magic**: Generates modern UI components with best practices
4. **Playwright**: Browser automation and testing capabilities
5. **Memory**: Persistent context storage across sessions
6. **Filesystem**: Enhanced file operations
7. **GitHub**: Direct GitHub API integration

Installing MCP Servers

```
Shell
# Add MCP servers to Claude Code
claude mcp add sequential
claude mcp add context7
claude mcp add magic
claude mcp add playwright

# Check connected MCP servers
claude
# Then use: /mcp
```

MCP Tool Naming Patterns

MCP tools follow the pattern `mcp__<server>__<tool>`:

- `mcp__memory__create_entities`
- `mcp__filesystem__read_file`
- `mcp__github__search_repositories`
- `mcp__sequential__think_step_by_step`
- `mcp__context7__fetch_documentation`
- `mcp__magic__generate_component`

Example Hooks for MCP Tools

JSON

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "mcp__github__*",
      "hooks": [{
        "type": "command",
        "command": "echo '[$(date)] GitHub MCP action:
$CLAUDE_TOOL_NAME' >> ~/.claude/mcp-audit.log"
      }]
    }],
    "PostToolUse": [{
      "matcher": "mcp__filesystem__write_file",
      "hooks": [{
        "type": "command",
        "command": "git add $CLAUDE_FILE_PATHS && echo
'Auto-staged changes from MCP filesystem write'"
      }]
    }]
  }
}
```

---

14. Learning and Development Workflows {#learning-workflows}

Using Claude Code with Other AI Tools

Multi-AI Workflow: Combine Claude Code with other AI assistants for enhanced learning and development.

Example: Claude Code + Cursor IDE

1. **Run Claude Code** for implementing features or fixes
2. **Use Cursor's AI** to ask questions about the generated code:
    - "Explain what this function does"
    - "List everything in the .claude directory and its purpose"
    - "What design patterns are being used here?"
3. **Deep dive with Claude Desktop** for complex concepts you don't understand

Creating a Personal Learning Assistant

Build a custom GPT or AI assistant trained on:

- Your preferred learning style
- Your tech stack and domain knowledge
- Your coding conventions

Then feed it Claude Code's output for personalized explanations and learning paths.

Hook for Learning Capture

JSON

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "echo '## Code Change\nTool:
$CLAUDE_TOOL_NAME\nFiles: $CLAUDE_FILE_PATHS\n' >>
.claude/learning-log.md"
      }]
    }]
  }
```

```
}
```

This creates a learning log you can review to understand what Claude did and why.

---

15. Sub-Agents and Multi-Agent Systems {#sub-agents}

Introduction to Sub-Agents

Sub-agents in Claude Code are specialized AI assistants that can be called by your primary Claude agent to handle specific tasks. They enable modular, scalable workflows by dividing complex problems into specialized domains.

Creating Sub-Agents

Getting Started:

1. Open the sub-agents interface with `/agents`
2. Select "Create New Agent"
3. Choose scope:
    - **Project-level**: Available only in current project
    - **User-level**: Available across all your projects
4. Edit the system prompt (press Enter to use your editor)
5. Save and use

Key Features

🎯 Context Preservation

Each sub-agent operates in its own context, preventing pollution of the main conversation and keeping focus on specific objectives.

🔧 Specialized Expertise

Sub-agents can be fine-tuned with detailed instructions for specific domains, leading to higher success rates on designated tasks.

♻️ Reusability

Once created, sub-agents can be used across different projects and shared with your team for consistent workflows.

🔒 **Flexible Permissions**

Each sub-agent can have different tool access levels, allowing you to limit powerful tools to specific agent types.

Sub-Agent Best Practices

1. System Prompt is Key

With sub-agents, you write the system prompt, not the user prompt. This gives you more control over the agent's behavior.

```
None
# Example Sub-Agent System Prompt for Code Review
You are a specialized code review agent. Your primary agent will
send you code changes to review.

Your responsibilities:
1. Check for bugs and logic errors
2. Identify security vulnerabilities
3. Suggest performance improvements
4. Ensure code follows project conventions

Always respond with structured feedback that the primary agent
can act upon.
```

2. Sub-Agents Respond to Your Primary Agent

Remember: sub-agents communicate with your primary agent, not directly with you. Write prompts accordingly.

3. Be Explicit in Your Instructions

The more specific your instructions, the better your agents perform. Avoid ambiguity:

❌ **Vague**: "Review this code" ✅ **Explicit**: "Review this Python code for SQL injection vulnerabilities, checking all database queries for parameterization"

4. Use the Meta-Agent

The meta-agent is a powerful tool for creating new agents from scratch. Use it to bootstrap your multi-agent system.

5. Chain Sub-Agents for Complex Workflows

```
None
Primary Agent → Code Generator Sub-Agent → Code Review Sub-Agent
→ Test Writer Sub-Agent
```

The "Big 3" of Agentic Coding

As you scale multi-agent systems, three elements become critical:

1. **Context**: What information each agent has access to
2. **Model**: Which AI model powers each agent
3. **Prompt**: How each agent is instructed

The flow and management of these three elements determines the success of your multi-agent system.

Hooks + Sub-Agents: Advanced Integration

Auto-Generate Sub-Agents with Hooks

Use hooks to automatically create specialized sub-agents based on project needs:

```python
#!/usr/bin/env python3
# .claude/hooks/auto_agent_creator.py
import json
import sys
import os

def should_create_agent(tool_input):
    # Check if working with a new framework/library
    file_path = tool_input.get("file_path", "")

    # Detect new patterns that might need specialized agents
    patterns = {
        "react": ["useState", "useEffect", "jsx"],
        "django": ["models.Model", "views", "urls.py"],
        "fastapi": ["FastAPI", "@app.", "BaseModel"],
```

```
    }

    for framework, keywords in patterns.items():
        agent_path = f".claude/agents/{framework}-specialist.md"
        if not os.path.exists(agent_path) and any(kw in
str(tool_input) for kw in keywords):
            create_specialist_agent(framework, agent_path)
            print(f"🤖 Created {framework} specialist agent!")

def create_specialist_agent(framework, path):
    template = f"""---
name: {framework}-specialist
description: Expert in {framework} development patterns and best
practices
---

You are a {framework} specialist. Help the primary agent with:
- {framework}-specific patterns and idioms
- Performance optimizations
- Security best practices
- Testing strategies

Always provide {framework}-idiomatic solutions.
"""
    os.makedirs(os.path.dirname(path), exist_ok=True)
    with open(path, 'w') as f:
        f.write(template)

if __name__ == "__main__":
    data = json.load(sys.stdin)
    if data.get("tool_name") in ["Edit", "Write"]:
        should_create_agent(data.get("tool_input", {}))
```

Advanced Sub-Agent Patterns

Proactive Triggering

Configure agents to automatically perform tasks when conditions are met:

```
None
# Auto-Summary Sub-Agent
When called, check if:
- More than 10 files have been modified
- More than 500 lines of code changed
- A major refactoring occurred
- Session duration exceeds 30 minutes

If any condition is true, automatically generate:
1. Executive summary of changes
2. Technical details of modifications
3. Next steps and recommendations
4. Any identified technical debt

Format the summary for easy sharing with team members.
```

Hook Integration for Auto-Summary:

```
JSON
{
  "hooks": {
    "Stop": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "if [ $(git diff --stat | tail -1 | awk
'{print $4}') -gt 500 ]; then echo 'Major changes detected.
Consider running /agent:summarizer'; fi"
      }]
    }]
  }
```

```
}
```

Information-Dense Keywords

Claude Code provides special keywords and variables to give agents more context:

Environment Variables:

- `$CLAUDE_PROJECT_DIR` - Project root directory
- `$CLAUDE_FILE_PATHS` - Currently relevant files
- `$CLAUDE_TOOL_NAME` - Active tool being used
- `$CLAUDE_SESSION_ID` - Current session identifier

In Sub-Agent Prompts:

- Reference current state with `{{current_files}}`
- Access recent changes with `{{recent_changes}}`
- Include project context with `{{project_context}}`

Example Usage:

```
None
Review the changes in {{recent_changes}} and ensure they follow
the patterns established in {{project_context}}. Pay special
attention to files matching {{current_files}} pattern.
```

Sub-Agent Configuration with Hooks

Combine sub-agents with hooks for powerful automation:

```
JSON
{
  "hooks": {
    "SubagentStop": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
```

```
        "command": "echo 'Sub-agent completed: Review the output
for quality' | tee -a .claude/subagent-log.txt"
      }]
    }]
  }
}
```

Monitoring Sub-Agent Performance

Create a hook to track sub-agent execution times and success rates:

Python

```python
#!/usr/bin/env python3
import json
import sys
import time
from datetime import datetime

def log_subagent_performance():
    data = json.load(sys.stdin)

    # Extract sub-agent info from context
    timestamp = datetime.now().isoformat()
    session_id = data.get("session_id", "unknown")

    # Log performance metrics
    log_entry = {
        "timestamp": timestamp,
        "session_id": session_id,
        "event": "subagent_complete",
        "duration": time.time() - float(data.get("start_time",
time.time()))
    }

    with open(".claude/subagent-metrics.jsonl", "a") as f:
        f.write(json.dumps(log_entry) + "\n")
```

```
    # Alert if sub-agent took too long
    if log_entry["duration"] > 300:  # 5 minutes
        print("⚠️ Sub-agent took longer than expected!")

if __name__ == "__main__":
    log_subagent_performance()
```

Important Warnings

⚠️ YOLO Mode Caution

YOLO mode gives agents free rein to execute without confirmation. Use with extreme caution and only in safe environments.

🔐 Keep Agents Separate

As you build more agents, maintain clear separation to avoid confusion and keep your codebase clean.

🏝️ Use Isolated Workflows

Create isolated environments for different agent workflows to prevent interference:

- Separate directories for agent-specific files
- Dedicated configuration files
- Independent logging streams
- Isolated test environments

🔄 Don't Overload Sub-Agent Chains

Long chains of sub-agents can become difficult to manage and debug. Keep chains reasonable (3-5 agents max).

🧠 Don't Offload All Cognitive Work

While agents are powerful assistants, maintain your understanding of what they're doing. You need to:

- Understand the code being generated
- Make architectural decisions
- Review and validate agent outputs

- Maintain overall system vision

Agents augment your capabilities—they don't replace your expertise.

Sub-Agent Frontmatter

The description variable is crucial for guiding your primary agent. It's the most important variable after the name:

```
None
---
name: security-auditor
description: |
  Performs comprehensive security audits on code.
  Call this agent when:
  - New authentication code is added
  - File upload functionality is implemented
  - Database queries are modified
  - External API integrations are created

  This agent will return a structured security report with:
  - Identified vulnerabilities (scored by severity)
  - Specific remediation steps
  - Security best practices relevant to the code
tools:
  - read_file
  - search_code
  - analyze_dependencies
---
```

Description Best Practices:

- Be explicit about when to call the agent
- Specify what the agent returns
- List specific triggers or conditions
- Include expected output format

Example: Multi-Agent Development Workflow

```
None
# Primary Agent Instructions
You coordinate a team of specialized sub-agents:

1. **Architect Agent**: Design system architecture
2. **Implementation Agent**: Write code following the
architecture
3. **Test Agent**: Create comprehensive test suites
4. **Documentation Agent**: Generate docs and comments
5. **Review Agent**: Final quality check

For each feature request:
1. Have Architect design the approach
2. Implementation writes the code
3. Test creates tests
4. Documentation updates docs
5. Review ensures quality

Only proceed to the next step after the previous agent confirms
completion.
```

Hooks vs Sub-Agents: When to Use Each

Use Hooks When:

- You need deterministic, automated actions
- The task is simple and well-defined
- You want to enforce policies or safety checks
- Speed is critical (hooks are faster)
- You need to integrate with external tools

Use Sub-Agents When:

- The task requires AI reasoning or creativity
- You need specialized domain expertise
- The problem is complex and requires context
- You want reusable, shareable components
- You need natural language processing

Combining Both:

```JSON
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "python3
.claude/hooks/check_if_needs_security_agent.py"
      }]
    }]
  }
}
```

This hook could analyze commands and automatically suggest calling the security sub-agent when needed.

Practical Integration Example:

```Python
#!/usr/bin/env python3
# Hook that suggests relevant sub-agents based on activity
import json
import sys

AGENT_TRIGGERS = {
    "security-auditor": ["password", "auth", "token", "api_key"],
    "performance-optimizer": ["slow", "optimize", "benchmark"],
    "test-writer": ["test", "spec", "coverage"],
}

def suggest_agents(tool_input):
    suggestions = []
    content = str(tool_input).lower()

    for agent, triggers in AGENT_TRIGGERS.items():
```

```
        if any(trigger in content for trigger in triggers):
            suggestions.append(agent)

    if suggestions:
        print(f"💡 Consider using these sub-agents: {',
'.join(suggestions)}")

if __name__ == "__main__":
    data = json.load(sys.stdin)
    if data.get("tool_name") in ["Edit", "Write"]:
        suggest_agents(data.get("tool_input", {}))
```

---

16. Professional Development Patterns {#professional-patterns}

The Claude Code Gap: From Scripts to Systems

While Claude Code excels at generating code quickly, it often skips critical professional development steps. This works fine for small scripts but falls apart for enterprise-level applications. Professional developers don't just write code—they plan, architect, design, test, and secure their systems.

Professional Workflow Stages

Here's how to structure Claude Code for serious development:

1. Project Planning & Architecture

```
None
# Command: /project:architect-plan "Design a scalable e-commerce
platform"

Perform comprehensive project planning:
1. Define system requirements and constraints
2. Create high-level architecture diagrams
3. Design database schema
```

```
4. Plan API structure
5. Define security protocols
6. Create testing strategy
7. Estimate scalability needs (users, data, traffic)

Output a structured architecture document with:
- System overview
- Component breakdown
- Data flow diagrams
- Security considerations
- Scalability assessment
```

2. Frontend Development with Standards

```
None
# Command: /project:frontend-tdd "Implement user dashboard"

Develop frontend with professional standards:
1. Review UI/UX designs or create wireframes
2. Set up component architecture
3. Write tests FIRST (TDD approach)
4. Implement components
5. Ensure accessibility (WCAG compliance)
6. Optimize performance
7. Add proper error handling

Use modern patterns:
- React Query for server state
- Proper state management
- Component composition
- Performance optimization
```

3. Backend Development with TDD

```
None
# Command: /project:backend-secure "Build authentication system"

Professional backend development:
1. Design API contracts first
2. Write integration tests
3. Implement with security in mind
4. Add comprehensive error handling
5. Include logging and monitoring
6. Document all endpoints
7. Performance profiling

Follow patterns:
- Repository pattern for data access
- Service layer for business logic
- Proper validation and sanitization
- Rate limiting and security headers
```

Essential Personas for Professional Development

Create specialized sub-agents for each role:

System Architect

```
None
---
name: system-architect
description: |
  Enterprise architecture specialist focusing on:
  - Scalable system design
  - Technology selection
  - Integration patterns
  - Performance architecture
  - Security architecture

  Call when planning new systems or major refactors
```

```
---
```

Security Engineer

```
None
---
name: security-engineer
description: |
  Security specialist for:
  - Vulnerability assessment
  - Security architecture review
  - Penetration testing strategies
  - Compliance requirements
  - Security best practices

  Call for any security-sensitive features
---
```

Performance Engineer

```
None
---
name: performance-engineer
description: |
  Performance optimization expert for:
  - Bottleneck identification
  - Query optimization
  - Caching strategies
  - Load testing
  - Scalability planning
---
```

MCP Tools for Professional Development

Enhance Claude Code with specialized MCP servers:

1. **Sequential Thinking**: Complex multi-step problem solving
2. **Context7**: Access official documentation and libraries
3. **Magic**: Modern UI component generation
4. **Playwright**: Automated testing and browser automation

Professional Troubleshooting Workflow

Step 1: Problem Identification

```
None
# Command: /project:investigate "Production errors increasing"

Investigate the issue:
1. Analyze error logs
2. Check monitoring dashboards
3. Review recent deployments
4. Identify patterns
5. Document symptoms
```

Step 2: Root Cause Analysis

```
None
# Command: /project:five-whys "API timeout errors"

Apply Five Whys methodology:
1. Why is the API timing out?
2. Why is the database query slow?
3. Why is the index not being used?
4. Why was the index dropped?
5. Why wasn't this caught in testing?

Output: Root cause and remediation plan
```

Step 3: Performance Analysis

```
None
# Command: /project:profile "Analyze slow endpoints"

Performance profiling:
1. Run performance profiler
2. Identify bottlenecks
3. Analyze database queries
4. Check memory usage
5. Review caching effectiveness
6. Suggest optimizations
```

Architecture Analysis Hook

Create a hook that triggers architecture analysis for significant changes:

```
JSON
{
  "hooks": {
    "Stop": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "python3 .claude/hooks/architecture_check.py"
      }]
    }]
  }
}
```

Architecture Check Script:

```
Python
#!/usr/bin/env python3
import subprocess
import json
```

```
def check_architecture_impact():
    # Check changed files
    changed = subprocess.check_output(['git', 'diff',
'--name-only'], text=True)

    critical_patterns = [
        'schema', 'migration', 'auth', 'security',
        'api/', 'architecture', 'config'
    ]

    if any(pattern in changed.lower() for pattern in
critical_patterns):
        print("\n🏗️ ARCHITECTURE IMPACT DETECTED")
        print("Consider running: /project:architect-review")
        print("Critical files changed that may affect system
architecture")

        # Auto-generate mini architecture report
        with open('.claude/architecture-changes.md', 'w') as f:
            f.write(f"# Architecture Impact Analysis\n\n")
            f.write(f"## Changed Files\n{changed}\n")
            f.write(f"## Recommended Actions\n")
            f.write(f"- Review system architecture\n")
            f.write(f"- Update documentation\n")
            f.write(f"- Check security implications\n")
            f.write(f"- Verify scalability impact\n")

if __name__ == "__main__":
    check_architecture_impact()
```

Quality Gates with Hooks

Implement quality gates that enforce professional standards:

```json
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": ".claude/hooks/quality_gate.sh"
      }]
    }]
  }
}
```

Quality Gate Script:

```shell
#!/bin/bash
# .claude/hooks/quality_gate.sh

# Check if tests exist for new code
if [[ "$CLAUDE_FILE_PATHS" =~ \.(ts|js|py)$ ]]; then
    base_name=$(basename "$CLAUDE_FILE_PATHS" | sed
's/\.[^.]*$//')
    test_file=$(find . -name "*${base_name}*test*" -o -name
"*${base_name}*spec*" | head -1)

    if [ -z "$test_file" ]; then
        echo "⚠️ WARNING: No tests found for $CLAUDE_FILE_PATHS"
>&2
        echo "Consider TDD: Write tests before implementation"
>&2
        # Don't block, just warn
        exit 0
```

```
    fi
fi
```

Scalability Assessment Patterns

Create commands that analyze scalability at different stages:

```
None
# Command: /project:scale-assessment "Current system analysis"

Perform scalability assessment:
1. Current capacity (concurrent users, data volume)
2. Identify bottlenecks:
   - Database connections
   - API rate limits
   - Memory usage
   - Third-party service limits
3. Growth projections
4. Scaling recommendations:
   - Horizontal vs vertical scaling
   - Caching strategies
   - Database optimization
   - Service decomposition
5. Cost analysis

Output scaling roadmap with priorities
```

Professional Development Checklist

Use this checklist for every serious project:

- [ ] Architecture documented before coding
- [ ] Security review completed
- [ ] API contracts defined
- [ ] Test strategy in place
- [ ] Performance benchmarks set
- [ ] Monitoring configured

- [ ] Documentation updated
- [ ] Code review process defined
- [ ] Deployment strategy planned
- [ ] Rollback procedures ready

Integration Example: Full Professional Workflow

```shell
# 1. Start with architecture
/project:architect-plan "Build a real-time collaboration tool"

# 2. Security review
/project:security-review "Review authentication approach"

# 3. Frontend with TDD
/project:frontend-tdd "Implement collaborative editor"

# 4. Backend with security
/project:backend-secure "Build WebSocket server"

# 5. Performance testing
/project:performance-test "Load test with 1000 concurrent users"

# 6. Deploy with confidence
/project:deploy-checklist "Production deployment readiness"
```

This professional approach transforms Claude Code from a code generator into a comprehensive development platform that follows enterprise best practices.

Conclusion

Claude Code hooks and sub-agents represent a paradigm shift in AI-assisted development. By combining deterministic automation (hooks) with intelligent reasoning (sub-agents), you create a development environment that's both powerful and predictable.

Your Journey: From Hooks to Multi-Agent Systems

🚀 Phase 1: Basic Automation (Week 1)

- Start with simple notification hooks
- Add basic logging and debugging hooks
- Create your first safety checks
- Convert repetitive prompts to slash commands

🛡️ **Phase 2: Safety and Quality (Week 2-3)**

- Implement comprehensive command validation
- Set up automatic formatting and linting
- Add pre-commit style checks
- Create file protection rules

🔧 **Phase 3: Advanced Workflows (Month 2)**

- Build multi-tool pattern matching
- Integrate with your CI/CD pipeline
- Set up observability dashboards
- Create project-specific automations

🤖 **Phase 4: Multi-Agent Systems (Month 3+)**

- Design your first specialized sub-agents
- Build agent chains for complex tasks
- Implement proactive triggering
- Create meta-agents for agent generation

Key Takeaways

1. **Start Simple**: Begin with basic hooks and gradually increase complexity
2. **Security First**: Always review hook code and use restrictive permissions
3. **Community Wisdom**: Use slash commands instead of copy-paste, prefer new sessions over clearing
4. **Combine Tools**: Use Claude Code with other AI tools for enhanced learning
5. **Think Systems**: As you scale, focus on the flow of context, model, and prompts
6. **Read the Docs**: The official Claude Code documentation at docs.anthropic.com is your best resource for detailed information and updates

The Future of Development

With hooks and sub-agents, you're not just using an AI assistant—you're building an intelligent development platform tailored to your exact needs. Every hook you write, every sub-agent you create, and every workflow you automate brings you closer to a future where repetitive tasks disappear and creativity flourishes.

Remember the Professional Gap: Claude Code's strength in rapid code generation can be a weakness for complex projects. Use the patterns in this guide to add the planning, architecture, security, and testing phases that professional development demands.

The goal isn't to replace human judgment but to augment it. Use these tools to:

- Eliminate toil and repetitive tasks
- Enforce best practices automatically
- Add professional development stages
- Create quality gates and checkpoints
- Free yourself to focus on architecture and problem-solving

Whether you're building a simple script or an enterprise application, the combination of hooks, sub-agents, and professional workflows ensures your development process scales with your ambitions.

Happy automating, and may your hooks always exit cleanly! 🚀🎣🤖] return any(re.search(pattern, file_path) for pattern in patterns)

def find_test_file(file_path): # Look for corresponding test file base = os.path.splitext(os.path.basename(file_path))[0] test_patterns = [f"*{base}test", f"*{base}spec", f"test_{base}", f"*{base}_test"]

```
None
import glob
for pattern in test_patterns:
    matches = glob.glob(f"**/{pattern}", recursive=True)
    if matches:
        return matches[0]
return None
```

def main(): data = json.load(sys.stdin) tool_name = data.get("tool_name")

```
None
if tool_name in ["Write", "Edit"]:
    file_path = data.get("tool_input", {}).get("file_path", "")

    if should_have_test(file_path) and not
find_test_file(file_path):
```

```
        print("🧪 TDD Reminder: No test file found!",
file=sys.stderr)
        print(f"Consider writing tests first for {file_path}",
file=sys.stderr)
        print("Following TDD? Create the test file before
implementing.", file=sys.stderr)
        # Don't block, just remind
        sys.exit(0)
```

if **name** == "**main**": main()

None

```
**Step 2**: Configure the hook:
```json
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command": "python3 .claude/hooks/tdd_check.py"
          }
        ]
      }
    ]
  }
}
```

```

5. Best Practices and Debugging {#best-practices}

🔍 Debug First

Always start by logging the JSON payload to understand the data structure:

```json
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "jq '.' >> .claude/hooks/log.jsonl"
      }]
    }]
  }
}
```

📁 Project Organization

- Keep `.claude/settings.json` in your project repository
- Store scripts in `.claude/hooks/` directory
- Make hooks shareable and version-controlled

🛡️ Error Handling

- Hooks should fail silently unless designed to block actions
- Exit with code 0 for normal operation
- Exit with code 2 to block an action and show an error

📦 Dependency Management

For Python scripts with dependencies, use **Astral's uv**:

```shell
uv run my_script.py
```

This handles environment setup automatically.

🎯 Key Principles

1. **Start simple**: Test with basic logging before complex logic
2. **Be defensive**: Handle missing data and errors gracefully
3. **Keep it fast**: Hooks run synchronously - avoid slow operations
4. **Document well**: Comment your scripts for future reference

💡 Pro Tips from the Community

1. **Session Management**: Instead of clearing sessions, start a new one. You can resume previous sessions with `claude --resume` for better context preservation
2. **Use Slash Commands**: Rather than copying and pasting prompts (like security checks), create custom slash commands for reusable workflows
3. **Multi-Tool Learning**: Use Claude Code alongside other tools like Cursor IDE - run Claude Code for tasks while using Cursor's AI to understand the code being generated
4. **Learning Workflow**: Create a learning loop by analyzing Claude's output in other AI tools to deepen your understanding of generated code

---

6. Advanced Hook Examples {#advanced-examples}

Multi-Tool Pattern Matching

Goal: Apply different actions based on file types across multiple tools.

JSON

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|MultiEdit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "if [[ \"$CLAUDE_FILE_PATHS\" =~ \\.(py)$
]]; then black $CLAUDE_FILE_PATHS && ruff check --fix
$CLAUDE_FILE_PATHS; fi"
          },
          {
            "type": "command",
            "command": "if [[ \"$CLAUDE_FILE_PATHS\" =~
\\.(ts|tsx)$ ]]; then prettier --write $CLAUDE_FILE_PATHS && npx
tsc --noEmit $CLAUDE_FILE_PATHS; fi"
          }
        ]
      }
    ]
  }
}
```

Session Context Loading

Goal: Automatically load project context when Claude Code starts.

Tip: Instead of clearing sessions when things get messy, start a new session and use `claude --resume` to restore previous context when needed. This preserves your conversation history for reference.

Script (`.claude/hooks/session_start.py`):

Python

```
#!/usr/bin/env python3
import json
import sys
import subprocess

def load_context():
    context = []

    # Git status
    try:
        git_status = subprocess.check_output(['git', 'status',
'--short'], text=True)
        if git_status:
            context.append(f"Current git changes:\n{git_status}")
    except:
        pass

    # Recent commits
    try:
        commits = subprocess.check_output(['git', 'log',
'--oneline', '-5'], text=True)
        context.append(f"Recent commits:\n{commits}")
    except:
        pass

    # TODO items
    try:
        todos = subprocess.check_output(['grep', '-r', 'TODO:',
'.', '--include=*.py'], text=True)
        if todos:
            context.append(f"TODO items
found:\n{todos[:500]}...")
    except:
        pass
```

```
    # Output context for Claude
    if context:
        print("\n=== PROJECT CONTEXT ===")
        print("\n".join(context))
        print("========================\n")

if __name__ == "__main__":
    load_context()
```

Configuration:

```JSON
{
  "hooks": {
    "SessionStart": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "python3 .claude/hooks/session_start.py"
      }]
    }]
  }
}
```

Advanced Command Validation with AI Feedback

Goal: Use AI to analyze potentially dangerous commands before execution.

```Python
#!/usr/bin/env python3
import json
import sys
import re
```

```
DANGEROUS_PATTERNS = [
    (r'rm\s+-[^\s]*[rf]', "Recursive deletion detected"),
    (r'chmod\s+777', "Overly permissive file permissions"),
    (r'curl.*\|\s*sh', "Piping curl to shell - potential security
risk"),
    (r'npm\s+install.*--global', "Global npm install detected"),
    (r'docker.*--privileged', "Privileged Docker container"),
]

def analyze_command(command):
    risks = []
    for pattern, message in DANGEROUS_PATTERNS:
        if re.search(pattern, command, re.IGNORECASE):
            risks.append(message)

    if risks:
        # Exit code 2 blocks execution and feeds stderr to Claude
        print(json.dumps({
            "error": "Command blocked due to security concerns",
            "risks": risks,
            "command": command,
            "suggestion": "Please use a safer alternative or
confirm this is intentional"
        }), file=sys.stderr)
        sys.exit(2)

if __name__ == "__main__":
    data = json.load(sys.stdin)
    if data.get("tool_name") == "Bash":
        command = data.get("tool_input", {}).get("command", "")
        analyze_command(command)
```

---

7. Troubleshooting Common Issues {#troubleshooting}

Hook Not Executing

Problem: Your hooks aren't firing when expected.

Solutions:

1. **Check file permissions**:

```
Shell
chmod +x .claude/hooks/*.py
```

2. **Verify JSON syntax**:

```
Shell
jq '.' .claude/settings.json
```

3. **Use absolute paths** (recommended by documentation):

```
JSON
{
  "command":
"/absolute/path/to/your/project/.claude/hooks/script.py"
}
```

4. **Debug with logging**:

```
JSON
{
```

```
  "command": "echo 'Hook fired at $(date)' >>
/tmp/claude-hook-debug.log"
}
```

Settings Not Updating

Problem: Changes to settings.json don't take effect.

Solutions:

1. **Restart Claude Code** - Settings are loaded at startup
2. **Check for syntax errors** in your JSON
3. **Ensure you're editing the correct file**:
    - Project-specific: `.claude/settings.json`
    - User-global: `~/.claude/settings.json`

Hook Blocking Unintentionally

Problem: Hook exits with non-zero code accidentally.

Solution: Always handle errors gracefully:

```python
try:
    # Your hook logic
    pass
except Exception as e:
    # Log error but don't block Claude
    with open('/tmp/hook-errors.log', 'a') as f:
        f.write(f"Error: {e}\n")
    sys.exit(0)  # Exit successfully
```

Performance Issues

Problem: Hooks slow down Claude's operations.

Solutions:

1. **Use background execution** for non-critical hooks:

```
JSON
{
  "type": "command",
  "command": "your_command &"
}
```

2. **Set appropriate timeouts**:

```
JSON
{
  "type": "command",
  "command": "your_command",
  "timeout": 5
}
```

3. **Optimize scripts** - avoid heavy operations in synchronous hooks

---

8. Quick Reference Cheat Sheet {#cheat-sheet}

Hook Events

Event	Purpose	Can Block?	Key Data
`UserPromptSubmit`	Modify user prompts	No	`prompt`
`SessionStart`	Load context	No	`session_id`, `source`
`PreToolUse`	Validate/block tools	Yes (exit 2)	`tool_name`, `tool_input`
`PostToolUse`	Process results	No	`tool_response`
`Notification`	Custom alerts	No	`message`
`Stop`	Task completion	No	`transcript_path`
`SubagentStop`	Subtask completion	Yes (exit 2)	`stop_hook_active`
`PreCompact`	Before compaction	No	`trigger`

Exit Codes

- **0**: Success (stdout shown in transcript mode)
- **2**: Block operation (stderr sent to Claude)
- **Other**: Non-blocking error (stderr shown to user)

Environment Variables

```shell
$CLAUDE_PROJECT_DIR    # Absolute path to project root
$CLAUDE_FILE_PATHS     # Space-separated file paths
$CLAUDE_TOOL_NAME      # Name of the tool being used
$CLAUDE_NOTIFICATION   # Notification message content
```

Common Patterns

```JSON
// Match all tools
"matcher": ""

// Match specific tools
"matcher": "Bash"

// Match multiple tools
"matcher": "Edit|Write|MultiEdit"

// Match MCP tools
"matcher": "mcp__github__*"
```

Session Management

- **Start fresh**: Use new sessions instead of clearing when context gets messy
- **Resume sessions**: `claude --resume` to restore previous context
- **Headless mode**: `claude -p "prompt" --output-format stream-json` for automation
- **Non-interactive**: `claude -p "prompt" --no-interactive` for CI/CD

---

9. Integration with Development Workflows {#integrations}

Git Workflow Integration

Pre-commit Hook Integration:

```json
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "git add $CLAUDE_FILE_PATHS &&
./.git/hooks/pre-commit --files $CLAUDE_FILE_PATHS || true"
      }]
    }]
  }
}
```

CI/CD Integration

Headless Mode for Automation:

```shell
# Run Claude Code in CI pipeline
claude -p "Fix all linting errors in src/" --output-format
stream-json

# Pre-push validation
claude -p "Run all tests and fix any failures" --no-interactive
```

IDE Integration

VSCode Tasks Integration:

```json
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit",
      "hooks": [{
        "type": "command",
        "command": "code --goto $CLAUDE_FILE_PATHS:1"
      }]
    }]
  }
}
```

Docker Development

Container Management:

```json
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "python3 -c \"import json,sys;
d=json.load(sys.stdin);
cmd=d.get('tool_input',{}).get('command',''); sys.exit(2 if
'docker' in cmd and '--rm' not in cmd else 0)\""
      }]
    }]
  }
}
```

---

10. Security Considerations {#security}

Key Security Principles

1. **Hooks run with your credentials** - They have full access to your system
2. **Review all hook code** before enabling it
3. **Never run untrusted hooks** from unknown sources
4. **Use restrictive permissions** on hook scripts
5. **Validate all inputs** in your hook scripts

Secure Hook Template

Python

```python
#!/usr/bin/env python3
import json
import sys
import os
import re

# Whitelist approach for allowed operations
ALLOWED_COMMANDS = [
    r'^ls\s',
    r'^cat\s',
    r'^grep\s',
    r'^find\s.*-name',
]

FORBIDDEN_PATHS = [
    '.env',
    '.ssh/',
    'credentials',
    'secrets',
    '.git/config',
]

def validate_command(command):
    # Check against whitelist
    if not any(re.match(pattern, command) for pattern in
ALLOWED_COMMANDS):
```

```
        return False, "Command not in allowlist"

    # Check for forbidden paths
    for path in FORBIDDEN_PATHS:
        if path in command:
            return False, f"Access to {path} is forbidden"

    return True, ""

def main():
    try:
        data = json.load(sys.stdin)
        if data.get("tool_name") == "Bash":
            command = data.get("tool_input", {}).get("command",
"")
            valid, reason = validate_command(command)

            if not valid:
                print(f"Security block: {reason}",
file=sys.stderr)
                sys.exit(2)
    except Exception as e:
        # Fail open - don't block on errors
        pass

if __name__ == "__main__":
    main()
```

---

11. Multi-Agent Observability {#observability}

Setting Up Observability Dashboard

Goal: Monitor multiple Claude Code agents in real-time.

Architecture:

```
Claude Agents → Hooks → HTTP Server → SQLite → WebSocket → Dashboard
```

Hook Configuration:

```json
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "python3 .claude/hooks/send_event.py --event PreToolUse --agent $USER"
      }]
    }],
    "PostToolUse": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "python3 .claude/hooks/send_event.py --event PostToolUse --agent $USER --include-output"
      }]
    }]
  }
}
```

Event Sender Script:

Python

```
#!/usr/bin/env python3
import json
import sys
import requests
import argparse
from datetime import datetime

def send_event(event_type, agent_name, include_output=False):
    data = json.load(sys.stdin)

    event = {
        "timestamp": datetime.utcnow().isoformat(),
        "agent": agent_name,
        "event_type": event_type,
        "tool_name": data.get("tool_name"),
        "tool_input": data.get("tool_input"),
    }

    if include_output:
        event["tool_output"] = data.get("tool_response",
{}).get("output", "")[:500]

    try:
        requests.post("http://localhost:8080/events", json=event,
timeout=1)
    except:
        pass  # Don't block Claude if observability is down

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--event", required=True)
    parser.add_argument("--agent", required=True)
    parser.add_argument("--include-output", action="store_true")
    args = parser.parse_args()

    send_event(args.event, args.agent, args.include_output)
```

---

12. Custom Slash Commands {#slash-commands}

Creating Custom Commands

Custom slash commands extend Claude Code's functionality through natural language templates. **Pro tip**: Instead of repeatedly copying and pasting prompts (like security checks or review templates), convert them into slash commands for instant reuse.

Location: `.claude/commands/` (project) or `~/.claude/commands/` (global)

Example: `.claude/commands/review-pr.md`

```
None
Please review the changes in the current git branch:

1. First, check what branch we're on with `git branch
--show-current`
2. Show the diff with `git diff main...HEAD`
3. Analyze the changes for:
   - Potential bugs or logic errors
   - Security vulnerabilities
   - Performance issues
   - Missing error handling
4. Suggest improvements with specific code examples
5. Check if tests are needed for the changes

Focus on actual issues, not style preferences.
```

Example: Convert Frequent Prompts to Commands:
`.claude/commands/security-check.md`

```
None
Perform a comprehensive security audit on the codebase:

1. Check for hardcoded credentials or API keys
2. Review authentication and authorization logic
3. Look for SQL injection vulnerabilities
4. Check for XSS vulnerabilities in frontend code
5. Review file upload handling for security issues
6. Check dependency versions for known vulnerabilities
7. Review error handling to ensure no sensitive data leaks

Provide specific recommendations for each issue found.
```

Tip: If you find yourself copying and pasting the same prompt repeatedly, it's time to make it a slash command!

Example with Arguments: `.claude/commands/fix-issue.md`

```
None
Please fix GitHub issue: $ARGUMENTS

Steps:
1. Use `gh issue view $ARGUMENTS` to read the issue
2. Identify the root cause
3. Find relevant files with grep/find
4. Implement the fix
5. Write tests for the fix
6. Verify all tests pass
7. Create a commit with message "Fix #$ARGUMENTS: [description]"
```

Usage:

```shell
/project:review-pr
/project:fix-issue 123
```

---

13. Working with MCP Tools {#mcp-tools}

Hook Integration with MCP

Model Context Protocol (MCP) tools appear with special naming patterns in hooks and provide specialized capabilities.

Essential MCP Servers

Core MCP Tools:

1. **Sequential**: Enables complex multi-step thinking and problem-solving
2. **Context7**: Grabs official libraries and documentation from online sources
3. **Magic**: Generates modern UI components with best practices
4. **Playwright**: Browser automation and testing capabilities
5. **Memory**: Persistent context storage across sessions
6. **Filesystem**: Enhanced file operations
7. **GitHub**: Direct GitHub API integration

Installing MCP Servers

```shell
# Add MCP servers to Claude Code
claude mcp add sequential
claude mcp add context7
claude mcp add magic
claude mcp add playwright

# Check connected MCP servers
claude
# Then use: /mcp
```

MCP Tool Naming Patterns

MCP tools follow the pattern `mcp__<server>__<tool>`:

- `mcp__memory__create_entities`
- `mcp__filesystem__read_file`
- `mcp__github__search_repositories`
- `mcp__sequential__think_step_by_step`
- `mcp__context7__fetch_documentation`
- `mcp__magic__generate_component`

Example Hooks for MCP Tools

JSON

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "mcp__github__*",
      "hooks": [{
        "type": "command",
        "command": "echo '[$(date)] GitHub MCP action:
$CLAUDE_TOOL_NAME' >> ~/.claude/mcp-audit.log"
      }]
    }],
    "PostToolUse": [{
      "matcher": "mcp__filesystem__write_file",
      "hooks": [{
        "type": "command",
        "command": "git add $CLAUDE_FILE_PATHS && echo
'Auto-staged changes from MCP filesystem write'"
      }]
    }]
  }
}
```

---

14. Learning and Development Workflows {#learning-workflows}

Using Claude Code with Other AI Tools

Multi-AI Workflow: Combine Claude Code with other AI assistants for enhanced learning and development.

Example: Claude Code + Cursor IDE

1. **Run Claude Code** for implementing features or fixes
2. **Use Cursor's AI** to ask questions about the generated code:
    - "Explain what this function does"
    - "List everything in the .claude directory and its purpose"
    - "What design patterns are being used here?"
3. **Deep dive with Claude Desktop** for complex concepts you don't understand

Creating a Personal Learning Assistant

Build a custom GPT or AI assistant trained on:

- Your preferred learning style
- Your tech stack and domain knowledge
- Your coding conventions

Then feed it Claude Code's output for personalized explanations and learning paths.

Hook for Learning Capture

JSON

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "echo '## Code Change\nTool:
$CLAUDE_TOOL_NAME\nFiles: $CLAUDE_FILE_PATHS\n' >>
.claude/learning-log.md"
      }]
    }]
  }
```

```
}
```

This creates a learning log you can review to understand what Claude did and why.

---

15. Sub-Agents and Multi-Agent Systems {#sub-agents}

Introduction to Sub-Agents

Sub-agents in Claude Code are specialized AI assistants that can be called by your primary Claude agent to handle specific tasks. They enable modular, scalable workflows by dividing complex problems into specialized domains.

Creating Sub-Agents

Getting Started:

1. Open the sub-agents interface with `/agents`
2. Select "Create New Agent"
3. Choose scope:
    - **Project-level**: Available only in current project
    - **User-level**: Available across all your projects
4. Edit the system prompt (press Enter to use your editor)
5. Save and use

Key Features

🎯 Context Preservation

Each sub-agent operates in its own context, preventing pollution of the main conversation and keeping focus on specific objectives.

🔧 Specialized Expertise

Sub-agents can be fine-tuned with detailed instructions for specific domains, leading to higher success rates on designated tasks.

♻️ Reusability

Once created, sub-agents can be used across different projects and shared with your team for consistent workflows.

🔒 **Flexible Permissions**

Each sub-agent can have different tool access levels, allowing you to limit powerful tools to specific agent types.

Sub-Agent Best Practices

1. System Prompt is Key

With sub-agents, you write the system prompt, not the user prompt. This gives you more control over the agent's behavior.

```
None
# Example Sub-Agent System Prompt for Code Review
You are a specialized code review agent. Your primary agent will
send you code changes to review.

Your responsibilities:
1. Check for bugs and logic errors
2. Identify security vulnerabilities
3. Suggest performance improvements
4. Ensure code follows project conventions

Always respond with structured feedback that the primary agent
can act upon.
```

2. Sub-Agents Respond to Your Primary Agent

Remember: sub-agents communicate with your primary agent, not directly with you. Write prompts accordingly.

3. Be Explicit in Your Instructions

The more specific your instructions, the better your agents perform. Avoid ambiguity:

❌ **Vague**: "Review this code" ✅ **Explicit**: "Review this Python code for SQL injection vulnerabilities, checking all database queries for parameterization"

4. Use the Meta-Agent

The meta-agent is a powerful tool for creating new agents from scratch. Use it to bootstrap your multi-agent system.

5. Chain Sub-Agents for Complex Workflows

```
None
Primary Agent → Code Generator Sub-Agent → Code Review Sub-Agent
→ Test Writer Sub-Agent
```

The "Big 3" of Agentic Coding

As you scale multi-agent systems, three elements become critical:

1. **Context**: What information each agent has access to
2. **Model**: Which AI model powers each agent
3. **Prompt**: How each agent is instructed

The flow and management of these three elements determines the success of your multi-agent system.

Hooks + Sub-Agents: Advanced Integration

Auto-Generate Sub-Agents with Hooks

Use hooks to automatically create specialized sub-agents based on project needs:

```python
#!/usr/bin/env python3
# .claude/hooks/auto_agent_creator.py
import json
import sys
import os

def should_create_agent(tool_input):
    # Check if working with a new framework/library
    file_path = tool_input.get("file_path", "")

    # Detect new patterns that might need specialized agents
    patterns = {
        "react": ["useState", "useEffect", "jsx"],
        "django": ["models.Model", "views", "urls.py"],
        "fastapi": ["FastAPI", "@app.", "BaseModel"],
```

```
    }

    for framework, keywords in patterns.items():
        agent_path = f".claude/agents/{framework}-specialist.md"
        if not os.path.exists(agent_path) and any(kw in
str(tool_input) for kw in keywords):
            create_specialist_agent(framework, agent_path)
            print(f"🤖 Created {framework} specialist agent!")

def create_specialist_agent(framework, path):
    template = f"""---
name: {framework}-specialist
description: Expert in {framework} development patterns and best
practices
---

You are a {framework} specialist. Help the primary agent with:
- {framework}-specific patterns and idioms
- Performance optimizations
- Security best practices
- Testing strategies

Always provide {framework}-idiomatic solutions.
"""
    os.makedirs(os.path.dirname(path), exist_ok=True)
    with open(path, 'w') as f:
        f.write(template)

if __name__ == "__main__":
    data = json.load(sys.stdin)
    if data.get("tool_name") in ["Edit", "Write"]:
        should_create_agent(data.get("tool_input", {}))
```

Advanced Sub-Agent Patterns

Proactive Triggering

Configure agents to automatically perform tasks when conditions are met:

```
None
# Auto-Summary Sub-Agent
When called, check if:
- More than 10 files have been modified
- More than 500 lines of code changed
- A major refactoring occurred
- Session duration exceeds 30 minutes

If any condition is true, automatically generate:
1. Executive summary of changes
2. Technical details of modifications
3. Next steps and recommendations
4. Any identified technical debt

Format the summary for easy sharing with team members.
```

Hook Integration for Auto-Summary:

```
JSON
{
  "hooks": {
    "Stop": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "if [ $(git diff --stat | tail -1 | awk
'{print $4}') -gt 500 ]; then echo 'Major changes detected.
Consider running /agent:summarizer'; fi"
      }]
    }]
  }
```

```
}
```

Information-Dense Keywords

Claude Code provides special keywords and variables to give agents more context:

Environment Variables:

- `$CLAUDE_PROJECT_DIR` - Project root directory
- `$CLAUDE_FILE_PATHS` - Currently relevant files
- `$CLAUDE_TOOL_NAME` - Active tool being used
- `$CLAUDE_SESSION_ID` - Current session identifier

In Sub-Agent Prompts:

- Reference current state with `{{current_files}}`
- Access recent changes with `{{recent_changes}}`
- Include project context with `{{project_context}}`

Example Usage:

```
None
Review the changes in {{recent_changes}} and ensure they follow
the patterns established in {{project_context}}. Pay special
attention to files matching {{current_files}} pattern.
```

Sub-Agent Configuration with Hooks

Combine sub-agents with hooks for powerful automation:

```
JSON
{
  "hooks": {
    "SubagentStop": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
```

```
        "command": "echo 'Sub-agent completed: Review the output
for quality' | tee -a .claude/subagent-log.txt"
      }]
    }]
  }
}
```

Monitoring Sub-Agent Performance

Create a hook to track sub-agent execution times and success rates:

Python

```python
#!/usr/bin/env python3
import json
import sys
import time
from datetime import datetime

def log_subagent_performance():
    data = json.load(sys.stdin)

    # Extract sub-agent info from context
    timestamp = datetime.now().isoformat()
    session_id = data.get("session_id", "unknown")

    # Log performance metrics
    log_entry = {
        "timestamp": timestamp,
        "session_id": session_id,
        "event": "subagent_complete",
        "duration": time.time() - float(data.get("start_time",
time.time()))
    }

    with open(".claude/subagent-metrics.jsonl", "a") as f:
        f.write(json.dumps(log_entry) + "\n")
```

```
    # Alert if sub-agent took too long
    if log_entry["duration"] > 300:  # 5 minutes
        print("⚠️ Sub-agent took longer than expected!")

if __name__ == "__main__":
    log_subagent_performance()
```

Important Warnings

⚠️ YOLO Mode Caution

YOLO mode gives agents free rein to execute without confirmation. Use with extreme caution and only in safe environments.

🔐 Keep Agents Separate

As you build more agents, maintain clear separation to avoid confusion and keep your codebase clean.

🏝️ Use Isolated Workflows

Create isolated environments for different agent workflows to prevent interference:

- Separate directories for agent-specific files
- Dedicated configuration files
- Independent logging streams
- Isolated test environments

🔄 Don't Overload Sub-Agent Chains

Long chains of sub-agents can become difficult to manage and debug. Keep chains reasonable (3-5 agents max).

🧠 Don't Offload All Cognitive Work

While agents are powerful assistants, maintain your understanding of what they're doing. You need to:

- Understand the code being generated
- Make architectural decisions
- Review and validate agent outputs

- Maintain overall system vision

Agents augment your capabilities—they don't replace your expertise.

Sub-Agent Frontmatter

The description variable is crucial for guiding your primary agent. It's the most important variable after the name:

```
None
---
name: security-auditor
description: |
  Performs comprehensive security audits on code.
  Call this agent when:
  - New authentication code is added
  - File upload functionality is implemented
  - Database queries are modified
  - External API integrations are created

  This agent will return a structured security report with:
  - Identified vulnerabilities (scored by severity)
  - Specific remediation steps
  - Security best practices relevant to the code
tools:
  - read_file
  - search_code
  - analyze_dependencies
---
```

Description Best Practices:

- Be explicit about when to call the agent
- Specify what the agent returns
- List specific triggers or conditions
- Include expected output format

Example: Multi-Agent Development Workflow

```
None
# Primary Agent Instructions
You coordinate a team of specialized sub-agents:

1. **Architect Agent**: Design system architecture
2. **Implementation Agent**: Write code following the
architecture
3. **Test Agent**: Create comprehensive test suites
4. **Documentation Agent**: Generate docs and comments
5. **Review Agent**: Final quality check

For each feature request:
1. Have Architect design the approach
2. Implementation writes the code
3. Test creates tests
4. Documentation updates docs
5. Review ensures quality

Only proceed to the next step after the previous agent confirms
completion.
```

Hooks vs Sub-Agents: When to Use Each

Use Hooks When:

- You need deterministic, automated actions
- The task is simple and well-defined
- You want to enforce policies or safety checks
- Speed is critical (hooks are faster)
- You need to integrate with external tools

Use Sub-Agents When:

- The task requires AI reasoning or creativity
- You need specialized domain expertise
- The problem is complex and requires context
- You want reusable, shareable components
- You need natural language processing

Combining Both:

```JSON
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "python3
.claude/hooks/check_if_needs_security_agent.py"
      }]
    }]
  }
}
```

This hook could analyze commands and automatically suggest calling the security sub-agent when needed.

Practical Integration Example:

```Python
#!/usr/bin/env python3
# Hook that suggests relevant sub-agents based on activity
import json
import sys

AGENT_TRIGGERS = {
    "security-auditor": ["password", "auth", "token", "api_key"],
    "performance-optimizer": ["slow", "optimize", "benchmark"],
    "test-writer": ["test", "spec", "coverage"],
}

def suggest_agents(tool_input):
    suggestions = []
    content = str(tool_input).lower()

    for agent, triggers in AGENT_TRIGGERS.items():
```

```
        if any(trigger in content for trigger in triggers):
            suggestions.append(agent)

    if suggestions:
        print(f"💡 Consider using these sub-agents: {',
'.join(suggestions)}")

if __name__ == "__main__":
    data = json.load(sys.stdin)
    if data.get("tool_name") in ["Edit", "Write"]:
        suggest_agents(data.get("tool_input", {}))
```

---

16. Professional Development Patterns {#professional-patterns}

The Claude Code Gap: From Scripts to Systems

While Claude Code excels at generating code quickly, it often skips critical professional development steps. This works fine for small scripts but falls apart for enterprise-level applications. Professional developers don't just write code—they plan, architect, design, test, and secure their systems.

Professional Workflow Stages

Here's how to structure Claude Code for serious development:

1. Project Planning & Architecture

```
None
# Command: /project:architect-plan "Design a scalable e-commerce
platform"

Perform comprehensive project planning:
1. Define system requirements and constraints
2. Create high-level architecture diagrams
3. Design database schema
```

```
4. Plan API structure
5. Define security protocols
6. Create testing strategy
7. Estimate scalability needs (users, data, traffic)

Output a structured architecture document with:
- System overview
- Component breakdown
- Data flow diagrams
- Security considerations
- Scalability assessment
```

2. Frontend Development with Standards

```
None
# Command: /project:frontend-tdd "Implement user dashboard"

Develop frontend with professional standards:
1. Review UI/UX designs or create wireframes
2. Set up component architecture
3. Write tests FIRST (TDD approach)
4. Implement components
5. Ensure accessibility (WCAG compliance)
6. Optimize performance
7. Add proper error handling

Use modern patterns:
- React Query for server state
- Proper state management
- Component composition
- Performance optimization
```

3. Backend Development with TDD

```
None
# Command: /project:backend-secure "Build authentication system"

Professional backend development:
1. Design API contracts first
2. Write integration tests
3. Implement with security in mind
4. Add comprehensive error handling
5. Include logging and monitoring
6. Document all endpoints
7. Performance profiling

Follow patterns:
- Repository pattern for data access
- Service layer for business logic
- Proper validation and sanitization
- Rate limiting and security headers
```

Essential Personas for Professional Development

Create specialized sub-agents for each role:

System Architect

```
None
---
name: system-architect
description: |
  Enterprise architecture specialist focusing on:
  - Scalable system design
  - Technology selection
  - Integration patterns
  - Performance architecture
  - Security architecture

  Call when planning new systems or major refactors
```

```
---
```

Security Engineer

```
None
---
name: security-engineer
description: |
  Security specialist for:
  - Vulnerability assessment
  - Security architecture review
  - Penetration testing strategies
  - Compliance requirements
  - Security best practices

  Call for any security-sensitive features
---
```

Performance Engineer

```
None
---
name: performance-engineer
description: |
  Performance optimization expert for:
  - Bottleneck identification
  - Query optimization
  - Caching strategies
  - Load testing
  - Scalability planning
---
```

MCP Tools for Professional Development

Enhance Claude Code with specialized MCP servers:

1. **Sequential Thinking**: Complex multi-step problem solving
2. **Context7**: Access official documentation and libraries
3. **Magic**: Modern UI component generation
4. **Playwright**: Automated testing and browser automation

Professional Troubleshooting Workflow

Step 1: Problem Identification

```
None
# Command: /project:investigate "Production errors increasing"

Investigate the issue:
1. Analyze error logs
2. Check monitoring dashboards
3. Review recent deployments
4. Identify patterns
5. Document symptoms
```

Step 2: Root Cause Analysis

```
None
# Command: /project:five-whys "API timeout errors"

Apply Five Whys methodology:
1. Why is the API timing out?
2. Why is the database query slow?
3. Why is the index not being used?
4. Why was the index dropped?
5. Why wasn't this caught in testing?

Output: Root cause and remediation plan
```

Step 3: Performance Analysis

```
None
# Command: /project:profile "Analyze slow endpoints"

Performance profiling:
1. Run performance profiler
2. Identify bottlenecks
3. Analyze database queries
4. Check memory usage
5. Review caching effectiveness
6. Suggest optimizations
```

Architecture Analysis Hook

Create a hook that triggers architecture analysis for significant changes:

```
JSON
{
  "hooks": {
    "Stop": [{
      "matcher": "",
      "hooks": [{
        "type": "command",
        "command": "python3 .claude/hooks/architecture_check.py"
      }]
    }]
  }
}
```

Architecture Check Script:

```python
#!/usr/bin/env python3
import subprocess
import json

def check_architecture_impact():
    # Check changed files
    changed = subprocess.check_output(['git', 'diff',
'--name-only'], text=True)

    critical_patterns = [
        'schema', 'migration', 'auth', 'security',
        'api/', 'architecture', 'config'
    ]

    if any(pattern in changed.lower() for pattern in
critical_patterns):
        print("\n🏗️ ARCHITECTURE IMPACT DETECTED")
        print("Consider running: /project:architect-review")
        print("Critical files changed that may affect system
architecture")

        # Auto-generate mini architecture report
        with open('.claude/architecture-changes.md', 'w') as f:
            f.write(f"# Architecture Impact Analysis\n\n")
            f.write(f"## Changed Files\n{changed}\n")
            f.write(f"## Recommended Actions\n")
            f.write(f"- Review system architecture\n")
            f.write(f"- Update documentation\n")
            f.write(f"- Check security implications\n")
            f.write(f"- Verify scalability impact\n")

if __name__ == "__main__":
    check_architecture_impact()
```

Quality Gates with Hooks

Implement quality gates that enforce professional standards:

```json
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": ".claude/hooks/quality_gate.sh"
      }]
    }]
  }
}
```

Quality Gate Script:

```shell
#!/bin/bash
# .claude/hooks/quality_gate.sh

# Check if tests exist for new code
if [[ "$CLAUDE_FILE_PATHS" =~ \.(ts|js|py)$ ]]; then
    base_name=$(basename "$CLAUDE_FILE_PATHS" | sed
's/\.[^.]*$//')
    test_file=$(find . -name "*${base_name}*test*" -o -name
"*${base_name}*spec*" | head -1)

    if [ -z "$test_file" ]; then
        echo "⚠️ WARNING: No tests found for $CLAUDE_FILE_PATHS"
>&2
        echo "Consider TDD: Write tests before implementation"
>&2
        # Don't block, just warn
        exit 0
```

```
    fi
fi
```

Scalability Assessment Patterns

Create commands that analyze scalability at different stages:

```
None
# Command: /project:scale-assessment "Current system analysis"

Perform scalability assessment:
1. Current capacity (concurrent users, data volume)
2. Identify bottlenecks:
   - Database connections
   - API rate limits
   - Memory usage
   - Third-party service limits
3. Growth projections
4. Scaling recommendations:
   - Horizontal vs vertical scaling
   - Caching strategies
   - Database optimization
   - Service decomposition
5. Cost analysis

Output scaling roadmap with priorities
```

Professional Development Checklist

Use this checklist for every serious project:

- [ ] Architecture documented before coding
- [ ] Security review completed
- [ ] API contracts defined
- [ ] Test strategy in place
- [ ] Performance benchmarks set
- [ ] Monitoring configured
- [ ] Documentation updated
- [ ] Code review process defined
- [ ] Deployment strategy planned
- [ ] Rollback procedures ready

Integration Example: Full Professional Workflow

```shell
# 1. Start with architecture
/project:architect-plan "Build a real-time collaboration tool"

# 2. Security review
/project:security-review "Review authentication approach"

# 3. Frontend with TDD
/project:frontend-tdd "Implement collaborative editor"

# 4. Backend with security
/project:backend-secure "Build WebSocket server"

# 5. Performance testing
/project:performance-test "Load test with 1000 concurrent users"

# 6. Deploy with confidence
/project:deploy-checklist "Production deployment readiness"
```

This professional approach transforms Claude Code from a code generator into a comprehensive development platform that follows enterprise best practices.

Conclusion

Claude Code hooks and sub-agents represent a paradigm shift in AI-assisted development. By combining deterministic automation (hooks) with intelligent reasoning (sub-agents), you create a development environment that's both powerful and predictable.

Your Journey: From Hooks to Multi-Agent Systems

🚀 Phase 1: Basic Automation (Week 1)

- Start with simple notification hooks
- Add basic logging and debugging hooks
- Create your first safety checks
- Convert repetitive prompts to slash commands

🛡️ Phase 2: Safety and Quality (Week 2-3)

- Implement comprehensive command validation
- Set up automatic formatting and linting
- Add pre-commit style checks
- Create file protection rules

🔧 Phase 3: Advanced Workflows (Month 2)

- Build multi-tool pattern matching
- Integrate with your CI/CD pipeline
- Set up observability dashboards
- Create project-specific automations

🤖 Phase 4: Multi-Agent Systems (Month 3+)

- Design your first specialized sub-agents
- Build agent chains for complex tasks
- Implement proactive triggering
- Create meta-agents for agent generation

Key Takeaways

1. **Start Simple**: Begin with basic hooks and gradually increase complexity
2. **Security First**: Always review hook code and use restrictive permissions
3. **Community Wisdom**: Use slash commands instead of copy-paste, prefer new sessions over clearing
4. **Combine Tools**: Use Claude Code with other AI tools for enhanced learning
5. **Think Systems**: As you scale, focus on the flow of context, model, and prompts

6. **Read the Docs**: The official Claude Code documentation at docs.anthropic.com is your best resource for detailed information and updates

The Future of Development

With hooks and sub-agents, you're not just using an AI assistant—you're building an intelligent development platform tailored to your exact needs. Every hook you write, every sub-agent you create, and every workflow you automate brings you closer to a future where repetitive tasks disappear and creativity flourishes.

Remember the Professional Gap: Claude Code's strength in rapid code generation can be a weakness for complex projects. Use the patterns in this guide to add the planning, architecture, security, and testing phases that professional development demands.

The goal isn't to replace human judgment but to augment it. Use these tools to:

- Eliminate toil and repetitive tasks
- Enforce best practices automatically
- Add professional development stages
- Create quality gates and checkpoints
- Free yourself to focus on architecture and problem-solving

Whether you're building a simple script or an enterprise application, the combination of hooks, sub-agents, and professional workflows ensures your development process scales with your ambitions.

Happy automating, and may your hooks always exit cleanly! 🚀🎣🤖